Symbolic and Distributed Artificial Intelligence

Course Notes

Contents

I Introduction **7**

1 Introduction to Artificial Intelligence **7**

1.1 About the course . . . 7
1.2 What is intelligence? What is AI? . . . 7
1.2.1 The word and a classic definition . . . 8
1.2.2 Many kinds of intelligence . . . 8
1.2.3 Thinking, fast and slow: System 1 and System 2 . . . 9
1.2.4 Artificial intelligence and intelligent agents . . . 10
1.3 A short history of AI . . . 11
1.3.1 Before computers . . . 12
1.3.2 1950: the Turing test . . . 13
1.3.3 1956: the Dartmouth workshop and the founding fathers . . . 14
1.3.4 1966–1972: Shakey the robot . . . 14
1.3.5 1970–1972: Prolog . . . 15
1.3.6 The AI winters and the Lisp machines . . . 15
1.3.7 1997–2016: from chess to Go, the data-driven era . . . 16
1.4 Some more recent achievements . . . 17
1.4.1 Games, medicine and emergent behaviour . . . 17
1.4.2 Self-driving cars: the SAE levels of driving automation . . . 18
1.4.3 Generative AI, protein folding and agentic AI . . . 18
1.5 But... risks, and the answers to them . . . 19
1.5.1 Things that went wrong, or might . . . 19
1.5.2 And hence: principles and guidelines . . . 21
1.6 Symbolic vs. sub-symbolic AI, and what this course is about . . . 21
1.6.1 Two families of AI . . . 21
1.6.2 Thinking fast and slow in AI . . . 22
1.6.3 What SDAI is about . . . 23

II Logic **25**

2 Propositional logic **25**
2.1 What is a logic? . . . 25
2.2 Propositions and connectives . . . 26
2.2.1 Propositions . . . 26
2.2.2 The connectives . . . 27
2.2.3 Building complex formulas . . . 29
2.3 Valuations, tautologies and consistency . . . 30
2.3.1 Valuations: giving a truth value to a formula . . . 30
2.3.2 Tautologies, consistency, inconsistency . . . 31
2.3.3 Logical equivalence and logical consequence . . . 32
2.4 Proof theory: natural deduction . . . 33
2.4.1 What is a proof? . . . 33
2.4.2 Basic rules without subproofs . . . 34
2.4.3 Rules with subproofs . . . 38
2.4.4 Derived rules . . . 45

3 First-order logic **52**
3.1 Why not propositional logic? . . . 53
3.2 Syntax of FOL: terms and predicates . . . 54
3.2.1 Terms: constants, variables, functions . . . 54
3.2.2 Predicates . . . 55
3.3 Quantifiers . . . 57
3.3.1 The three quantifiers and their syntax . . . 58
3.3.2 Free and bound variables . . . 59
3.3.3 Quantifiers as big conjunctions and disjunctions . . . 59
3.4 Decidability . . . 61
3.5 Proof theory for first-order logic . . . 62
3.5.1 Frege (Hilbert-style) proofs . . . 62
3.5.2 Provability and soundness . . . 64
3.5.3 Sequent calculus LK . . . 65
3.5.4 LKe: adding equality . . . 66
3.5.5 Natural deduction rules for quantifiers . . . 66

III Agents and Multi-Agent Systems **67**

4 Agents and multi-agent systems **68**
4.1 What is an agent? . . . 68
4.1.1 The weak definition . . . 68
4.1.2 The strong definition . . . 70
4.2 What is a multi-agent system? . . . 70
4.3 Three ways of using MAS . . . 72
4.3.1 MAS as a software engineering metaphor . . . 72

4.3.2 Agent-based development . . . 72
4.3.3 MAS for simulation . . . 74
4.4 A short history of agents and MAS . . . 74
4.4.1 From artificial intelligence . . . 75
4.4.2 From object-oriented programming . . . 75
4.4.3 From human-computer interaction . . . 76
4.4.4 Agents as intentional systems . . . 77
4.4.5 Two ancestors of MAS: Contract Net and Actors . . . 78

5 Agent architectures **80**
5.1 Agents and their environments . . . 80
5.1.1 Three classic definitions of agent . . . 80
5.1.2 Kinds of environments . . . 81
5.2 What is an agent architecture? . . . 82
5.3 Reactive architectures . . . 84
5.3.1 Basic ideas . . . 84
5.3.2 Rodney Brooks and the subsumption architecture . . . 86
5.3.3 Steels' Mars explorer . . . 87
5.3.4 Advantages and limitations . . . 88
5.4 Deliberative and hybrid architectures . . . 90
5.4.1 Deliberative architectures . . . 90
5.4.2 Hybrid architectures . . . 91

6 Engineering multi-agent systems **92**
6.1 Recap and the SteaMas scenario . . . 92
6.2 Agent-oriented software engineering . . . 93
6.3 Research directions in AOSE . . . 95
6.3.1 1. Agent-oriented architectures . . . 95
6.3.2 2. AOSE methodologies and notation techniques . . . 96
6.3.3 3. Agent-oriented programming languages . . . 98
6.3.4 4. MAS infrastructures and toolkits . . . 99
6.3.5 5. Agent-based modelling and simulation . . . 100
6.3.6 6. Integration of symbolic and sub-symbolic approaches . . . 100
6.4 Forth into the SteaMas future: designing the MAS . . . 100
6.5 Conclusions: where are all the agents? . . . 103
6.6 Engineering safe MAS: formal and runtime verification . . . 104
6.6.1 Safe and reliable autonomous systems . . . 104
6.6.2 Verification techniques . . . 105
6.6.3 Runtime verification . . . 106
6.6.4 Conclusions . . . 107

IV Prolog **108**

7 Prolog **108**

7.1 Prolog basics . . . 109
7.1.1 Predicates and facts . . . 109
7.1.2 Making queries . . . 110
7.1.3 Variables and multiple directions . . . 110
7.1.4 Compound queries and constraints . . . 111
7.1.5 Rules: clause bodies . . . 113
7.1.6 Recursive predicates . . . 114
7.1.7 Disjunction . . . 115
7.1.8 Terms and variables . . . 115
7.1.9 Unification . . . 116
7.1.10 Recursive structures: numbers as terms . . . 118
7.2 Lists . . . 119
7.2.1 List syntax . . . 119
7.2.2 List membership . . . 120
7.2.3 List concatenation: append . . . 121
7.3 Nondeterminism, arithmetic, all solutions . . . 122
7.3.1 Arithmetic in Prolog: a short primer . . . 122
7.3.2 Managing nondeterminism . . . 123
7.3.3 Fibonacci numbers . . . 124
7.3.4 Lists and arithmetic . . . 125
7.3.5 All-solutions predicates . . . 127
7.4 Terms, comparison, program database . . . 128
7.4.1 Handling terms in general: univ, functor, arg . . . 128
7.4.2 Term comparison . . . 130
7.4.3 Variable and type testing . . . 131
7.4.4 Program access and manipulation . . . 133
7.5 Logical foundations . . . 134
7.5.1 Clauses and Horn clauses . . . 134
7.5.2 SLD resolution . . . 136
7.5.3 SLD trees . . . 137

V Cognitive Agents, Communication and Platforms 139

8 Cognitive agents: BDI and AgentSpeak(L) 139
8.1 From logic-based agents to BDI . . . 139
8.2 The BDI architecture . . . 140
8.3 The AgentSpeak(L) language . . . 142
8.3.1 Ingredients . . . 142
8.3.2 Syntax . . . 143
8.3.3 Informal semantics: the reasoning cycle . . . 144
8.4 Example: Jelly-fish busters . . . 145

9 Agent communication 147
9.1 Communication as action . . . 148

9.2 FIPA ACL: the agent communication language . . . 150
9.2.1 Message structure . . . 150
9.2.2 Communicative acts . . . 151
9.2.3 Anatomy of a communicative act: accept-proposal . . . 152
9.3 Ontologies, content languages, interaction protocols . . . 154
9.3.1 The FIPA Ontology Agent . . . 154
9.3.2 Content languages . . . 155
9.3.3 Interaction protocols . . . 155

10 JADE: a platform for multi-agent systems 157
10.1 JADE agents . . . 157
10.1.1 The HelloWorld agent and AIDs . . . 158
10.1.2 Arguments and termination . . . 158
10.2 Behaviours and the execution model . . . 159
10.3 Communication and yellow pages . . . 161
10.3.1 The communication model . . . 161
10.3.2 Waiting for messages: block() . . . 162
10.3.3 The yellow pages service . . . 163

11 Agent-based modelling and simulation 165

List of whiteboards

1 Terms and atoms (written after the FOL lectures) 56
2 Modus Ponens: Socrates is mortal (written during the FOL proof-theory lecture) 63
3 Reactive architectures: pros and cons (written during the agent architectures lecture) 89
4 Methodology questions (written during the SteaMas discussion) 101
5 MAS architecture and role level (written during the SteaMas discussion) . . 102
6 Development: how? (written during the SteaMas discussion) 103
7 From cognitive agents to Jason (written during the BDI / Jason lectures) . 141

The professor wrote these on the whiteboard during the lectures. They are exam material: each one is redrawn in a clean form, and the original photo is shown next to it.

Part I

Introduction

1 Introduction to Artificial Intelligence

[Slides · Thinking machines: 1–97]

Before studying logic, Prolog and agents, the course opens with a big-picture tour: what *intelligence* is, how *artificial* intelligence (AI) was born and grew, what it can do today, why people are worried about it, and where this course sits in the AI landscape. This chapter covers:

1. what intelligence is (several definitions, Kahneman's System 1 and System 2);
2. what artificial intelligence is, and why it is defined through *agents*;
3. a short history of AI, from Aristotle to self-driving cars;
4. recent achievements (AlphaStar, AlphaFold, generative and agentic AI);
5. the risks, and the principles and rules written to control them;
6. symbolic vs. sub-symbolic AI, and why this course is about the *symbolic* and *distributed* side.

1.1 About the course

[Slides · Thinking machines: 1–3]

The course **Symbolic and Distributed Artificial Intelligence** (SDAI, sometimes written S&DAI) is taught by prof. Viviana Mascardi at the University of Genoa. All material (slides, announcements, lab instructions) is published on the course page of the university e-learning platform (AulaWeb).

The course has two souls, already visible in its name:

- **Symbolic**: intelligence obtained by manipulating *symbols* with explicit rules: logic (Part II of these notes) and the logic programming language **Prolog** (Part IV);
- **Distributed**: intelligence spread over many autonomous software **agents** that interact: agents, multi-agent systems, their architectures, their communication and the platforms used to build them (Parts III and V).

Section 1.6 explains, at the end of this chapter, why these two things belong together.

1.2 What is intelligence? What is AI?

[Slides · Thinking machines: 4–12]

To build *artificial* intelligence we should first agree on what intelligence is. There is no single accepted answer; the slides show several complementary views.

1.2.1 The word and a classic definition

[Slides · Thinking machines: 4–6]

The word comes from Latin *intelligĕre*: "to understand", literally "to read inside" (*intus legere*) or "to read between" (*inter legere*), that is, to read beyond the surface of things.

Definition — Intelligence (Mainstream Science on Intelligence, 1994)

Intelligence is a very general mental capability that, among other things, involves the ability to **reason**, **plan**, **solve problems**, **think abstractly**, **comprehend complex ideas**, **learn quickly** and **learn from experience**. It is not merely book-learning, a narrow academic skill, or test-taking smarts. Rather, it reflects a broader and deeper capability for comprehending our surroundings: "catching on", "making sense" of things, or "figuring out" what to do.

The definition was published in the *Wall Street Journal* in December 1994 and signed by 52 scientists. The professor remarks that the statement also contains claims she does not feel comfortable with: it is a reference point, not an undisputed truth. The seven abilities in bold (highlighted in red on slide 6) are worth remembering: each of them has become a branch of AI.

Ability in the definition	AI field that tries to reproduce it
reason	logic, automated reasoning, Prolog (this course)
plan	automated planning (e.g. the STRIPS planner of Shakey)
solve problems	search algorithms, constraint solving
think abstractly	knowledge representation, ontologies
comprehend complex ideas	natural language understanding
learn quickly / from experience	machine learning, reinforcement learning

Table 1: The abilities in the 1994 definition and the AI research areas that study them.

1.2.2 Many kinds of intelligence

[Slides · Thinking machines: 7–8]

Intelligence is not a single number. Two influential theories:

Definition — Multiple intelligences (H. Gardner, *Frames of Mind*, 1983)

Gardner proposed that humans have several distinct intelligences:

1. musical-rhythmic and harmonic;
2. visual-spatial;
3. verbal-linguistic;
4. logical-mathematical;
5. bodily-kinesthetic;
6. interpersonal (understanding others);
7. intrapersonal (understanding oneself);
8. naturalistic (recognising and classifying nature);
9. existential (asking deep questions about life).

Definition — Emotional intelligence (D. Goleman, 1995)

Emotional intelligence (EI) is the capability of individuals to recognise their own emotions and those of others, discern between different feelings and label them appropriately, use emotional information to guide thinking and behaviour, and manage and/or adjust emotions to adapt to environments or achieve one's goal(s).

Intuition — Why does an AI course care?

If intelligence has many faces, then "artificial intelligence" can also mean many different things. A chess program shows logical-mathematical intelligence; a robot that walks on rough ground shows bodily-kinesthetic intelligence; a chatbot that adapts its tone to an angry customer imitates a bit of emotional and interpersonal intelligence. A *multi-agent system* (the main topic of this course) needs *interpersonal* (social) intelligence: agents must understand and negotiate with each other.

1.2.3 Thinking, fast and slow: System 1 and System 2

[Slides · Thinking machines: 9–11]

In the book *Thinking, Fast and Slow* (2011), the psychologist and Nobel prize winner Daniel Kahneman describes the mind as two cooperating "systems".

Definition — System 1 and System 2

- **System 1**: fast, automatic, frequent, emotional, stereotypic, unconscious.
- **System 2**: slow, effortful, infrequent, logical, calculating, conscious.

System 1 can… (in order of complexity)	**System 2 is needed to…**
determine that an object is farther than another	prepare for the start of a sprint
localise the source of a sound	direct your attention to the clowns at the circus
complete the phrase "war and …"	look for the woman with grey hair
show disgust at a gruesome image	try to recognise a sound
solve $2 + 2 =$?	sustain a faster-than-normal walking rate
read text on a billboard	judge if a behaviour is appropriate in a social setting
drive a car on an empty road	count the number of A's in a text
think of a good chess move (if you are a chess master)	give someone your phone number
understand simple sentences	park into a tight parking space
	compare the price/quality ratio of two washing machines
	determine the validity of a complex logical reasoning

Table 2: Kahneman's examples (slides 10–11). Notice that the same task can move from System 2 to System 1 with practice: a chess master "sees" a good move.

Worked example — the bat and the ball

[Not in the slides: added material]

A bat and a ball cost 1.10 euro in total. The bat costs 1 euro more than the ball. How much does the ball cost? System 1 shouts "10 cents!". System 2, if you wake it up, checks: if the ball costs x, the bat costs $x + 1$, so

$$\begin{aligned} x + (x + 1) &= 1.10 \\ 2x &= 0.10 \\ x &= 0.05. \end{aligned}$$

The ball costs 5 cents (check: $0.05 + 1.05 = 1.10$). Fast intuition is cheap but can be wrong; slow reasoning is expensive but verifiable. Keep this in mind: it is exactly the difference between *sub-symbolic* and *symbolic* AI (Section 1.6).

1.2.4 Artificial intelligence and intelligent agents

[Slides · Thinking machines: 12]

Definition — Artificial intelligence

Artificial intelligence (AI) is intelligence demonstrated by machines, unlike the natural intelligence displayed by humans and animals. The leading AI textbook, *Artificial Intelligence: A Modern Approach* (S. Russell and P. Norvig, 4th edition),

defines the field as the study of **intelligent agents**: any device that **perceives** its environment and **takes actions** that maximise its chance of successfully achieving its **goals**.

This definition is the bridge to the whole course: AI is not defined by *how* the device works inside (neural network, logic rules, ...) but by its behaviour as an **agent**, a loop of perceiving and acting towards goals.

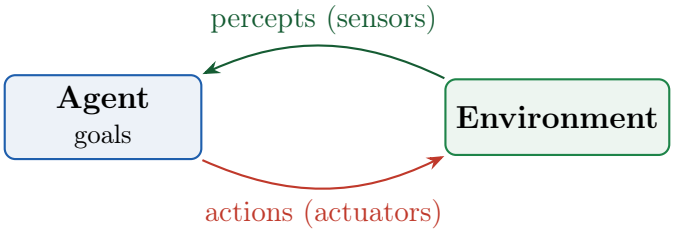


Figure 1: An intelligent agent: it perceives the environment and acts on it to reach its goals. Part III of these notes is entirely about this picture.

Why this matters in practice

Everyday intelligent agents: a robot vacuum cleaner (perceives obstacles and dirt, acts by moving and sucking, goal: a clean floor); a smart thermostat (perceives temperature and your habits, acts on the boiler, goal: comfort at minimum cost); a trading bot (perceives prices, acts by buying/selling, goal: profit); a navigation app (perceives traffic, acts by suggesting a route, goal: get you there fast).

1.3 A short history of AI

[Slides · Thinking machines: 13–67]

The slides build a timeline one event at a time (slides 16–67). The complete timeline is redrawn in Figure 2; the sub-sections below explain each step.

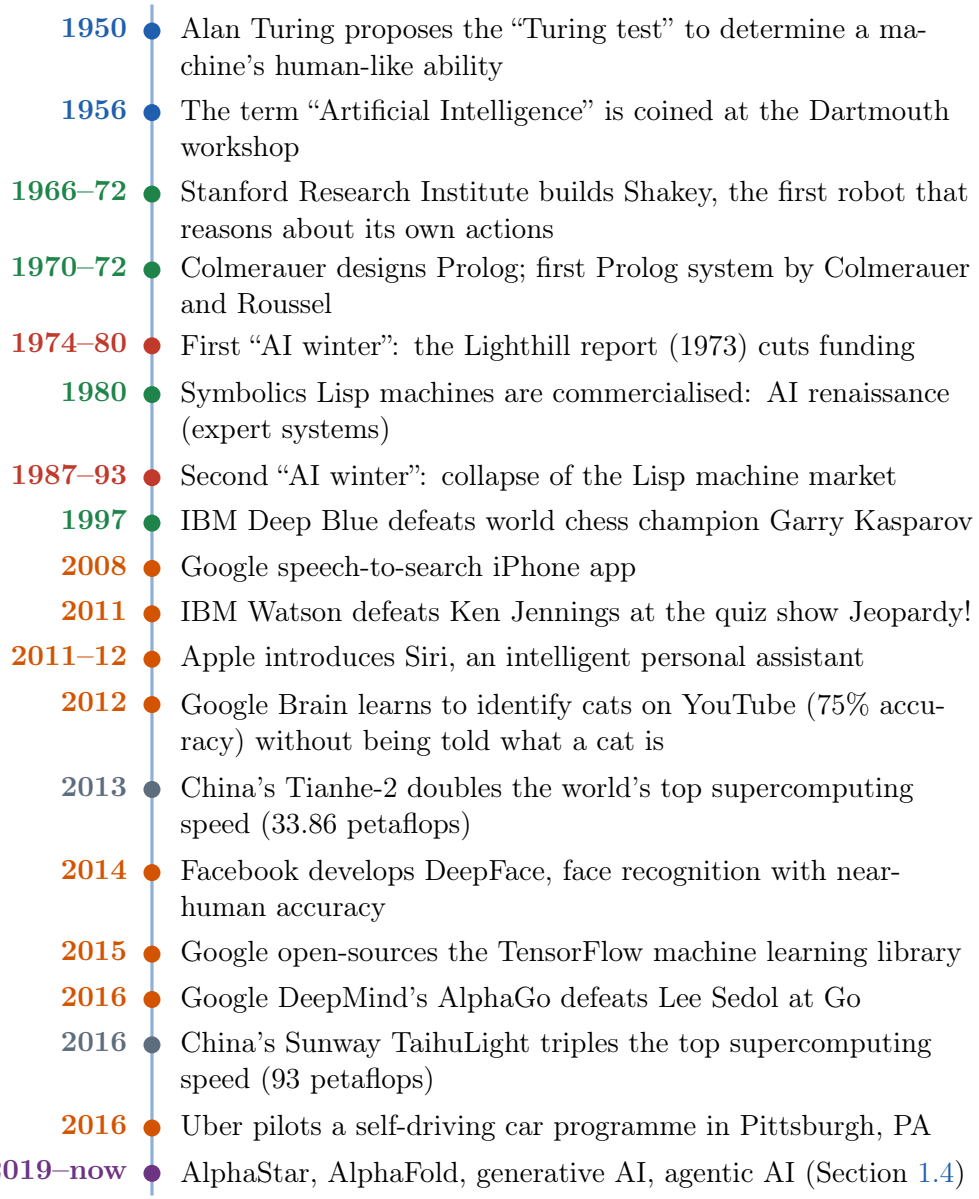


Figure 2: The AI timeline of the slides (slide 67, extended). Colours: foundations, symbolic AI successes, AI winters, data-driven / neural successes, computing power, today. Notice the shift: until the 1990s the big results are symbolic (reasoning, planning, chess search); after 2008 they are mostly learned from data.

1.3.1 Before computers

[Slides · Thinking machines: 13–15]

The slides show three portraits, without text. Here is why each person is on the "before" list (the explanations are added):

- **Aristotle** (384–322 BC) invented the first system of formal logic, the *syllogism*: "All men are mortal; Socrates is a man; therefore Socrates is mortal". The very

same example will come back in first-order logic (Whiteboard 2) and in Prolog.

- **Gottfried Wilhelm von Leibniz** (1646–1716) dreamed of a universal symbolic language (*characteristica universalis*) and a "calculus of reasoning" (*calculus ratiocinator*), so that disputes could be settled by saying "let us calculate!". He also studied binary arithmetic and built a mechanical calculator. This is the dream of symbolic AI: reasoning as computation.
- **Mary Shelley** (1797–1851) wrote *Frankenstein* (1818), the story of an artificial being created by a scientist, and of the consequences of creating it: the first great story about the risks of artificial creatures (see Section 1.5).

1.3.2 1950: the Turing test

[Slides · Thinking machines: 16–23]

In 1950 Alan Turing published the paper *Computing Machinery and Intelligence* in the philosophy journal *Mind*. Instead of answering the vague question "Can machines think?", he replaced it with a game that can actually be played, the **imitation game**, now called the **Turing test**.

Definition — Turing test

A human interrogator chats (in writing) with two hidden partners: a human and an AI system. By asking questions, the interrogator must decide which one is the machine. If the interrogator cannot reliably tell them apart, the machine passes the test.

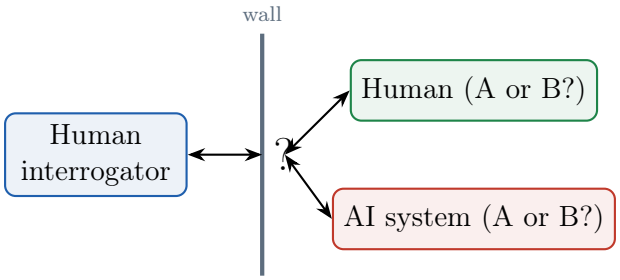


Figure 3: The imitation game (slide 22): the interrogator only sees written answers and must guess which partner is the machine.

Intuition — Behaviour instead of definitions

Turing's trick is to judge intelligence by *observable behaviour*, not by what happens inside. The agent definition of AI (Section 1.2.4) has the same spirit. The slides add a comic ("Turing test 2208") where the human fails: "the machine realised immediately that it was me being the dumb human". Today large language models make the classic Turing test much less decisive than it looked in 1950.

An open parenthesis (slides 19–21). What does it exactly mean to "think", to "perceive", to be "conscious of being alive"? The slides link to the neuroscientist Anil

Seth (TED talk *Your brain hallucinates your conscious reality*): according to him, perception is a *controlled hallucination*, the brain's best guess about the causes of its sensory signals, constantly corrected by the senses. If we cannot define these words precisely for humans, judging them in machines is even harder. (Closed parenthesis.)

1.3.3 1956: the Dartmouth workshop and the founding fathers

[Slides · Thinking machines: 24–33]

In summer 1956 a workshop at Dartmouth College (USA) coined the term **"Artificial Intelligence"**. Its proposal contained the founding conjecture of the field:

> *"Every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it."*

The slides list the participants (the "founding fathers of AI") and, step by step, what each of them is remembered for:

Person	**Known for**
John McCarthy	the *Advice Taker* (a program reasoning with logical sentences, which inspired Prolog), the Lisp language, ALGOL
Marvin Minsky	artificial neural networks, the *society of mind* (the mind as a society of simple interacting agents: an ancestor of multi-agent systems!)
Claude Shannon	information theory, cryptography, natural language processing
Ray Solomonoff	machine learning, prediction, Bayesian frameworks
Allen Newell & Herbert Simon	the *Logic Theory Machine* (proved theorems of *Principia Mathematica*) and the *General Problem Solver*
Arthur Samuel	search, alpha-beta pruning, machine learning (a checkers program that learned to play better than its author)
Oliver Selfridge	Pandemonium theory (pattern recognition by many competing "demons")
Nathaniel Rochester	IBM, designer of the IBM 701 computer
Trenchard More	natural deduction (the proof system of Section 2.4)

Table 3: The founding fathers of AI and their contributions (slides 26–33).

Intuition — Symbolic and sub-symbolic, from day one

Already at Dartmouth both souls of AI were present: logic and symbols (McCarthy, Newell and Simon, More) and learning and neural networks (Minsky, Solomonoff, Samuel). The rest of the history is a pendulum between the two.

1.3.4 1966–1972: Shakey the robot

[Slides · Thinking machines: 34–36]

Shakey was the first general-purpose mobile robot able to *reason about its own actions*. Other robots had to be instructed on each individual step; Shakey could analyse a command and break it down into basic chunks by itself. The project combined robotics, computer vision and natural language processing, and was the first to meld *logical reasoning* and *physical action*. It was developed at the AI Center of the Stanford Research Institute (now SRI International) from about 1966 to 1972 (project manager Charles Rosen), programmed mainly in LISP, and its planning component was the **STRIPS** planner. As the first logical, **goal-based agent**, Shakey lived in a limited world (a few rooms with boxes).

Worked example — what "breaking a command into chunks" means

[Not in the slides: added material]

Command: "push the box from room A to room B". A planner like STRIPS knows actions with preconditions and effects, e.g. `goto(Place)`, `push(Box, From, To)`, and finds the sequence: `goto(room_A)` → `goto(box)` → `push(box, room_A, door)` → `push(box, door, room_B)`. Your navigation app and a warehouse robot do the same kind of planning today. Shakey is a deliberative agent in the sense of Section 5.

1.3.5 1970–1972: Prolog

[Slides · Thinking machines: 37]

In 1970 Alain Colmerauer designed the **Prolog** language (from the French *programmation en logique*); the first Prolog system was implemented by Colmerauer and Philippe Roussel in 1972 in Marseille. Prolog is the language of Part IV of these notes: a program is a set of logical facts and rules, and running it means proving a query.

1.3.6 The AI winters and the Lisp machines

[Slides · Thinking machines: 38–43]

Early AI promised a lot. When the promises were not kept, funding collapsed: these periods are called **AI winters**.

- **First AI winter (1974–1980).** The **Lighthill report** (*Artificial Intelligence: A General Survey*, Sir James Lighthill, 1973, for the British Science Research Council) gave a very pessimistic evaluation: "in no part of the field have discoveries made so far produced the major impact that was then promised". It formed the basis for the British government's decision to end support for AI research in all but three universities (Edinburgh, Sussex, Essex). Its main technical criticism: AI had failed to address the **combinatorial explosion** that appears when problems are taken from toy worlds to real-world domains.
- **AI renaissance (1980).** Symbolics commercialises **Lisp machines**: computers with hardware support for Lisp operations (data type testing, CDR coding,

incremental garbage collection), running large Lisp programs very efficiently, with operating systems written entirely in Lisp. They powered the boom of *expert systems* (rule-based programs encoding the knowledge of human experts).

- **Second AI winter (1987–1993).** With the microcomputer revolution, cheap desktop PCs soon ran Lisp programs even faster than Lisp machines, without special hardware. The Lisp machine market collapsed: most makers went out of business by the early 1990s, and AI lost credibility again.

Worked example — combinatorial explosion in numbers

[Not in the slides: added material]

Chess has about 35 legal moves per position. Looking d moves ahead means examining about 35^d positions:

$$35^2 = 1\,225, \qquad 35^4 = 1\,500\,625, \qquad 35^6 \approx 1.8 \times 10^9, \qquad 35^{10} \approx 2.8 \times 10^{15}.$$

Each extra level multiplies the work by 35. An algorithm that works on a toy problem can become hopeless on a slightly bigger one: this is what the Lighthill report meant. Deep Blue (1997) beat it on chess with special hardware plus pruning (alpha-beta, Samuel's legacy).

1.3.7 1997–2016: from chess to Go, the data-driven era

[Slides · Thinking machines: 44–67]

- **1997**: IBM **Deep Blue** defeats the world chess champion Garry Kasparov (symbolic AI: search plus hand-written evaluation).
- **2008**: Google releases a **speech-to-search** iPhone app: you speak, the recording is sent to Google's servers, and the results come back in seconds.
- **2011**: IBM **Watson** defeats the champion Ken Jennings at the quiz show *Jeopardy!* (natural language question answering).
- **2011/2012**: Apple introduces **Siri**, an intelligent personal assistant.
- **2012**: after seeing 10 million images from YouTube videos in three days, the 16,000-computer network of **Google Brain** (one billion connections) began to recognise cats, even though it had never been taught what a cat looked like (unsupervised learning, 75% accuracy).
- **2013**: China's Tianhe-2 doubles the world's top supercomputing speed (33.86 petaflops); **2016**: Sunway TaihuLight triples it (93 petaflops). Raw computing power is one of the reasons for the success of deep learning.
- **2014**: Facebook's **DeepFace** identifies human faces in images: a nine-layer neural network with over 120 million connection weights, trained on four million images uploaded by Facebook users, said to be 97% accurate (vs. 85% for the FBI's Next Generation Identification system). It uses a 3D model to virtually rotate faces so that they look at the camera.
- **2015**: Google open-sources **TensorFlow**, a library for numerical computation with data flow graphs, now widely used for machine learning.

- **2016**: Google DeepMind's **AlphaGo** defeats Lee Sedol at Go, a game whose search space is far too large for Deep Blue-style brute force.
- **2016**: Uber pilots a **self-driving car** programme in Pittsburgh, PA.

Watch out — Two kinds of "winning"

Deep Blue and AlphaGo both beat a world champion, but in very different ways. Deep Blue *searched* millions of positions per second with rules written by humans (System 2 style, symbolic). AlphaGo *learned* an intuition for good moves from millions of games with neural networks, combined with a search (System 1 + System 2). Do not file both under "AI wins at games" without noting the difference: it is the theme of Section 1.6.

1.4 Some more recent achievements

[Slides · Thinking machines: 68–79]

After 2016 the pace accelerated. The slides show a selection of results; for each one, here is what it is and why it matters.

1.4.1 Games, medicine and emergent behaviour

[Slides · Thinking machines: 68–71]

- **AlphaStar** (DeepMind, 2019). StarCraft II is a real-time strategy video game: many units, partial information (you do not see the whole map), real-time decisions. AlphaStar won its historic series against professional players (announced on January 24, 2019, after matches recorded in December 2018; e.g. 5–0 against the pro player MaNa).
- **Lung cancer screening** (*Nature Medicine*, May 2019). A three-dimensional deep learning model analyses low-dose chest CT scans end to end and detects lung cancer with performance comparable to, or better than, expert radiologists.
- **Emergent tool use from multi-agent autocurricula** (OpenAI, ICLR 2020). Agents play *hide-and-seek* in a simulated world with movable boxes and ramps. Nobody tells them how to use the objects; just by competing, the teams discover a sequence of strategies: hiders build shelters with boxes, seekers use ramps to jump in, hiders then lock the ramps away, seekers "surf" on boxes, and so on.

Why this matters in practice — A multi-agent system in action

The hide-and-seek experiment is a *multi-agent system*: several autonomous agents, in a shared environment, competing (hiders vs. seekers) and cooperating (inside a team). Complex behaviour *emerges* from simple goals and interaction, with no central controller. Keep this example in mind for Part III (agents and MAS) and for agent-based simulation (Section 11).

1.4.2 Self-driving cars: the SAE levels of driving automation

[Slides · Thinking machines: 72–74]

How "autonomous" is an autonomous car? The SAE J3016 standard defines six levels.

Level	What the human must do	What the features do	Examples
0	You **are driving**, even if features are on; constantly supervise	warnings and momentary assistance	automatic emergency braking, blind-spot warning, lane departure warning
1	same as level 0	steering **or** brake/acceleration support	lane centering **or** adaptive cruise control
2	same as level 0	steering **and** brake/acceleration support	lane centering **and** adaptive cruise control together
3	You are **not driving** while features are engaged, but you must take over when the system requests it	drives the vehicle under limited conditions	traffic jam chauffeur
4	You are not driving; you will never be required to take over	drives under limited conditions, never needs you	local driverless taxi; pedals and steering wheel may not be installed
5	You are not driving	drives everywhere, in all conditions	as level 4, but anywhere

Table 4: SAE levels of driving automation (slide 72). Blue rows: *driver support* features (the human drives). Green rows: *automated driving* features (the system drives).

The slides note (2024 news) that a **Level 3** car can now be bought in the US: the Mercedes-Benz system works only on a small selection of roads, at limited speed and only in some states, and its wide adoption will require regulators to catch up.

Intuition — Levels of autonomy, again later

The question "how much can the system decide by itself, and when must a human step in?" is the question of **autonomy**, the defining property of agents (Section 4). The jump from level 2 to level 3 is exactly the moment in which responsibility moves from the human to the agent.

1.4.3 Generative AI, protein folding and agentic AI

[Slides · Thinking machines: 75–79]

On slide 75 the professor asked ChatGPT itself: "which are the three most impressive achievements in the AI research field in the last three years (short answer)". The answer:

1. **Generative AI**: models such as GPT-4 and DALL-E generate coherent text, images and creative content.
2. **AI in protein folding**: DeepMind's AlphaFold accurately predicts protein structures, solving a 50-year-old problem in biology.
3. **AI alignment and safety research**: progress in interpretability, alignment and safety (e.g. by OpenAI and Anthropic) to control the behaviour of advanced AI.

Definition — Protein folding problem

A protein is a chain of amino acids. The **protein folding problem** is the question of how a protein's amino acid sequence dictates its three-dimensional atomic structure. It first emerged around 1960, with the first atomic-resolution protein structures.

AlphaFold2 (DeepMind) predicts the 3D structure of proteins from their amino acid sequence with near-experimental accuracy; more than 200 million predicted structures have been released. Knowing a protein's shape tells biologists how it works, which speeds up drug discovery and protein design (2023 review; PNAS 2024, "AlphaFold two years on").

Finally, slide 79: **agentic AI**, presented as "the next evolution of artificial intelligence". An agentic AI system does not just answer one question: given a goal, it plans, calls tools (search engines, APIs, other programs), observes the results and acts again, autonomously, over many steps.

Why this matters in practice — Old ideas, new engines

Agentic AI is the 2020s name for an idea this course has studied for decades: **autonomous agents** that perceive, decide and act towards goals, and **multi-agent systems** in which several of them coordinate. The difference is the engine: a large language model instead of (or together with) explicit beliefs, goals and plans. Understanding classic agents (BDI, communication protocols, platforms) is what makes it possible to engineer agentic AI systems that are reliable.

1.5 But... risks, and the answers to them

[Slides · Thinking machines: 80–87]

The slides now switch tone ("But..."): AI systems already take or support decisions that affect people, and things can go wrong.

1.5.1 Things that went wrong, or might

[Slides · Thinking machines: 80–84]

Case	What happened	The underlying risk
State v. Loomis (2016–17)	The Wisconsin Supreme Court allowed judges to use an algorithmic risk-assessment score (predicting the risk of re-offending) in sentencing, but required a written warning about its limits. The defendant could not inspect how the proprietary algorithm worked.	opaque decisions about people's freedom; possible bias; no way to contest the reasoning
"Smart courts" in China (2019)	Courts adopt AI assistants (e.g. Shanghai's "System 206") to check evidence and support judges.	automation of justice; who is accountable?
ChatGPT banned in Italy (2023)	The Italian data protection authority temporarily blocked ChatGPT over privacy concerns (personal data used for training, no age checks).	privacy and data protection
AI leaders at the UN Security Council	The heads of OpenAI (Sam Altman) and Anthropic (Dario Amodei) gave separate briefings on AI safety to the UN Security Council.	global safety and security
Incidents with AI agents (slide 84)	News reports about security incidents involving AI platforms and AI agents that were misused or hijacked.	autonomous agents acting in the real world can be attacked or misbehave

Table 5: The cautionary examples of slides 80–84.

Intuition — Why symbolic AI helps here

In State v. Loomis the core problem is that nobody could *see* why the system gave a certain score. A symbolic system (a set of logical rules, a Prolog program, a BDI

agent with explicit plans) can *explain* its conclusions: "I concluded X because of rule R applied to facts F_1 and F_2". Explainability, verifiability and accountability are among the strongest arguments for the symbolic side of AI.

1.5.2 And hence: principles and guidelines

[Slides · Thinking machines: 85–87]

- **Asilomar AI Principles** (Future of Life Institute, 2017): 23 principles developed at the 2017 Asilomar conference, grouped into research issues, ethics and values (e.g. safety, transparency, responsibility, human control, privacy), and longer-term issues.
- **Ethics Guidelines for Trustworthy AI** (European Commission, High-Level Expert Group on AI, 8 April 2019). Trustworthy AI must be *lawful*, *ethical* and *robust*, and should meet seven key requirements (listed below, not in the slides).
- **Calls to slow down** (New York Times, September 2026): top AI leaders, among them the head of Anthropic, argued that capabilities are advancing so quickly that greater safety controls are needed across the industry.

Key result — The seven requirements for trustworthy AI (EU, 2019)

[Not in the slides: added material]

1. human agency and oversight;
2. technical robustness and safety;
3. privacy and data governance;
4. transparency (including explainability);
5. diversity, non-discrimination and fairness;
6. societal and environmental well-being;
7. accountability.

These guidelines were followed by the EU *AI Act* (in force since 2024), which regulates AI systems according to their level of risk.

1.6 Symbolic vs. sub-symbolic AI, and what this course is about

[Slides · Thinking machines: 88–97]

The deck ends by going "back to the first slides": Kahneman's two systems (Section 1.2.3) give the key to read the whole history of AI.

1.6.1 Two families of AI

[Slides · Thinking machines: 89–90]

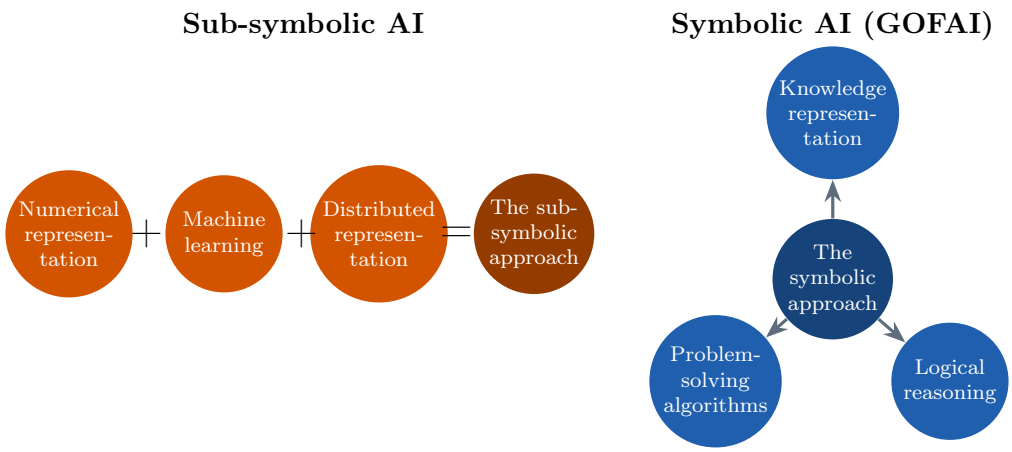


Figure 4: The two families of AI (slides 89–90). Sub-symbolic AI: knowledge is numbers spread over many parameters, learned from data. Symbolic AI: knowledge is explicit symbols, manipulated by reasoning and search.

Definition — Sub-symbolic AI

AI where knowledge is encoded as **numbers** (e.g. the weights of a neural network), **learned** from data (machine learning), and **distributed**: no single number means "cat" or "stop sign"; the meaning is spread over millions of parameters.

Definition — Symbolic AI ("Good Old-Fashioned AI", GOFAI)

AI where knowledge is written explicitly as **symbols** that humans can read (facts, rules, logical formulas, plans), and intelligence is obtained by **knowledge representation**, **logical reasoning** and **problem-solving algorithms** (search, planning).

Worked example — the same knowledge, two ways

Knowledge: "Socrates is a human; all humans are mortal".

- *Symbolic*: two lines of Prolog (Part IV), `human(socrates).` and `mortal(X) :- human(X).` Asking `?- mortal(socrates).` gives `true`, and the system can show the proof: rule + fact.
- *Sub-symbolic*: a language model has read millions of sentences about Socrates and mortality; asked "is Socrates mortal?" it answers "yes" because this is the most likely continuation. The "knowledge" is somewhere in billions of weights and cannot be pointed at.

1.6.2 Thinking fast and slow in AI

[Slides · Thinking machines: 88, 91]

The paper *Thinking Fast and Slow in AI* (G. Booch, F. Rossi and others, AAAI 2021) draws a loose parallel between Kahneman's two systems and the two main lines of work in AI:

	System 1 ≈ machine learning	System 2 ≈ symbolic AI
Style	data-driven, fast, intuitive	knowledge-driven, slow, deliberate
Good at	perception from sensory data (seeing, reading, hearing); building (possibly imprecise and biased) models from data	complex problems: logic, search, optimisation, planning, with explicit knowledge, symbols and high-level concepts
Weak at	common-sense reasoning, basic notions of causality, explanations	noisy perception, learning from raw data, scaling to huge open-ended domains
Example	recognise a pedestrian in a camera image	decide that the car must yield because traffic rules say so

Table 6: The parallel of slide 91 ("Thinking Fast and Slow in AI"), with examples.

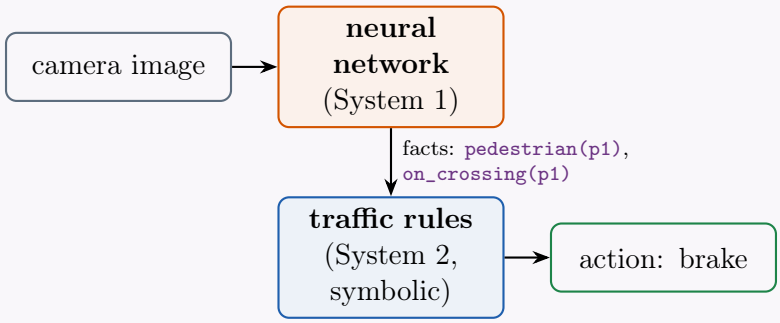
As in humans, the two sets of capabilities are **symbiotic**: this supports **hybrid, neurosymbolic** approaches to AI.

Definition — Neurosymbolic AI

Neurosymbolic AI combines neural networks (sub-symbolic learning) with symbolic representations and reasoning, to obtain systems that are more versatile and more dependable than either alone (slides 92–95: articles in *Communications of the ACM* 2022, IBM Research, the book *Neuro-Symbolic Artificial Intelligence: The State of the Art*, and the journal *Neurosymbolic Artificial Intelligence*, founded in 2023).

Worked example — a neurosymbolic self-driving car

[Not in the slides: added material]



The neural part turns pixels into *symbols* ("there is a pedestrian on the zebra crossing"); the symbolic part applies explicit, auditable rules ("if a pedestrian is on a crossing, then yield"). If the car brakes, we can explain why.

1.6.3 What SDAI is about

[Slides · Thinking machines: 96–97]

Key result — The course in one paragraph (slide 96)

The Symbolic and Distributed AI course is about the **symbolic component of neurosymbolic AI**. It exploits symbolic AI in the context of **distributed AI systems**, where (symbolic, but not only) intelligent software **agents** distributed across different machines operate **autonomously**, to **coordinate/collaborate** in order to achieve some common goal, or to **compete** to access shared resources. A system consisting of these distributed agents, as well as their environment and the organisations they are part of, is named **multi-agent system** (MAS).

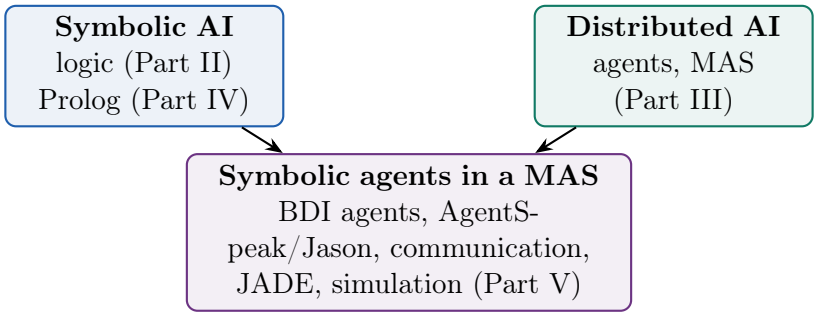


Figure 5: How the parts of these notes fit together.

Welcome on board, and let's have fun with S&DAI!

Cheat sheet — Introduction to AI

- **Intelligence**: general capability to reason, plan, solve problems, think abstractly, comprehend complex ideas, learn quickly and learn from experience. Many kinds exist (Gardner's 9 intelligences, emotional intelligence).
- **System 1** = fast, automatic, intuitive; **System 2** = slow, effortful, logical (Kahneman, 2011). Parallel in AI: System 1 ≈ machine learning, System 2 ≈ symbolic AI (logic, planning, search).
- **AI** (Russell & Norvig): the study of *intelligent agents*, devices that perceive their environment and act to maximise the chance of achieving their goals.
- **Key dates**: 1950 Turing test; 1956 Dartmouth workshop (the term "AI"); 1966–72 Shakey; 1970–72 Prolog; 1974–80 and 1987–93 AI winters; 1997 Deep Blue; 2011 Watson; 2012 Google Brain cats; 2016 AlphaGo; 2019 AlphaStar; 2020+ AlphaFold, generative AI; now agentic AI.
- **Risks**: biased decisions (State v. Loomis), privacy (ChatGPT blocked in Italy), safety and security of autonomous agents. **Responses**: Asilomar principles (2017), EU Ethics guidelines for trustworthy AI (2019).
- **Sub-symbolic AI** = numerical representation + machine learning + distributed representation. **Symbolic AI** (GOFAI) = knowledge representation + logical reasoning + problem-solving algorithms. **Neurosymbolic AI** combines them.
- **SDAI** = the symbolic component of neurosymbolic AI, used inside distributed systems of autonomous agents: a **multi-agent system (MAS)**.

Part II

Logic

2 Propositional logic

[Slides · Propositional logic: 1–21; Proof theory (prop.): 1–11]

Symbolic AI represents knowledge with *symbols* and reasons with *rules*. The mathematical tool that makes this precise is **logic**. This chapter studies the simplest logic, **propositional logic**, in three steps that will repeat for every logic of the course:

1. **syntax**: which strings of symbols are legal formulas;
2. **semantics**: what a formula means, i.e. when it is true (truth tables, valuations, tautologies);
3. **proof theory**: how to derive new formulas from old ones by applying rules mechanically (natural deduction).

The next chapter (first-order logic) repeats the same three steps for a richer language, and Prolog (Part IV) is a programming language built on a fragment of it.

2.1 What is a logic?

[Slides · Propositional logic: 1–3]

When most people say "logic" they mean either *propositional logic* or *first-order predicate logic*. In fact the word is much broader: philosophers, computer scientists and mathematicians have studied literally hundreds of logics (temporal logics, modal logics, logics of belief, ...). What do they have in common?

Definition — Logic (slide 2)

Any formal system can be considered a logic if it has:

- a well-defined **syntax**: it defines the syntactically acceptable objects of the language, called **well-formed formulae** (wff), or simply **formulae**;
- a well-defined **semantics**: it associates each formula with a **meaning**;
- a well-defined **proof theory**: it is concerned with **manipulating** formulae according to certain rules.

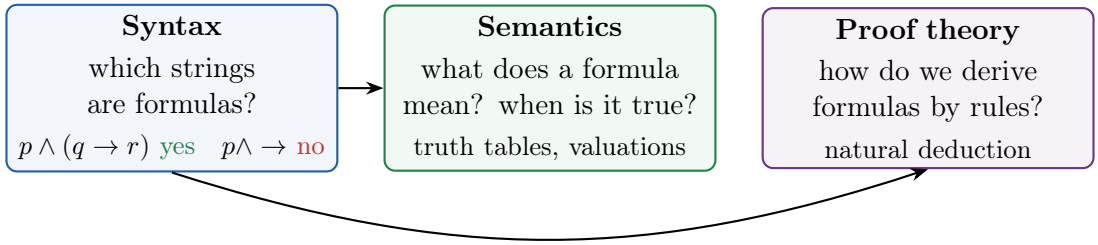


Figure 6: The three ingredients of every logic. Semantics and proof theory are two independent ways of talking about formulas: one with *truth*, one with *rules*. A good proof theory derives exactly the formulas that are semantically valid.

Intuition — A programming-language analogy

Think of a programming language. Its *syntax* says which programs compile (`x = 3 +;` does not). Its *semantics* says what a compiling program does when it runs. A *proof theory* is like a set of safe refactoring rules: you can transform the code step by step without ever running it, and the rules guarantee that the meaning is preserved. Logic works the same way, with formulas instead of programs and truth instead of behaviour.

Why this matters in practice

Every symbolic AI system makes these three choices. A Prolog program (Part IV) has a syntax (facts and rules), a semantics (what is logically implied by the program) and a proof procedure (SLD resolution) that the interpreter runs. A BDI agent (Part V) stores its beliefs as logical formulas and uses the same machinery to decide which plans apply.

2.2 Propositions and connectives

[Slides · Propositional logic: 4–18]

2.2.1 Propositions

[Slides · Propositional logic: 4–6]

Propositional logic is the simplest, and most abstract, logic we can study. Its building blocks are propositions.

Definition — Proposition (slide 4)

A **proposition** is a statement that can be either true or false; it must be one or the other, and it cannot be both.

Propositions	Not propositions
the reactor is on	are you going out somewhere? (a question)
the wing-flaps are up	$2 + 3$ (a number, not a statement)
John Major is prime minister	close the door! (an order; added)

Table 7: Examples from slide 4. (John Major was the UK prime minister when the slides were written; the sentence is still a proposition today: it is simply false.)

Key result — The "it is true that" test (slide 5)

To check whether a statement is a proposition, prefix it with *"It is true that . . . "* and see whether the result makes grammatical sense. "It is true that the reactor is on" makes sense; "It is true that are you going out somewhere?" does not.

Definition — Atomic proposition (slide 5)

An **atomic proposition** is one whose truth or falsity does not depend on the truth or falsity of any other proposition. Intuitively, atomic propositions are the *smallest* propositions: they cannot be split into smaller propositions. All the propositions above are atomic.

Instead of writing propositions in full, we abbreviate them with **propositional variables**: by convention lower-case letters $p, q, r, \ldots$. When we do so we must say what each letter means ("let p be *John Major is prime minister*"). An alternative is to use self-explaining names such as *reactor_is_on*, so that the meaning of the variable is obvious. (Prolog and AgentSpeak use exactly this second style: `reactor_on`, `door_open`.)

2.2.2 The connectives

[Slides · Propositional logic: 7–17]

Atomic propositions alone are "pretty boring". **Connectives** combine propositions into bigger, *compound* propositions. Different books use different symbols; these notes use the first column, the slides use a mix of all of them.

Symbol	Read	Name	Other notations	Arity
$\wedge$	and	conjunction	& .	binary
$\vee$	or	disjunction	| +	binary
$\neg$	not	negation	$\sim$	unary
$\rightarrow$	implies, if. . . then	implication (conditional)	$\Rightarrow$ $\supset$	binary
$\leftrightarrow$	if and only if (iff)	biconditional	$\Leftrightarrow$ $\equiv$	binary

Table 8: The connectives (slide 7). The propositional-logic slides write $\Rightarrow$ and $\Leftrightarrow$; the proof-theory slides write &, $\sim$ and $\rightarrow$. They all mean the same thing.

How do we define the meaning of a connective? With a **truth table**: a table listing the value of the compound formula for *every possible combination* of values of its components.

Key result — Number of rows of a truth table (slide 9)

If a formula contains n different atomic propositions, its truth table has 2^n rows, because each proposition can take one of 2 values (T or F) independently of the others: $\underbrace{2 \times 2 \times \cdots \times 2}_{n \text{ times}} = 2^n$. For example $n = 2 \Rightarrow 4$ rows, $n = 3 \Rightarrow 8$ rows,

$n = 10 \Rightarrow 1024$ rows.

The five truth tables. Here are the definitions of the slides, all in one table (write T for true and F for false):

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \to q$	$p \leftrightarrow q$
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

Table 9: Truth tables of all connectives (slides 9, 11, 13, 16, 17). Orange: the two rows where $p \to q$ is true "by default" because p is false. Red: the *only* row where an implication is false.

In words:

- **And** (slides 8–9): $p \wedge q$ is true iff *both* p and q are true.
- **Or** (slides 10–11): $p \vee q$ is true iff p is true, or q is true, or both. This is the **inclusive** or.
- **Implies** (slides 12–14): $p \to q$ is true iff *either p is false or q is true*. In $p \to q$, p is the **antecedent** and q the **consequent**.
- **Iff** (slides 15–16): $p \leftrightarrow q$ is true iff p and q are both true or both false (they have the same value).
- **Not** (slide 17): $\neg p$ is true iff p is false. It is the only *unary* connective (one argument); the others are *binary*.

Watch out — Inclusive vs. exclusive "or"

In everyday language "tea or coffee?" usually means *one of the two, not both* (exclusive or). The logical $\vee$ is **inclusive**: $p \vee q$ is true also when both are true. The exclusive or (XOR) can be written $(p \vee q) \wedge \neg(p \wedge q)$, or $\neg(p \leftrightarrow q)$.

Understanding implication. The slides warn that $\to$ is the hardest connective to understand, and yet extremely important. The rule of thumb of slide 14: *$p \to q$ means "if p is true, then q is true". If p is false, we don't care about q, and by default $p \to q$ evaluates to T.*

Intuition — Implication as a promise

[Not in the slides: added material]

A parent says: "*If you pass the exam (p), I will buy you a bike (q)*". When has the parent *broken the promise*?

pass (p)	bike (q)	what happened	promise kept? ($p \to q$)
F	F	failed, no bike	T (no reason to complain)
F	T	failed, bike anyway	T (generous, but not a lie)
T	F	passed, no bike	**F (the promise is broken!)**
T	T	passed, got the bike	T

The promise is only broken in one case: the condition happened and the result did not. When the condition did not happen, the promise was simply never "tested", so it cannot be false; in two-valued logic, "not false" means true.

Watch out — Implication is not causality

$p \to q$ only says "it is not the case that p is true and q false". It does not say that p *causes* q. "If $2 + 2 = 5$ then the Moon is made of cheese" is *true* (false antecedent). "If Rome is in Italy then $2 + 2 = 4$" is *true* too, although the two facts have nothing to do with each other. This is called **material implication**.

Why this matters in practice

Every `if` in a program, every rule in a firewall or a smart-home app is an implication. Smart-home rule: $motion \wedge night \to light_on$. During the day the rule is "not tested" and the light can be on or off without violating it, exactly as the truth table says. A *logic-based agent* (Section 8) is little more than a collection of such rules plus a reasoning engine.

2.2.3 Building complex formulas

[Slides · Propositional logic: 18]

Connectives can be nested as deeply as we want, and parentheses disambiguate. If p, q, r, s, t are atomic propositions, then these are formulae:

$$p \wedge q \to r \qquad p \wedge (q \to r) \qquad (p \wedge (q \to r)) \vee s \qquad ((p \wedge (q \to r)) \vee s) \wedge t$$

whereas none of these is a formula:

$$p \wedge \qquad p \wedge q) \qquad p\neg$$

($p\wedge$ misses the right operand; $p\wedge q)$ has an unbalanced parenthesis; in $p\neg$ the negation is on the wrong side).

Definition — Well-formed formulas of propositional logic

[Not in the slides: added material]

The set of formulas is defined *recursively*:

1. every atomic proposition $(p, q, r, \ldots)$ is a formula;
2. if φ is a formula, then $\neg\varphi$ is a formula;
3. if φ and ψ are formulas, then $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, $(\varphi \to \psi)$, $(\varphi \leftrightarrow \psi)$ are

formulas;
4. nothing else is a formula.

Key result — Precedence (how to read a formula without all its parentheses)

[Not in the slides: added material]

From the strongest (binds first) to the weakest: $\neg$, then $\wedge$, then $\vee$, then $\rightarrow$, then $\leftrightarrow$. So $p \wedge q \rightarrow r$ means $(p \wedge q) \rightarrow r$, and $\neg p \vee q$ means $(\neg p) \vee q$. Implication associates to the right: $p \rightarrow q \rightarrow r$ means $p \rightarrow (q \rightarrow r)$. When in doubt, write the parentheses.

Worked example — the structure (parse tree) of a formula

The formula $((p \wedge (q \rightarrow r)) \vee s) \wedge t$ from slide 18 has this tree: the **main connective** (the last one applied) is at the root.

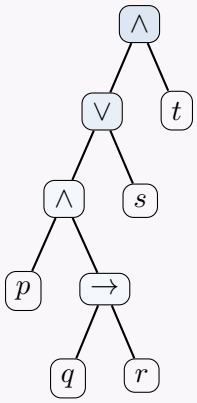


Figure 7: Parse tree: the leaves are atomic propositions, internal nodes are connectives. Truth values are computed bottom-up, from the leaves to the root.

2.3 Valuations, tautologies and consistency

[Slides · Propositional logic: 19–21]

2.3.1 Valuations: giving a truth value to a formula

[Slides · Propositional logic: 19–20]

Given a formula, can you tell if it is true or not? **No**: you usually need to know the truth values of its atomic propositions first. Is $p \vee q$ true? It depends on p and q.

Definition — Valuation (slide 19)

A **valuation** is a function v that assigns a truth value (T or F) to each atomic (primitive) proposition. In a programming language it would be a function `Val(p: AtomicProp): BOOLEAN`. Given a valuation, we can compute for *any* formula

whether it is true or false, by applying the truth tables bottom-up.

Worked example — evaluating $(p \vee q) \to r$ (slide 20)

Let $v(p) = \mathsf{F}$, $v(q) = \mathsf{T}$, $v(r) = \mathsf{F}$. Then

$$\begin{aligned} v\big((p \vee q) \to r\big) &= (v(p) \vee v(q)) \to v(r) && \triangleleft \text{ replace each letter by its value} \\ &= (\mathsf{F} \vee \mathsf{T}) \to \mathsf{F} \\ &= \mathsf{T} \to \mathsf{F} && \triangleleft\ \mathsf{F} \vee \mathsf{T} = \mathsf{T} \text{ (table of } \vee) \\ &= \mathsf{F} && \triangleleft\ \mathsf{T} \to \mathsf{F} = \mathsf{F} \text{ (table of } \to) \end{aligned}$$

So under this valuation the formula is **false**: the antecedent $p \vee q$ holds but the consequent r does not, which is the only way to falsify an implication.

Key result — Rows = valuations

Each row of a truth table corresponds to exactly one valuation of the atomic propositions of the formula. A truth table therefore shows the value of the formula under *all* possible valuations (2^n of them).

2.3.2 Tautologies, consistency, inconsistency

[Slides · Propositional logic: 21]

Some formulas have interesting properties that do not depend on the particular valuation.

Definition — Tautology, consistent, inconsistent (slide 21)

1. A formula is a **tautology** iff it is true under *every* valuation.
2. A formula is **consistent** iff it is true under *at least one* valuation (also called *satisfiable*).
3. A formula is **inconsistent** iff it is *not* made true under any valuation (also called *contradiction* or *unsatisfiable*).

Since each row of a truth table is a valuation, truth tables decide these properties: look at the last column. All T ⇒ tautology; at least one T ⇒ consistent; no T ⇒ inconsistent.

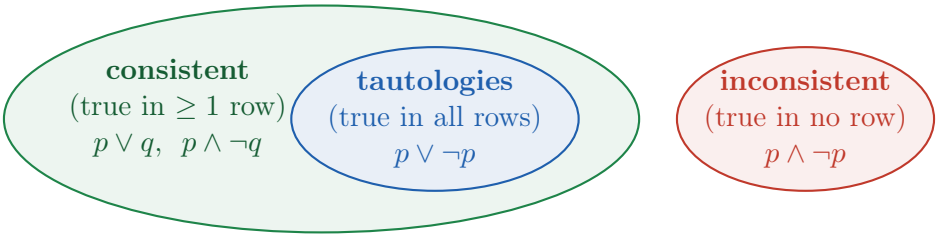


Figure 8: Every formula is either consistent or inconsistent. Tautologies are a special case of consistent formulas. The negation of a tautology is inconsistent and vice versa.

Worked example — one formula of each kind

p	$p \vee \neg p$	$p \wedge \neg p$	$p \to \neg p$
F	T	F	T
T	T	F	F
	tautology	inconsistent	consistent, not a tautology

$p \vee \neg p$ ("it rains or it does not rain") is always true: it carries no information about the world. $p \wedge \neg p$ can never be true.

Worked example — modus ponens is a tautology, affirming the consequent is not

[Not in the slides: added material]

Consider the two formulas $A = (p \wedge (p \to q)) \to q$ and $B = (q \wedge (p \to q)) \to p$. Full truth table, one column per sub-formula:

p	q	$p \to q$	$p \wedge (p \to q)$	A	$q \wedge (p \to q)$	B
F	F	T	F	T	F	T
F	T	T	F	T	T	F
T	F	F	F	T	F	T
T	T	T	T	T	T	T

A is a **tautology**: whenever p and $p \to q$ are both true, q is true. This is the reasoning pattern *modus ponens* (Section 2.4). B is **not**: in row 2 (p false, q true) both q and $p \to q$ are true but p is false. Concretely: "if it rains, the street is wet; the street is wet; so it rains" is wrong, because someone may have washed the street.

2.3.3 Logical equivalence and logical consequence

[Not in the slides: added material]

Two more notions, which the slides use implicitly (slide 16 and the proof-theory slides), are worth stating precisely.

Definition — Logical equivalence

Two formulas φ and ψ are **logically equivalent**, written $\varphi \equiv \psi$, iff they have the same truth value under *every* valuation, i.e. iff $\varphi \leftrightarrow \psi$ is a **tautology**.

Watch out — About slide 16

Slide 16 says: "If $p \leftrightarrow q$ is true, then p and q are said to be logically equivalent". Read it carefully: what is meant is that $p \leftrightarrow q$ is true under *every* valuation (a tautology). If $p \leftrightarrow q$ is true under *one* valuation, p and q merely happen to have the same value there. Example: $p =$ "it is Monday" and $q =$ "it rains" may both be true today, so $p \leftrightarrow q$ is true today, but they are certainly not logically equivalent.

Definition — Logical consequence (valid argument)

A formula ψ is a **logical consequence** of the premises $\varphi_1, \ldots, \varphi_n$, written $\varphi_1, \ldots, \varphi_n \models \psi$, iff *every valuation that makes all the premises true also makes* ψ *true*. Equivalently: there is no valuation with all premises true and ψ false; equivalently, $(\varphi_1 \wedge \cdots \wedge \varphi_n) \to \psi$ is a tautology. An argument whose conclusion is a logical consequence of its premises is called **(deductively) valid**.

Worked example — a few useful equivalences, checked by truth table

p	q	$p \to q$	$\neg p \vee q$	$\neg q \to \neg p$	$\neg(p \wedge q)$	$\neg p \vee \neg q$
F	F	T	T	T	T	T
F	T	T	T	T	T	T
T	F	F	F	F	T	T
T	T	T	T	T	F	F

Equal columns $\Rightarrow$ equivalent formulas: $p \to q \equiv \neg p \vee q \equiv \neg q \to \neg p$ (the last one is the *contrapositive*), and $\neg(p \wedge q) \equiv \neg p \vee \neg q$ (one of De Morgan's laws, used again in Section 2.4).

Why this matters in practice — Truth tables in real software

Deciding whether a formula is consistent ("is there a valuation that makes it true?") is the famous **SAT problem**. A truth table needs 2^n rows, which explodes quickly (the combinatorial explosion of Section 1.3); modern **SAT solvers** use clever search instead and routinely handle formulas with millions of variables. They are used to verify hardware circuits, find bugs in software, schedule exams and timetables, and solve puzzles such as Sudoku ("cell (1,1) contains 5" is an atomic proposition).

2.4 Proof theory: natural deduction

[Slides · Proof theory (prop.): 1–11]

Truth tables check validity by brute force: they look at *all* valuations. With 20 atomic propositions that is more than a million rows. Humans (and many programs) prove things differently: they start from what they know and take small, obviously correct steps until they reach the conclusion. This is what a **proof system** formalises.

2.4.1 What is a proof?

[Slides · Proof theory (prop.): 1]

Definition — Proof system, proof, derivation (page 1)

A **proof system** is a set of rules for constructing proofs. A **proof** is a series of sentences, each of which is either a **premise** or is justified by applying one of the

rules of the system to *earlier* sentences of the series. A sentence justified in this way is said to be **derived** from the premises.

The rules are designed with a very important limitation: they only allow you to derive a conclusion from premises if the argument is **deductively valid**, i.e. if it is not possible for all the premises to be true while the conclusion is false (the notion of logical consequence of Section 2.3). This is what makes it legitimate to call the series of sentences a proof.

The system used in the course is **natural deduction**: each rule lets you add a sentence that must be true given that certain earlier sentences are true, just like you would convince a friend "in clear, simple steps". For each connective there are two kinds of rules:

- **Elim** (elimination) rules break apart a sentence containing the connective (from $p \wedge q$ get p);
- **Intro** (introduction) rules build a sentence containing the connective (from p and q get $p \wedge q$);

plus a special rule **Reit** (reiteration: repeat a sentence you already have). These basic rules are enough to prove *any* truth-functionally valid argument. Since some proofs become very long, a few **derived rules** are added later to make proofs shorter (they prove nothing new).

Key result — How a proof is written (Fitch format)

Sentences are numbered consecutively. The premises come first, and a horizontal line is drawn below the last premise. Every following line is a (subsidiary or final) conclusion; next to it we write the rule that justifies it and the numbers of the earlier lines it is applied to, e.g. "$\wedge$ Elim: 1". Temporary assumptions open a **subproof**, drawn further to the right with its own vertical line (see below).

Intuition — Proofs and truth tables are two roads to the same place

A truth table asks: "in every possible world where the premises hold, does the conclusion hold?" A proof says: "here is a chain of small steps, each obviously safe, from the premises to the conclusion". For propositional logic the two roads lead to exactly the same valid arguments: the proof system is **sound** (it never proves an invalid argument) and **complete** (every valid argument has a proof).

2.4.2 Basic rules without subproofs

[Slides · Proof theory (prop.): 1–2]

Key result — Reit, $\wedge$ Elim, $\wedge$ Intro, $\rightarrow$ Elim, $\vee$ Intro

Rule	Schema	In words
Reit	$\dfrac{p}{p}$	you may repeat any sentence you already have
$\wedge$ Elim	$\dfrac{p \wedge q}{p} \quad \dfrac{p \wedge q}{q}$	from a conjunction, write either conjunct (as many times as you like)
$\wedge$ Intro	$\dfrac{p \quad q}{p \wedge q}$	join any two sentences you already have with $\wedge$
$\rightarrow$ Elim	$\dfrac{p \rightarrow q \quad p}{q}$	from a conditional and its antecedent, write its consequent (**Modus Ponens**)
$\vee$ Intro	$\dfrac{p}{p \vee q} \quad \dfrac{p}{q \vee p}$	from any sentence, write a disjunction containing it, with *any* other sentence, in either order

Read a schema as "if you have the sentences above the line, you may write the one below".

Why are they safe? $\wedge$ Elim: if "p and q" is true, p is true. $\wedge$ Intro: if both are true, so is their conjunction. $\rightarrow$ Elim: the only way $p \rightarrow q$ can be true with p true is that q is true (look at the truth table). $\vee$ Intro: a disjunction only needs *one* true disjunct, so adding any other disjunct is harmless.

Worked example — Example 1: $\{b \wedge c,\ j \wedge k\} \mathbin{/\!\therefore} c \wedge k$

1	$b \wedge c$	Premise
2	$j \wedge k$	Premise
3	c	$\wedge$ Elim: 1
4	k	$\wedge$ Elim: 2
5	$c \wedge k$	$\wedge$ Intro: 3, 4

(Premises are listed in braces; "$/\,\therefore$" reads "therefore".)

Worked example — Example 2: $\{g,\ h,\ (g \wedge h) \to (r \wedge s)\} \mathrel{/} \therefore\ s$

1	g	Premise
2	h	Premise
3	$(g \wedge h) \to (r \wedge s)$	Premise
4	$g \wedge h$	$\wedge$ Intro: 1, 2
5	$r \wedge s$	$\to$ Elim: 3, 4
6	s	$\wedge$ Elim: 5

Strategy: the conclusion s is "inside" $r \wedge s$, which is the consequent of line 3. To unlock it with $\to$ Elim we need the antecedent $g \wedge h$, which we build first.

Watch out — Modus Ponens vs. Affirming the Consequent

$\to$ Elim is the valid argument form **Modus Ponens**: $p \to q,\ p \mathrel{/} \therefore\ q$. A common error is the similar but **invalid** pattern **Affirming the Consequent**: $p \to q,\ q \mathrel{/} \therefore\ p$. The truth table of Section 2.3 gives the counterexample (p false, q true). "If the battery is dead, the car won't start; the car won't start; so the battery is dead" is wrong: maybe the tank is empty.

Worked example — Example 3: $\{(a \vee b) \to c,\ b\} \mathrel{/} \therefore\ c$

1	$(a \vee b) \to c$	Premise
2	b	Premise
3	$a \vee b$	$\vee$ Intro: 2
4	c	$\to$ Elim: 1, 3

From b, $\vee$ Intro could build $a \vee b$ or $b \vee a$ (or $b \vee z$ for any z). We choose $a \vee b$ because that is exactly the antecedent we need for line 1.

Exercise G1.A — From English to a proof (page 2–3)

Identify the premises and conclusion, translate using the letters given, then build a proof.

1. The starter must be malfunctioning. The car won't start, but the lights are working. If the lights work, the battery must be charged. If the car won't start when the battery is charged, there must be a problem with the starter. (s = the starter is functioning properly; c = the car starts; w = the lights are working; b = the battery is charged.)
2. If Mia moved out of her parents' house, she must be able to afford to. She can afford it only if she found both a roommate and a job. Since she moved out, she must have found a job. (m = Mia moved out; a = Mia can afford to move out; r = Mia found a roommate; j = Mia found a job.)

Solution

1. The conclusion is the first sentence ("the starter must be malfunctioning" $= \neg s$); "but" is logically an "and". Premises: $\neg c \wedge w$, $w \to b$, $(\neg c \wedge b) \to \neg s$.

1	$\neg c \wedge w$	Premise
2	$w \to b$	Premise
3	$(\neg c \wedge b) \to \neg s$	Premise
4	$\neg c$	$\wedge$ Elim: 1
5	w	$\wedge$ Elim: 1
6	b	$\to$ Elim: 2, 5
7	$\neg c \wedge b$	$\wedge$ Intro: 4, 6
8	$\neg s$	$\to$ Elim: 3, 7

2. "A only if B" means $A \to B$ (if A happens, B must hold). Premises: $m \to a$, $a \to (r \wedge j)$, m. Conclusion: j.

1	$m \to a$	Premise
2	$a \to (r \wedge j)$	Premise
3	m	Premise
4	a	$\to$ Elim: 1, 3
5	$r \wedge j$	$\to$ Elim: 2, 4
6	j	$\wedge$ Elim: 5

Lesson: "A only if B" is $A \to B$, *not* $B \to A$. "If" introduces the antecedent, "only if" introduces the consequent.

Exercise G1.B — Derive the conclusion from the premises (page 3)

1. $a,\ a \to b,\ (a \wedge b) \to c \ /\ \therefore\ b \wedge c$
2. $g \wedge h,\ g \to (d \to k),\ h \to d \ /\ \therefore\ d \wedge k$
3. $a \wedge b,\ (b \vee c) \to (d \wedge e),\ e \to (b \to s) \ /\ \therefore\ s$
4. $(m \vee o) \to k,\ k \to (l \wedge r),\ m \ /\ \therefore\ k \wedge (r \vee s)$

Solution

1.

1	a	Premise
2	$a \to b$	Premise
3	$(a \wedge b) \to c$	Premise
4	b	$\to$ Elim: 2, 1
5	$a \wedge b$	$\wedge$ Intro: 1, 4
6	c	$\to$ Elim: 3, 5
7	$b \wedge c$	$\wedge$ Intro: 4, 6

2.

1	$g \wedge h$	Premise
2	$g \to (d \to k)$	Premise
3	$h \to d$	Premise
4	g	$\wedge$ Elim: 1
5	h	$\wedge$ Elim: 1
6	$d \to k$	$\to$ Elim: 2, 4
7	d	$\to$ Elim: 3, 5
8	k	$\to$ Elim: 6, 7
9	$d \wedge k$	$\wedge$ Intro: 7, 8

3.

1	$a \wedge b$	Premise
2	$(b \vee c) \to (d \wedge e)$	Premise
3	$e \to (b \to s)$	Premise
4	b	$\wedge$ Elim: 1
5	$b \vee c$	$\vee$ Intro: 4
6	$d \wedge e$	$\to$ Elim: 2, 5
7	e	$\wedge$ Elim: 6
8	$b \to s$	$\to$ Elim: 3, 7
9	s	$\to$ Elim: 8, 4

4.

1	$(m \vee o) \to k$	Premise
2	$k \to (l \wedge r)$	Premise
3	m	Premise
4	$m \vee o$	$\vee$ Intro: 3
5	k	$\to$ Elim: 1, 4
6	$l \wedge r$	$\to$ Elim: 2, 5
7	r	$\wedge$ Elim: 6
8	$r \vee s$	$\vee$ Intro: 7
9	$k \wedge (r \vee s)$	$\wedge$ Intro: 5, 8

Lesson: work *backwards* from the conclusion. In 4, $k \wedge (r \vee s)$ needs k and $r \vee s$; $r \vee s$ needs only r ($\vee$ Intro); r is inside $l \wedge r$; and so on, until you reach the premises.

2.4.3 Rules with subproofs

[Slides · Proof theory (prop.): 3–5]

The remaining Intro and Elim rules need a new ingredient: we temporarily **assume** something that was not given as a premise, see what follows, and then give the assumption up. The part of a proof in which we hold a temporary assumption is a **subproof**, drawn to the right of a new vertical line (its first line, the assumption, is underlined).

Key result — The subproof restriction (page 10)

A rule may only be applied to sentences that appear (1) **earlier** in the proof and (2) **outside any subproof that has already ended**. Once a subproof ends, its lines may depend on an assumption we no longer hold, so they cannot be used individually any more; the only way to use a finished subproof is as a whole, with

one of the four rules below. Subproofs can be nested, but an inner subproof must end before the outer one.

Key result — ¬ Intro, ¬ Elim, → Intro, ∨ Elim

1	|	| p	Assumption
2	|	| ⋮	
3	|	| q	
4	|	| $\neg q$	
5	| $\neg p$		¬ Intro

¬ Intro (proof by contradiction)

1	|	| $\neg p$	Assumption
2	|	| ⋮	
3	|	| q	
4	|	| $\neg q$	
5	| p		¬ Elim

¬ Elim

1	|	| p	Assumption
2	|	| ⋮	
3	|	| q	
4	| $p \to q$		→ Intro

→ Intro (conditional proof)

1	| $p \vee q$		
2	|	| p	Assumption
3	|	| r	
4	|	| q	Assumption
5	|	| r	
6	| r		∨ Elim

∨ Elim (proof by cases)

- ¬ **Intro**: if an assumption leads to a sentence *and* its negation (a contradiction), the assumption cannot be true: close the subproof and write its negation.
- ¬ **Elim**: exactly like ¬ Intro, but the assumption is a negation $\neg p$; from the contradiction, conclude p.
- → **Intro**: assume the antecedent p; if you derive q, close the subproof and write $p \to q$.
- ∨ **Elim**: from $p \vee q$, open two subproofs, one assuming p and one assuming q; anything r derived in *both* can be written outside them.

Citation style: the assumption line through the contradictory pair, e.g. "¬ Intro: 3–(4,5)"; for ∨ Elim, the disjunction and both subproofs, e.g. "∨ Elim: 1, 4–6, 7–9".

Intuition — Everyday versions of these rules

- ¬ *Intro, reductio ad absurdum*: "Suppose the butler did it. Then he was in the kitchen at 9. But he was seen in the garden at 9. Contradiction: the butler did not do it."
- → *Intro, conditional proof*: "Suppose I take the 8:00 train. Then I arrive at 9:30, so I am in time for the meeting. Hence: if I take the 8:00 train, I am in time." You never claimed to take the train.
- ∨ *Elim, proof by cases*: "Either it rains or it is sunny. If it rains I take the bus; if it is sunny I take the bus (too hot to walk). So I take the bus."

Worked example — Example 4 ($\neg$ Intro): $\{\neg(g \wedge h),\ h\} \therefore \neg g$

"It's not the case that both Gary and Harvey will be promoted. Harvey is getting a promotion. So Gary won't be promoted."

1	$\neg(g \wedge h)$	Premise
2	h	Premise
3	$\quad g$	Assumption
4	$\quad g \wedge h$	$\wedge$ Intro: 2, 3
5	$\quad \neg(g \wedge h)$	Reit: 1
6	$\neg g$	$\neg$ Intro: 3—(4, 5)

Suppose Gary will be promoted (3). Then both will be promoted (4). But we were told they will not both be promoted (5). So the assumption is false (6). Note the use of Reit: the rule requires the contradictory pair *inside* the subproof, so we copy line 1 in.

Watch out — Wording in the slides

In the slides the assumption lines of subproofs are labelled "Premise". They are *temporary* premises (assumptions) that are discharged when the subproof ends, unlike the real premises at the top. In these notes they are labelled "Assumption".

Worked example — Example 5 ($\neg$ Elim): $\{\neg a \to n,\ \neg(n \vee e)\} \therefore a$

"If Alicia doesn't get the job, Nina will. But there is no way Nina or Eloise will get the job. So Alicia will get it."

1	$\neg a \to n$	Premise
2	$\neg(n \vee e)$	Premise
3	$\quad \neg a$	Assumption
4	$\quad n$	$\to$ Elim: 1, 3
5	$\quad n \vee e$	$\vee$ Intro: 4
6	$\quad \neg(n \vee e)$	Reit: 2
7	a	$\neg$ Elim: 3—(5, 6)

Worked example — Example 6 ($\to$ Intro): $\{a \to p,\ p \to (m \wedge r),\ m \to q\} \therefore a \to q$

"If Alberto gets his way, the policy will be approved. In that case, Marta and Roger will be angry. Marta will quit if she is angry. So if Alberto gets his way, Marta will quit."

1	$a \to p$	Premise
2	$p \to (m \wedge r)$	Premise
3	$m \to q$	Premise
4	$\quad a$	Assumption
5	$\quad p$	$\to$ Elim: 1, 4
6	$\quad m \wedge r$	$\to$ Elim: 2, 5
7	$\quad m$	$\wedge$ Elim: 6
8	$\quad q$	$\to$ Elim: 3, 7
9	$a \to q$	$\to$ Intro: 4—8

Worked example — Example 7 (nested subproofs): $\{b \to \neg(c \wedge d),\ a \to (b \to d)\} \therefore (a \wedge b) \to \neg c$

1	$b \to \neg(c \wedge d)$	Premise
2	$a \to (b \to d)$	Premise
3	$\quad a \wedge b$	Assumption
4	$\quad a$	$\wedge$ Elim: 3
5	$\quad b \to d$	$\to$ Elim: 2, 4
6	$\quad b$	$\wedge$ Elim: 3
7	$\quad d$	$\to$ Elim: 5, 6
8	$\quad\quad c$	Assumption
9	$\quad\quad c \wedge d$	$\wedge$ Intro: 7, 8
10	$\quad\quad \neg(c \wedge d)$	$\to$ Elim: 1, 6
11	$\quad \neg c$	$\neg$ Intro: 8—(9, 10)
12	$(a \wedge b) \to \neg c$	$\to$ Intro: 3—11

To prove an implication, assume its antecedent (3) and aim for the consequent $\neg c$; to prove a negation, assume the un-negated sentence (8) and aim for a contradiction. Inside the inner subproof we may use lines 1–7, because they are earlier and in subproofs that are still open.

Worked example — Example 8 ($\vee$ Elim): Rosa arrives late

"Rosa will get a ride with either Don or Glenn. If Don gives her a ride, she'll be picked up late and she'll arrive late. If Glenn gives her a ride, he'll make a stop along the way and Rosa will arrive late. So Rosa will arrive late."

1	$d \lor g$	Premise
2	$d \to (p \land a)$	Premise
3	$g \to (s \land a)$	Premise
4	$\quad d$	Assumption
5	$\quad p \land a$	$\to$ Elim: 2, 4
6	$\quad a$	$\land$ Elim: 5
7	$\quad g$	Assumption
8	$\quad s \land a$	$\to$ Elim: 3, 7
9	$\quad a$	$\land$ Elim: 8
10	a	$\lor$ Elim: 1, 4—6, 7—9

Since Rosa will be late either way, we don't need to know *which* alternative of line 1 is true. (The slides contain a typo, "If Don gives May a ride": it is Rosa.)

Worked example — Example 9 ($\lor$ Elim): $\{l \to (h \land u),\ i \to s,\ l \lor i\} \mathrel{/\!\!\therefore} u \lor s$

"If Luis is given the job, he'll be happy but Igor will be upset. If the job goes to Igor, Luis will be upset. One of them will get the job, so one of them will be upset."

1	$l \to (h \land u)$	Premise
2	$i \to s$	Premise
3	$l \lor i$	Premise
4	$\quad l$	Assumption
5	$\quad h \land u$	$\to$ Elim: 1, 4
6	$\quad u$	$\land$ Elim: 5
7	$\quad u \lor s$	$\lor$ Intro: 6
8	$\quad i$	Assumption
9	$\quad s$	$\to$ Elim: 2, 8
10	$\quad u \lor s$	$\lor$ Intro: 9
11	$u \lor s$	$\lor$ Elim: 3, 4—7, 8—10

The two cases end with *different* facts (u, s), so we first make both cases end with the *same* sentence $u \lor s$ using $\lor$ Intro.

Exercise G2 — Derive the conclusion from the premises, basic rules only (page 6)

1. $\neg b,\ \neg(\neg a \wedge \neg b) \mathrel{/\!\!\therefore} a$
2. $j \to (k \wedge m),\ m \to \neg k \mathrel{/\!\!\therefore} \neg j$
3. $c \to (d \wedge e),\ d \to \neg h \mathrel{/\!\!\therefore} c \to \neg h$
4. $(m \vee r) \to s,\ s \to (p \to q) \mathrel{/\!\!\therefore} (r \wedge p) \to q$
5. $(b \wedge c) \to d,\ (d \wedge g) \to h,\ b \wedge g \mathrel{/\!\!\therefore} c \to h$
6. $r \to \neg(p \wedge q),\ w \to (r \to q) \mathrel{/\!\!\therefore} w \to \neg(p \wedge r)$
7. $a \vee n,\ n \to (e \wedge s),\ a \to s \mathrel{/\!\!\therefore} s$
8. $j \to (k \vee l),\ (j \wedge k) \to m,\ j,\ l \to (m \wedge o) \mathrel{/\!\!\therefore} m$
9. $a \to (b \vee c),\ d \to \neg b,\ c \to \neg(a \wedge d) \mathrel{/\!\!\therefore} a \to \neg d$

Solution

All nine arguments are valid (checked by truth table). Strategy hints: to prove $\neg X$, assume X and find a contradiction ($\neg$ Intro); to prove $X \to Y$, assume X and derive Y ($\to$ Intro); to use $X \vee Y$, reason by cases ($\vee$ Elim).

1.

1	$\neg b$	Premise
2	$\neg(\neg a \wedge \neg b)$	Premise
3	$\quad \neg a$	Assumption
4	$\quad \neg a \wedge \neg b$	$\wedge$ Intro: 3, 1
5	$\quad \neg(\neg a \wedge \neg b)$	Reit: 2
6	a	$\neg$ Elim: 3—(4, 5)

2.

1	$j \to (k \wedge m)$	Premise
2	$m \to \neg k$	Premise
3	$\quad j$	Assumption
4	$\quad k \wedge m$	$\to$ Elim: 1, 3
5	$\quad k$	$\wedge$ Elim: 4
6	$\quad m$	$\wedge$ Elim: 4
7	$\quad \neg k$	$\to$ Elim: 2, 6
8	$\neg j$	$\neg$ Intro: 3—(5, 7)

3.

1	$c \to (d \wedge e)$	Premise
2	$d \to \neg h$	Premise
3	$\quad c$	Assumption
4	$\quad d \wedge e$	$\to$ Elim: 1, 3
5	$\quad d$	$\wedge$ Elim: 4
6	$\quad \neg h$	$\to$ Elim: 2, 5
7	$c \to \neg h$	$\to$ Intro: 3—6

4.

1	$(m \vee r) \to s$	Premise
2	$s \to (p \to q)$	Premise
3	$\quad r \wedge p$	Assumption
4	$\quad r$	$\wedge$ Elim: 3
5	$\quad m \vee r$	$\vee$ Intro: 4
6	$\quad s$	$\to$ Elim: 1, 5
7	$\quad p \to q$	$\to$ Elim: 2, 6
8	$\quad p$	$\wedge$ Elim: 3
9	$\quad q$	$\to$ Elim: 7, 8
10	$(r \wedge p) \to q$	$\to$ Intro: 3—9

5.

1	$(b \wedge c) \to d$	Premise
2	$(d \wedge g) \to h$	Premise
3	$b \wedge g$	Premise
4	$\quad c$	Assumption
5	$\quad b$	$\wedge$ Elim: 3
6	$\quad b \wedge c$	$\wedge$ Intro: 5, 4
7	$\quad d$	$\to$ Elim: 1, 6
8	$\quad g$	$\wedge$ Elim: 3
9	$\quad d \wedge g$	$\wedge$ Intro: 7, 8
10	$\quad h$	$\to$ Elim: 2, 9
11	$c \to h$	$\to$ Intro: 4—10

6. Two nested subproofs: $\to$ Intro outside, $\neg$ Intro inside.

1	$r \to \neg(p \wedge q)$	Premise
2	$w \to (r \to q)$	Premise
3	$\quad w$	Assumption
4	$\quad r \to q$	$\to$ Elim: 2, 3
5	$\quad\quad p \wedge r$	Assumption
6	$\quad\quad r$	$\wedge$ Elim: 5
7	$\quad\quad q$	$\to$ Elim: 4, 6
8	$\quad\quad p$	$\wedge$ Elim: 5
9	$\quad\quad p \wedge q$	$\wedge$ Intro: 8, 7
10	$\quad\quad \neg(p \wedge q)$	$\to$ Elim: 1, 6
11	$\quad \neg(p \wedge r)$	$\neg$ Intro: 5—(9, 10)
12	$w \to \neg(p \wedge r)$	$\to$ Intro: 3—11

7.

1	$a \vee n$	Premise
2	$n \to (e \wedge s)$	Premise
3	$a \to s$	Premise
4	$\quad a$	Assumption
5	$\quad s$	$\to$ Elim: 3, 4
6	$\quad n$	Assumption
7	$\quad e \wedge s$	$\to$ Elim: 2, 6
8	$\quad s$	$\wedge$ Elim: 7
9	s	$\vee$ Elim: 1, 4—5, 6—8

8.

1	$j \to (k \vee l)$	Premise
2	$(j \wedge k) \to m$	Premise
3	j	Premise
4	$l \to (m \wedge o)$	Premise
5	$k \vee l$	$\to$ Elim: 1, 3
6	$\quad k$	Assumption
7	$\quad j \wedge k$	$\wedge$ Intro: 3, 6
8	$\quad m$	$\to$ Elim: 2, 7
9	$\quad l$	Assumption
10	$\quad m \wedge o$	$\to$ Elim: 4, 9
11	$\quad m$	$\wedge$ Elim: 10
12	m	$\vee$ Elim: 5, 6—8, 9—11

9. The hardest one. Plan: assume a, then assume d and look for a contradiction between $a \wedge d$ and $\neg(a \wedge d)$. To get $\neg(a \wedge d)$ we need c, which must come from $b \vee c$ by cases. In the case b we have both b and $\neg b$, and from a contradiction anything follows (here c, via $\neg$ Elim).

1	$a \to (b \vee c)$	Premise
2	$d \to \neg b$	Premise
3	$c \to \neg(a \wedge d)$	Premise
4	$\quad a$	Assumption
5	$\quad b \vee c$	$\to$ Elim: 1, 4
6	$\quad\quad d$	Assumption
7	$\quad\quad \neg b$	$\to$ Elim: 2, 6
8	$\quad\quad a \wedge d$	$\wedge$ Intro: 4, 6
9	$\quad\quad\quad b$	Assumption
10	$\quad\quad\quad\quad \neg c$	Assumption
11	$\quad\quad\quad\quad b$	Reit: 9
12	$\quad\quad\quad\quad \neg b$	Reit: 7
13	$\quad\quad\quad c$	$\neg$ Elim: 10—(11, 12)
14	$\quad\quad\quad c$	Assumption
15	$\quad\quad\quad c$	Reit: 14
16	$\quad\quad c$	$\vee$ Elim: 5, 9—13, 14—15
17	$\quad\quad \neg(a \wedge d)$	$\to$ Elim: 3, 16
18	$\quad \neg d$	$\neg$ Intro: 6—(8, 17)
19	$a \to \neg d$	$\to$ Intro: 4—18

Lesson: lines 9–16 prove the *disjunctive syllogism* ($b \vee c,\ \neg b \ /\ \therefore\ c$) with basic rules only. It is so useful that it becomes a derived rule in the next sub-section, and with it this proof shrinks from 19 to 11 lines.

2.4.4 Derived rules

[Slides · Proof theory (prop.): 6–8, 11]

Derived rules express patterns of argument that we could prove using only the basic rules. Anything we prove with them, we could prove without them: they are useful not because of *what* they let us prove, but because they let us prove it more quickly.

Key result — The derived rules

Rule	Schema	In words
Modus Tollens	$\dfrac{p \to q \qquad \neg q}{\neg p}$	from a conditional and the negation of its consequent, derive the negation of its antecedent
Hypothetical syllogism	$\dfrac{p \to q \qquad q \to r}{p \to r}$	chain conditionals (any number, in any order)
Disjunctive syllogism	$\dfrac{p \lor q \quad \neg p}{q} \qquad \dfrac{p \lor q \quad \neg q}{p}$	from a disjunction and the negation of one disjunct, derive the other
Double negation ($\neg\neg$)	$\neg\neg p \equiv p$	drop or add $\neg\neg$, also inside a longer sentence
De Morgan's laws	$\neg(p \land q) \equiv \neg p \lor \neg q$ $\neg(p \lor q) \equiv \neg p \land \neg q$	substitute one form for the other, also inside a longer sentence

In proofs these rules are abbreviated **MT** (Modus Tollens), **HS** (hypothetical syllogism), **DS** (disjunctive syllogism), $\neg\neg$ and **DeM** (De Morgan). Double negation and De Morgan are **substitution rules**: since the two forms are logically equivalent (same truth table column), one may replace the other even as a *part* of a bigger sentence. The other rules apply only to whole lines.

Worked example — why Modus Tollens is safe: a proof with basic rules

[Not in the slides: added material]

1	$p \to q$	Premise
2	$\neg q$	Premise
3	$\quad p$	Assumption
4	$\quad q$	$\to$ Elim: 1, 3
5	$\quad \neg q$	Reit: 2
6	$\neg p$	$\neg$ Intro: 3—(4, 5)

Every use of Modus Tollens can be replaced by these 4 extra lines: that is what "derived" means.

Worked example — Example 10 (Modus Tollens): Raul's newspaper

"Raul would have picked up his newspaper if he were home. He must not be home, because he did not pick up his paper." $\{h \to n,\ \neg n\} \mathrel{/} \therefore\ \neg h$

1	$h \to n$	Premise
2	$\neg n$	Premise
3	$\neg h$	MT: 1, 2

This may remind you of the invalid form *affirming the consequent*; the negations make the difference: Modus Tollens is valid.

Worked example — Examples 11–13 (hypothetical and disjunctive syllogism)

Ex. 11

1	$a \to b$	Premise
2	$b \to c$	Premise
3	$c \to d$	Premise
4	$d \to e$	Premise
5	$a \to e$	HS: 1—4

Ex. 12

1	$(j \wedge k) \to m$	Premise
2	$\neg l$	Premise
3	$k \vee l$	Premise
4	$\quad j$	Assumption
5	$\quad k$	DS: 2, 3
6	$\quad j \wedge k$	$\wedge$ Intro: 4, 5
7	$\quad m$	$\to$ Elim: 1, 6
8	$j \to m$	$\to$ Intro: 4—7

Ex. 13

1	$(p \wedge q) \vee (r \wedge s)$	Premise
2	$p \to \neg(q \vee u)$	Premise
3	$\quad p \wedge q$	Assumption
4	$\quad p$	$\wedge$ Elim: 3
5	$\quad q$	$\wedge$ Elim: 3
6	$\quad q \vee u$	$\vee$ Intro: 5
7	$\quad \neg(q \vee u)$	$\to$ Elim: 2, 4
8	$\neg(p \wedge q)$	$\neg$ Intro: 3—(6, 7)
9	$r \wedge s$	DS: 1, 8
10	s	$\wedge$ Elim: 9

In Example 12 the slides list j as a fourth premise, but it is really the assumption of the subproof (it is discharged by $\to$ Intro in line 8).

Worked example — Example 14 (double negation): Ned is sick

"If Ned wasn't sick or on vacation, he'd be at work. He's not at work and not on vacation, so he must be sick." $\{\neg(s \vee o) \to w,\ \neg w \wedge \neg o\} \mathrel{/} \therefore\ s$

1	$\neg(s \vee o) \to w$	Premise
2	$\neg w \wedge \neg o$	Premise
3	$\neg w$	$\wedge$ Elim: 2
4	$\neg\neg(s \vee o)$	MT: 1, 3
5	$s \vee o$	$\neg\neg$: 4
6	$\neg o$	$\wedge$ Elim: 2
7	s	DS: 5, 6

Modus Tollens negates the antecedent $\neg(s \vee o)$, producing $\neg\neg(s \vee o)$; double negation removes the two $\neg$.

Worked example — Example 15 (De Morgan): Javier is sick

"If Javier wasn't sick or on vacation, he'd be at work. He's neither at work nor on vacation, so he must be sick." $\{\neg(s \vee o) \to w,\ \neg(w \vee o)\} \mathrel{/} \therefore\ s$

1	$\neg(s \vee o) \to w$	Premise
2	$\neg(w \vee o)$	Premise
3	$\neg w \wedge \neg o$	DeM: 2
4	$\neg w$	$\wedge$ Elim: 3
5	$\neg\neg(s \vee o)$	MT: 1, 4
6	$s \vee o$	$\neg\neg$: 5
7	$\neg o$	$\wedge$ Elim: 3
8	s	DS: 6, 7

Watch out — Typos in Examples 14–16 of the slides

In Example 15 the slides justify lines 4 and 7 with "$\wedge$ Elim: 2", but line 2 is $\neg(w \vee o)$, not a conjunction: the conjunction is line 3 (obtained by De Morgan), so the correct citation is "$\wedge$ Elim: 3" (as written above). In Example 14 the premise set is printed as $\{\neg(s \vee o) \supset w,\ \neg w\}$, while the proof uses $\neg w \wedge \neg o$ (and $\supset$ is just another symbol for $\to$). In Example 16, "Modus Tollens: 4.5" means lines 4,5.

Worked example — Examples 16–17 (De Morgan inside longer proofs)

Ex. 16: "Ed couldn't have passed history. If he had passed history or sociology, he would have graduated. If he'd graduated, he would have gone to the ceremony, but he didn't." $\{(h \vee s) \to g,\ (g \to c) \wedge \neg c\} \mathrel{/} \therefore\ \neg h$

1	$(h \lor s) \to g$	Premise
2	$(g \to c) \land \neg c$	Premise
3	$g \to c$	$\land$ Elim: 2
4	$(h \lor s) \to c$	HS: 1, 3
5	$\neg c$	$\land$ Elim: 2
6	$\neg(h \lor s)$	MT: 4, 5
7	$\neg h \land \neg s$	DeM: 6
8	$\neg h$	$\land$ Elim: 7

Ex. 17

1	$a \to (b \land c)$	Premise
2	$c \to d$	Premise
3	$\quad \neg d$	Assumption
4	$\quad \neg c$	MT: 2, 3
5	$\quad \neg b \lor \neg c$	$\lor$ Intro: 4
6	$\quad \neg(b \land c)$	DeM: 5
7	$\quad \neg a$	MT: 1, 6
8	$\neg d \to \neg a$	$\to$ Intro: 3—7

Exercise G3 — Derive the conclusion, derived rules allowed (page 9)

1. $(a \land b) \to c,\ \neg c \land \neg d \ /\ \therefore\ \neg a \lor \neg b$
2. $\neg(j \land k) \to m,\ j,\ \neg(l \lor m) \ /\ \therefore\ \neg k$
3. $d \to c,\ c \to \neg g,\ g \lor h \ /\ \therefore\ d \to h$
4. $j \to k,\ \neg l \to \neg(k \lor m) \ /\ \therefore\ j \to l$
5. $\neg b \to \neg c,\ \neg(a \lor b),\ (\neg a \land \neg c) \to d \ /\ \therefore\ d$
6. $\neg g \to (h \lor e),\ \neg h \land \neg e,\ g \lor m \ /\ \therefore\ m$
7. $\neg(a \lor b) \to c,\ \neg(b \lor c) \ /\ \therefore\ \neg a$
8. $r \to s,\ g \to h,\ (s \lor u) \to g \ /\ \therefore\ r \to h$
9. $o \to j,\ j \to (k \lor m),\ \neg(j \land m) \ /\ \therefore\ o \to k$

Solution

Watch out: exercises 2, 6 and 7 are not valid as printed in the slides. A truth table check finds, for each of them, a valuation that makes all premises true and the conclusion false, so *no* correct proof can exist:

- **2**: $j = k = \mathsf{T}$, $l = m = \mathsf{F}$. The premises actually prove k (not $\neg k$): the conclusion was probably meant to be k.
- **6**: $g = \mathsf{T}$, $h = e = m = \mathsf{F}$. The premises prove g, and $g \lor m$ then says nothing about m. With the third premise $\neg g \lor m$ ("$g \to m$") the argument becomes valid.
- **7**: $a = \mathsf{T}$, $b = c = \mathsf{F}$. The premises prove a (not $\neg a$).

Below, the valid versions are proved.

1.

1	$(a \wedge b) \to c$	Premise
2	$\neg c \wedge \neg d$	Premise
3	$\neg c$	$\wedge$ Elim: 2
4	$\neg(a \wedge b)$	MT: 1, 3
5	$\neg a \vee \neg b$	DeM: 4

2 (conclusion corrected to k).

1	$\neg(j \wedge k) \to m$	Premise
2	j	Premise
3	$\neg(l \vee m)$	Premise
4	$\neg l \wedge \neg m$	DeM: 3
5	$\neg m$	$\wedge$ Elim: 4
6	$\neg\neg(j \wedge k)$	MT: 1, 5
7	$j \wedge k$	$\neg\neg$: 6
8	k	$\wedge$ Elim: 7

3.

1	$d \to c$	Premise
2	$c \to \neg g$	Premise
3	$g \vee h$	Premise
4	$\quad d$	Assumption
5	$\quad d \to \neg g$	HS: 1, 2
6	$\quad \neg g$	$\to$ Elim: 5, 4
7	$\quad h$	DS: 3, 6
8	$d \to h$	$\to$ Intro: 4—7

4.

1	$j \to k$	Premise
2	$\neg l \to \neg(k \vee m)$	Premise
3	$\quad j$	Assumption
4	$\quad k$	$\to$ Elim: 1, 3
5	$\quad k \vee m$	$\vee$ Intro: 4
6	$\quad \neg\neg(k \vee m)$	$\neg\neg$: 5
7	$\quad \neg\neg l$	MT: 2, 6
8	$\quad l$	$\neg\neg$: 7
9	$j \to l$	$\to$ Intro: 3—8

5.

1	$\neg b \to \neg c$	Premise
2	$\neg(a \vee b)$	Premise
3	$(\neg a \wedge \neg c) \to d$	Premise
4	$\neg a \wedge \neg b$	DeM: 2
5	$\neg a$	$\wedge$ Elim: 4
6	$\neg b$	$\wedge$ Elim: 4
7	$\neg c$	$\to$ Elim: 1, 6
8	$\neg a \wedge \neg c$	$\wedge$ Intro: 5, 7
9	d	$\to$ Elim: 3, 8

6 (third premise corrected to $\neg g \lor m$).

1	$\neg g \to (h \lor e)$	Premise
2	$\neg h \land \neg e$	Premise
3	$\neg g \lor m$	Premise
4	$\neg(h \lor e)$	DeM: 2
5	$\neg\neg g$	MT: 1, 4
6	m	DS: 3, 5

7 (conclusion corrected to a).

1	$\neg(a \lor b) \to c$	Premise
2	$\neg(b \lor c)$	Premise
3	$\neg b \land \neg c$	DeM: 2
4	$\neg c$	$\land$ Elim: 3
5	$\neg\neg(a \lor b)$	MT: 1, 4
6	$a \lor b$	$\neg\neg$: 5
7	$\neg b$	$\land$ Elim: 3
8	a	DS: 6, 7

8.

1	$r \to s$	Premise
2	$g \to h$	Premise
3	$(s \lor u) \to g$	Premise
4	$\quad r$	Assumption
5	$\quad s$	$\to$ Elim: 1, 4
6	$\quad s \lor u$	$\lor$ Intro: 5
7	$\quad g$	$\to$ Elim: 3, 6
8	$\quad h$	$\to$ Elim: 2, 7
9	$r \to h$	$\to$ Intro: 4—8

9.

1	$o \to j$	Premise
2	$j \to (k \lor m)$	Premise
3	$\neg(j \land m)$	Premise
4	$\quad o$	Assumption
5	$\quad j$	$\to$ Elim: 1, 4
6	$\quad k \lor m$	$\to$ Elim: 2, 5
7	$\quad \neg j \lor \neg m$	DeM: 3
8	$\quad \neg\neg j$	$\neg\neg$: 5
9	$\quad \neg m$	DS: 7, 8
10	$\quad k$	DS: 6, 9
11	$o \to k$	$\to$ Intro: 4—10

In exercise 6 (line 5), Modus Tollens gives $\neg\neg g$, which is exactly the negation of the first disjunct $\neg g$ of line 3, so disjunctive syllogism applies directly. **Lesson**: before trying to prove something, a quick truth-table check (or a counterexample) can save you a lot of time; proof rules can only prove valid arguments.

Cheat sheet — The proof rules at a glance (pages 10–11)

General restriction	a rule applies only to sentences that are earlier and outside any subproof that has already ended
Reit	repeat any sentence you already have
$\wedge$ Elim / $\wedge$ Intro	take a conjunct / join two sentences with $\wedge$
$\rightarrow$ Elim (Modus Ponens)	from $p \rightarrow q$ and p, write q
$\vee$ Intro	from p, write $p \vee q$ or $q \vee p$ (any q)
$\neg$ Intro / $\neg$ Elim	assumption p (resp. $\neg p$) leads to q and $\neg q$: write $\neg p$ (resp. p)
$\rightarrow$ Intro	subproof from p to q: write $p \rightarrow q$
$\vee$ Elim	from $p \vee q$ and subproofs $p \rightsquigarrow r$, $q \rightsquigarrow r$: write r
Derived	Modus Tollens, hypothetical syllogism, disjunctive syllogism, $\neg\neg$, De Morgan

Cheat sheet — Propositional logic

- A **logic** = syntax (well-formed formulas) + semantics (meaning, truth) + proof theory (rules to manipulate formulas).
- **Proposition**: a statement that is either true or false, never both. **Atomic** if its truth does not depend on other propositions.
- Connectives and their truth conditions: $p \wedge q$ true iff both true; $p \vee q$ true iff at least one true; $\neg p$ true iff p false; $p \rightarrow q$ **false only when** p **is true and** q **is false**; $p \leftrightarrow q$ true iff p and q have the same value.
- n atomic propositions $\Rightarrow 2^n$ rows in the truth table.
- **Valuation**: assigns T/F to each atomic proposition; each row of a truth table is a valuation.
- **Tautology**: true under every valuation. **Consistent** (satisfiable): true under at least one. **Inconsistent**: true under none.
- **Natural deduction**: Intro and Elim rule for each connective, plus Reit. $\neg$Intro, $\neg$Elim, $\rightarrow$Intro and $\vee$Elim need **subproofs** (temporary assumptions); lines inside a closed subproof cannot be reused.
- $\rightarrow$Elim = **Modus Ponens** ($p \rightarrow q, p \ / \ q$). Invalid look-alike: **affirming the consequent** ($p \rightarrow q, q \ / \ p$).
- Derived rules: Modus Tollens ($p \rightarrow q, \neg q \ / \ \neg p$), hypothetical syllogism, disjunctive syllogism, double negation, De Morgan ($\neg(p \wedge q) \equiv \neg p \vee \neg q$, $\neg(p \vee q) \equiv \neg p \wedge \neg q$).

3 First-order logic

[Slides · FOL: 1–21; Proof theory (FOL): 1–4]

Propositional logic treats "all monitors are ready" as one indivisible block, true or false. It cannot look *inside* the sentence and see that it talks about *all* objects of a certain kind. **First-order logic** (FOL, also called *predicate logic*) can: it has names for objects, functions, relations between objects and, above all, **quantifiers**

("for all", "there exists"). It is the logic behind Prolog and behind the beliefs of BDI agents.

This chapter covers, in the order of the slides:

1. why propositional logic is not enough;
2. the syntax of FOL: terms (constants, variables, functions) and predicates;
3. quantifiers ($\forall$, $\exists$, $\exists_1$) and how to translate English into FOL;
4. decidability: FOL has no truth-table method, and no method at all that always works;
5. proof theory for FOL: Frege/Hilbert-style axioms, soundness, the sequent calculus LK.

3.1 Why not propositional logic?

[Slides · FOL: 2–3]

Consider the argument:

$$\frac{\begin{array}{l}\text{all monitors are ready}\\ \text{X12 is a monitor}\end{array}}{\text{therefore X12 is ready}}$$

Intuitively it is *sound*: if we accept that the two premises (above the line) are true, we must accept that the conclusion is true too. Each of the three statements is a proposition (true or false), so we could try propositional logic:

$$p = \text{"all monitors are ready"}, \quad q = \text{"X12 is a monitor"}, \quad r = \text{"X12 is ready"}.$$

But then the argument becomes $p,\ q \ /\ \therefore\ r$, which is **not** valid: the valuation $p = q = \mathsf{T}$, $r = \mathsf{F}$ makes the premises true and the conclusion false. The sense of the argument is lost, because the connection between "monitors", "X12" and "ready" is hidden inside the atomic propositions.

Intuition — Propositional logic is too coarse-grained

Propositional logic sees sentences as black boxes. The validity of the monitor argument depends on what is *inside* the boxes: the same individual (X12), the same property (being a monitor, being ready), and the word "all". We need a logic that can name individuals, express properties and relations, and quantify over all individuals: first-order logic. In FOL the argument becomes

$$\forall x\,(\mathit{Monitor}(x) \to \mathit{Ready}(x)), \quad \mathit{Monitor}(X12) \ /\ \therefore\ \mathit{Ready}(X12)$$

and it is valid (Section 3.5).

Why this matters in practice

The slides come from a Software Engineering course, where FOL is used for **formal specification** (e.g. the Z notation): "every reactor has a temperature between 100 and 1000" is a requirement about *all* reactors. In AI the same language describes knowledge bases ("every employee has a manager"), database queries (SQL is essentially FOL), and the beliefs of agents.

3.2 Syntax of FOL: terms and predicates

[Slides · FOL: 4–13]

First-order logic is a generalisation of propositional logic with much greater expressive power. Its language has two layers:

- **terms** talk about *objects* (numbers, people, monitors);
- **predicates** (atoms) talk about *facts* on those objects, and are true or false.

Formulas are then built from atoms with the connectives of propositional logic and with quantifiers (Section 3.3).

3.2.1 Terms: constants, variables, functions

[Slides · FOL: 5–10]

Definition — Term (slide 5)

The basic components of FOL are **terms**. A term denotes some *object*, other than true or false.

The slides use a *typed* version of FOL (as in the Z specification language): every object belongs to a set, its *type* ($\mathbb{N}$, $\mathbb{Z}$, Person, Monitor, ...).

Definition — Constant and variable (slide 6)

- A **constant** of type T is a name that denotes some *particular* object in the set T. Examples: 8 (the number eight, in $\mathbb{N}$ and $\mathbb{Z}$); *mike* (the individual Michael Wooldridge); S after "let S be the set $\{1, 2, 3\}$" (by saying what it denotes we have given it an **interpretation**).
- A **variable** of type T is a name that can denote *any* value in the set T. A variable of type $\mathbb{N}$ can stand for any natural number.

Definition — Functional term (slides 7–8)

FOL has a set of **function symbols**; each denotes a function. Each function symbol has an **arity** $n \in \mathbb{N}$ (its number of arguments), an argument type for each argument and a return type. If f is a function symbol of type T, with arity n and

argument types $T_1, \ldots, T_n$, and $\tau_1, \ldots, \tau_n$ are terms of types $T_1, \ldots, T_n$, then

$$f(\tau_1, \ldots, \tau_n)$$

is a **functional term** (of type T).

This is just like a function in a programming language: the slides' Modula-2 example `PROCEDURE f(a1:T1; ... an:Tn) : T;` declares a function with n typed arguments and return type T.

Worked example — the function *plus* (slide 9)

Let *plus* take two natural numbers and return their sum. Then:

Acceptable terms	Not acceptable
$plus(2, 3)$	$plus(-1, 0)$ ($-1 \notin \mathbb{N}$: wrong type)
$plus(0, 1)$	$plus(0.1, 2)$ ($0.1 \notin \mathbb{N}$: wrong type)
$plus(plus(1, 2), 4)$	$plus(1)$ (wrong arity; added)
$plus(plus(plus(0, 1), 2), 4)$	

Terms can be nested: the argument of a function can itself be a functional term.

Writing $2 + 3$ instead of $plus(2, 3)$ is merely a convention (infix notation); for logic, $+, -, /, *, \sqrt{}, \sin, \cos, \ldots$ are all function symbols. For instance $(x + 3) * \sin 90$ is the term $times(plus(x, 3), \sin(90))$ (slide 10).

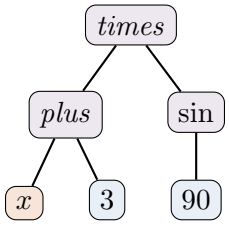


Figure 9: The term $(x+3)*\sin 90$ as a tree. Purple: function symbols; blue: constants; orange: variables. Leaves are always constants or variables.

3.2.2 Predicates

[Slides · FOL: 11–13]

Definition — Predicate (atom) (slide 11)

FOL contains a stock of **predicate symbols**, standing for relationships between objects. Each has an arity and a type for each argument. If P is a predicate symbol of arity n with argument types $T_1, \ldots, T_n$, and $\tau_1, \ldots, \tau_n$ are terms of those types, then

$$P(\tau_1, \ldots, \tau_n)$$

is a **predicate** (also called an **atom** or atomic formula), which will be either true or false under some interpretation.

Worked example — the predicate *gt* (slide 12)

Let gt ("greater than") take two natural numbers. Then $gt(4,3)$ is a predicate that evaluates to *true*, $gt(3,4)$ is a predicate that evaluates to *false*, while $gt(-1,2)$ is not a predicate at all (-1 has the wrong type). The usual mathematical relations $>, <, =, \geq, \leq, \neq, \in, \subset, \subseteq$ are predicate symbols written infix. Predicates combine with the connectives: $(2 > 3) \wedge (6 = 7) \vee (\sqrt{4} = 2)$.

Watch out — Terms vs. predicates: the most common confusion

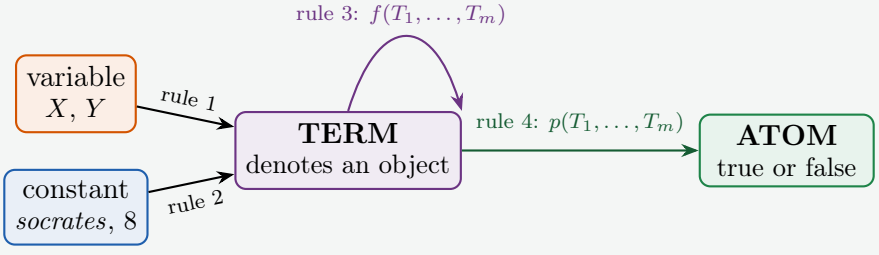
A **term** names an *object*; it is never true or false. A **predicate** states a *fact*; it is always true or false (under an interpretation). $plus(2,3)$ is a term (it denotes 5); $gt(plus(2,3),4)$ is a predicate (it is true). Writing $gt(4,3) \wedge 5$ is nonsense: you cannot "and" a number. Likewise $mother(ann)$ (a function: *the mother of Ann*, a person) differs from $mother(mary, ann)$ (a predicate: *Mary is the mother of Ann*, a fact).

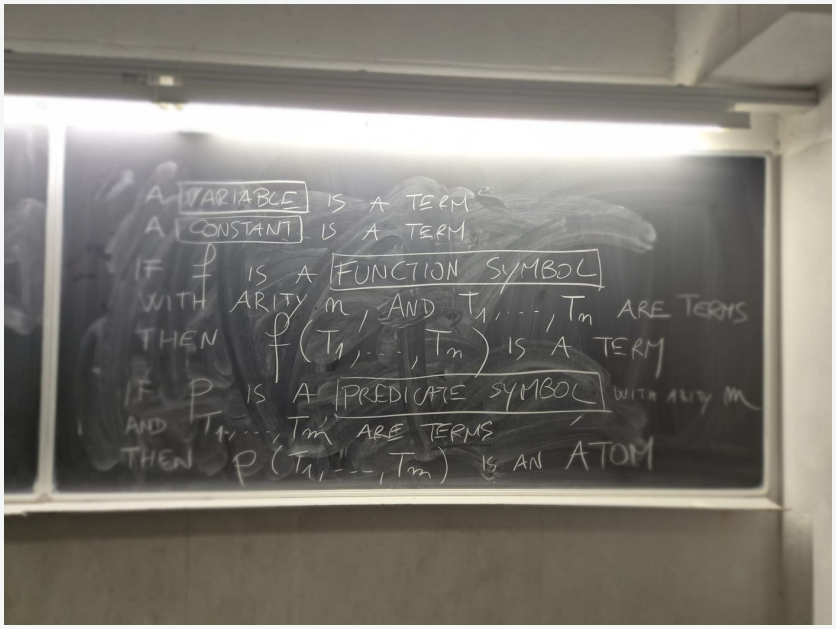
Some vocabulary (slide 13):

- a predicate with variables, e.g. $gt(x,3)$, cannot be evaluated until we know what x stands for (an interpretation of the variables);
- a predicate that contains **no variables** is a **proposition** (e.g. $gt(4,3)$);
- predicates of arity 1 are called **properties**, for example $Man(x)$, $Mortal(x)$, $Malfunctioning(x)$, all with one argument;
- predicates of arity 0 (no arguments) are called **primitive propositions**: they are exactly the atomic propositions of propositional logic. So propositional logic is the special case of FOL where all predicates have arity 0.

Whiteboard 1 — Terms and atoms written after the FOL lectures

Rule	Definition (as on the board)
1	A **variable** is a term.
2	A **constant** is a term.
3	If f is a **function symbol** with arity m, and $T_1, \ldots, T_m$ are terms, then $f(T_1, \ldots, T_m)$ is a term.
4	If p is a **predicate symbol** with arity m, and $T_1, \ldots, T_m$ are terms, then $p(T_1, \ldots, T_m)$ is an **atom**.





Original photo of the whiteboard

The definition is **recursive**: rule 3 builds terms out of terms, so terms can be nested as deeply as we want ($f(g(X), c)$). Rule 4 is applied *once*, on top: an atom contains terms, but an atom is never an argument of a function or of another predicate. Example with X a variable, *socrates* a constant, *father* a function of arity 1, *older* a predicate of arity 2: $father(socrates)$ is a term (rule 3), $older(father(socrates), X)$ is an atom (rule 4). Exactly the same definitions hold in Prolog (Section 7), where variables start with an upper-case letter, as on the board.

Why this matters in practice — A small knowledge base

[Not in the slides: added material]

A university information system could use: constants *alice*, *bob*, *sdai*; a function $advisor(x)$ (the advisor of student x); predicates $student(x)$, $enrolled(x, c)$, $passed(x, c, grade)$. The atom $enrolled(alice, sdai)$ is a row in a database table; $enrolled(advisor(bob), sdai)$ says that Bob's advisor attends SDAI. This is exactly how Prolog programs and agents' belief bases store knowledge.

3.3 Quantifiers

[Slides · FOL: 14–20]

Quantification is the central part of first-order logic. With terms and predicates alone we still cannot say:

- all men have a mother;
- every natural number has a prime factor.

These sentences speak about *all* objects, without naming them. We need quanti-

fiers.

3.3.1 The three quantifiers and their syntax

[Slides · FOL: 15–18]

Definition — Quantifiers (slide 15)

Symbol	Name	Read
$\forall$	universal quantifier	"for all …"
$\exists$	existential quantifier	"there exists …" (at least one)
$\exists_1$	unique quantifier	"there exists a unique …" (exactly one)

In the Z notation of the slides a quantified formula has the form

$$\text{quantifier} \quad \text{signature} \bullet \text{predicate}$$

where the signature is *variable : type*. Example: $\forall x : Man \bullet Mortal(x)$.

Worked example — reading quantified formulas (slides 17–18)

Formula	Meaning
$\forall x : Man \bullet Mortal(x)$	for all x of type Man, x is mortal (all men are mortal)
$\forall x : Man \bullet \exists_1 y :$ $Woman \bullet MotherOf(x, y)$	every man has exactly one mother (y is the mother of x)
$\exists m :$ $Monitor \bullet MonitorState(m, ready)$	there is a monitor in the ready state
$\forall r : Reactor \bullet \exists_1 t :$ $100..1000 \bullet Temp(r) = t$	every reactor has a temperature in the range 100 to 1000
$\exists n : \mathbb{N} \bullet n = n * n$	some natural number is equal to its own square (true: 0 and 1)
$\exists c : EC \bullet Borders(c, Albania)$	some EC country borders Albania (Greece)
$\forall m, n : Person \bullet \neg Superior(m, n)$	no person is superior to another
$\forall m : Person \bullet \neg\exists n :$ $Person \bullet Superior(m, n)$	ditto (same meaning)

Without types. Most AI texts (and Prolog) use untyped FOL: there is one big domain of objects, and types become ordinary predicates. The typed formulas translate with a fixed pattern that is worth memorising.

Key result — Typed to untyped quantifiers

[Not in the slides: added material]

$$\forall x : T \bullet P(x) \quad \text{becomes} \quad \forall x \left(T(x) \to P(x)\right)$$
$$\exists x : T \bullet P(x) \quad \text{becomes} \quad \exists x \left(T(x) \wedge P(x)\right)$$

$\forall$ goes with $\to$, $\exists$ goes with $\wedge$.

Watch out — The two classic translation mistakes

- "All men are mortal" is $\forall x\,(Man(x) \to Mortal(x))$, **not** $\forall x\,(Man(x) \wedge Mortal(x))$: the latter says that *everything* (every chair, every number) is a man and is mortal.
- "Some monitor is ready" is $\exists x\,(Monitor(x) \wedge Ready(x))$, **not** $\exists x\,(Monitor(x) \to Ready(x))$: the latter is true as soon as there exists any object that is not a monitor (false antecedent!), even if no monitor is ready.

Watch out — The order of quantifiers matters

[Not in the slides: added material]

$\forall x\,\exists y\,MotherOf(x, y)$: everybody has a mother (possibly a different one for each x): true.
$\exists y\,\forall x\,MotherOf(x, y)$: there is one y who is the mother of everybody: false. Swapping $\forall$ and $\exists$ changes the meaning; swapping two $\forall$ (or two $\exists$) does not.

3.3.2 Free and bound variables

[Not in the slides: added material]

A quantifier $\forall x$ or $\exists x$ **binds** the variable x inside its **scope** (the formula it is applied to). An occurrence of a variable that is not bound by any quantifier is **free**. In $\forall x\,(gt(x, y) \to gt(x, 0))$, x is bound and y is free. A formula with no free variables is a **sentence** (or *closed* formula): only sentences are simply true or false in an interpretation; a formula with free variables, like $gt(x, 3)$, needs a value for its free variables (slide 13). This distinction comes back in the proof rules of Section 3.5.

3.3.3 Quantifiers as big conjunctions and disjunctions

[Slides · FOL: 19–20]

Key result — Quantifiers over a finite set (slide 19)

Universal quantification is like conjunction, existential quantification like disjunction:

$$\forall n : \{2, 4, 6\} \bullet Even(n) \quad \text{is the same as} \quad Even(2) \wedge Even(4) \wedge Even(6)$$
$$\exists n : \{7, 8, 9\} \bullet Prime(n) \quad \text{is the same as} \quad Prime(7) \vee Prime(8) \vee Prime(9)$$

Key result — Duality of $\forall$ and $\exists$ (slide 20)

$$\begin{aligned} \forall x : T \bullet P(x) \quad &\leftrightarrow \quad \neg\, \exists x : T \bullet \neg P(x) \\ \exists x : T \bullet P(x) \quad &\leftrightarrow \quad \neg\, \forall x : T \bullet \neg P(x) \end{aligned}$$

Saying that everything has some property is the same as saying that there is nothing that does not have it. Saying that something has the property is the same as saying that it is not the case that everything lacks it.

Intuition — Duality is De Morgan in disguise

Over the finite set $\{a, b, c\}$:

$$\begin{aligned} \neg\, \forall x\, P(x) &= \neg\big(P(a) \wedge P(b) \wedge P(c)\big) && \triangleleft\ \forall = \text{big} \wedge \\ &\equiv \neg P(a) \vee \neg P(b) \vee \neg P(c) && \triangleleft\ \text{De Morgan} \\ &= \exists x\, \neg P(x) && \triangleleft\ \exists = \text{big} \vee \end{aligned}$$

Negating both sides gives the first duality law. Practical reading: to refute "all monitors are ready" you need one monitor that is *not* ready (a counterexample).

Key result — The unique quantifier, defined with the other two

[Not in the slides: added material]

$\exists_1 x\, P(x) \;\equiv\; \exists x \big(P(x) \wedge \forall y\, (P(y) \to y = x)\big)$: there is an x with the property, and any y with the property is that same x.

Exercise Q.1 — Translate into FOL (added)

Use the predicates $\mathit{Student}(x)$, $\mathit{Course}(x)$, $\mathit{Attends}(x, y)$, $\mathit{Passed}(x, y)$.

1. Every student attends some course.
2. Some course is attended by every student.
3. No student passed a course without attending it.
4. Not every student passed SDAI.

Solution

1. $\forall x \big(\mathit{Student}(x) \to \exists y\, (\mathit{Course}(y) \wedge \mathit{Attends}(x, y))\big)$
2. $\exists y \big(\mathit{Course}(y) \wedge \forall x\, (\mathit{Student}(x) \to \mathit{Attends}(x, y))\big)$ (stronger than 1: the order of quantifiers changed)
3. $\neg\exists x\, \exists y \big(\mathit{Student}(x) \wedge \mathit{Passed}(x, y) \wedge \neg \mathit{Attends}(x, y)\big)$, equivalently $\forall x\, \forall y \big(\mathit{Student}(x) \wedge \mathit{Passed}(x, y) \to \mathit{Attends}(x, y)\big)$ (by duality and De Morgan).
4. $\neg\forall x\, (\mathit{Student}(x) \to \mathit{Passed}(x, \mathit{sdai}))$, equivalently $\exists x\, (\mathit{Student}(x) \wedge \neg \mathit{Passed}(x, \mathit{sdai}))$: some student did not pass SDAI.

Lesson: always check the pattern $\forall \ldots \to$, $\exists \ldots \wedge$.

3.4 Decidability

[Slides · FOL: 21]

In propositional logic some formulas are tautologies (true under all interpretations), and there is a *procedure*, the truth-table method, that tells in a finite time whether any formula is a tautology.

Definition — Valid formula (slide 21)

A formula of FOL that is true under all interpretations is said to be **valid** (it is the FOL analogue of a tautology).

Truth tables do not work for FOL: an interpretation chooses a domain of objects (possibly infinite, like $\mathbb{N}$) and a meaning for every symbol, so there are infinitely many interpretations and a "truth table" would be infinite. Is there *any other* procedure guaranteed to tell us, in a finite amount of time, whether a FOL formula is or is not valid? **No.**

Key result — Undecidability of FOL (Church and Turing, 1936)

There is no algorithm that, given any FOL formula, always terminates and correctly answers whether the formula is valid. FOL is said to be **undecidable**.

Intuition — Semi-decidable: "yes" can be found, "no" may never come

[Not in the slides: added material]

FOL is *semi-decidable*: there are procedures (e.g. enumerating all proofs, or resolution) that, if the formula *is* valid, will find a proof in finite time. If the formula is *not* valid, the procedure may run forever, and at no point can you tell "not valid" apart from "valid, but the proof has not been found yet". You will meet exactly this behaviour in Prolog: a query either succeeds, fails, or loops forever (Section 7).

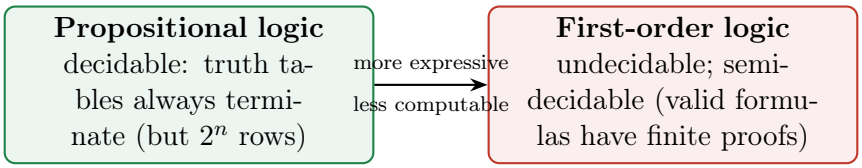


Figure 10: The price of expressive power: FOL can say much more, but no algorithm can decide validity for all its formulas.

Why this matters in practice

This is why practical symbolic AI uses *fragments* of FOL with better computational behaviour: Prolog uses Horn clauses with a specific search strategy (Section 7), description logics (behind ontologies and the Semantic Web) are decidable fragments, and agent languages such as AgentSpeak keep beliefs as ground atoms.

3.5 Proof theory for first-order logic

[Slides · Proof theory (FOL): 1–4]

The source for this part is a set of university lecture notes (Math 260A, UCSD, 2012), more technical than the rest of the course. The goal here is to understand the *ideas*: what a proof system for FOL looks like, what soundness means, and how quantifiers are handled.

In propositional logic the simplest proof system is the truth table. In FOL there is no analogue: the truth tables would be infinite (Section 3.4). We go directly to **Frege-style** systems, called **Hilbert-style** systems in FOL: *many axioms, very few rules.*

3.5.1 Frege (Hilbert-style) proofs

[Slides · Proof theory (FOL): 1–2]

Definition — Axioms of the Frege system for FOL (page 1)

- **Propositional axioms**: all axioms of propositional Frege systems, such as $\varphi \to (\psi \to \varphi)$, where now φ, ψ can be any FOL formulas.
- **Quantifier axioms**: for any formula φ and any term t,

$$\varphi(t) \to \exists x\, \varphi(x) \qquad\qquad \forall x\, \varphi(x) \to \varphi(t)$$

  (provided t is *substitutable* for x; the notes mostly ignore this technical condition).
- **Equality axioms** (if $=$ is in the language):
  - $\forall x\,(x = x)$ (reflexivity)
  - $\forall x \forall y\,(x = y \to y = x)$ (symmetry)
  - $\forall x \forall y \forall z\,(x = y \wedge y = z \to x = z)$ (transitivity)
  - $\forall \vec{x}\, \forall \vec{y}\,\big(x_1 = y_1 \wedge \cdots \wedge x_k = y_k \to f(x_1, \ldots, x_k) = f(y_1, \ldots, y_k)\big)$ for every function f of arity k
  - $\forall \vec{x}\, \forall \vec{y}\,\big(x_1 = y_1 \wedge \cdots \wedge x_k = y_k \to (P(x_1, \ldots, x_k) \to P(y_1, \ldots, y_k))\big)$ for every relation P of arity k (substitution of equals)

Watch out — Corrections to page 1 of the notes

- The notes say "t should be substitutable for t": it should read "for x".
- The last two equality axioms are printed without the hypothesis $x_1 = y_1 \wedge \cdots \wedge x_k = y_k$, which would make them say that f gives the same value on *any* arguments (false). The correct versions are written above: equal inputs give equal outputs, and equal objects have the same properties.

Intuition — Reading the quantifier axioms

$\forall x\,\varphi(x) \to \varphi(t)$: "if something holds for everything, it holds for any particular thing t" (*instantiation*). From "all humans are mortal" you may conclude "if Socrates is human, Socrates is mortal". $\varphi(t) \to \exists x\,\varphi(x)$: "if it holds for some particular t, then it holds for something" (*generalisation to* $\exists$). From $Ready(X12)$ conclude "some monitor is ready".

Key result — Rules of inference (pages 1–2)

Modus Ponens: $\dfrac{\varphi \qquad \varphi \to \psi}{\psi}$ $\qquad$ $\dfrac{\psi \to \varphi}{\psi \to \forall x\,\varphi}$ $\qquad$ $\dfrac{\varphi \to \psi}{\exists x\,\varphi \to \psi}$

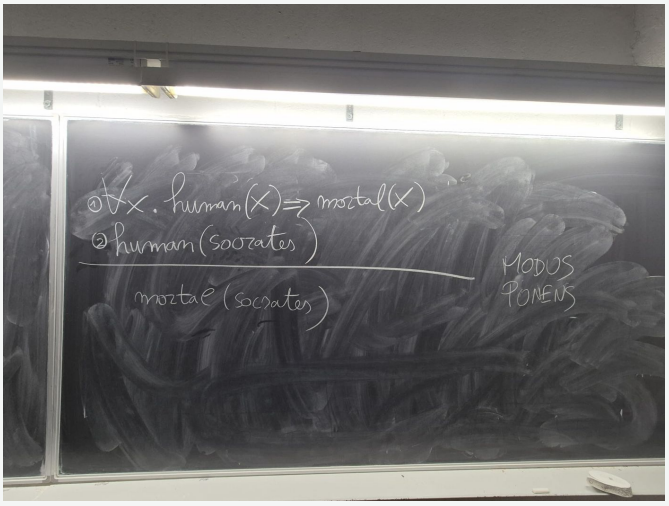
The two quantifier rules require that x **is not free in** ψ.

Intuition — Why "x not free in ψ"?

Rule 2: if $\psi \to \varphi(x)$ has been proved and ψ says nothing about x, then the proof worked for an arbitrary x, so $\psi \to \forall x\,\varphi$. Without the condition we could go from $x = 0 \to x = 0$ to $x = 0 \to \forall x\,(x = 0)$, i.e. "if this number is 0 then every number is 0": wrong. Rule 3: if $\varphi(x)$ implies ψ whatever x is, then knowing that *some* x satisfies φ is enough to get ψ, provided ψ itself does not depend on which x it was.

Whiteboard 2 — Modus Ponens: Socrates is mortal written during the FOL proof-theory lecture

$$\dfrac{\begin{array}{ll}① & \forall x\,.\ human(x) \Rightarrow mortal(x) \\ ② & human(socrates)\end{array}}{mortal(socrates)} \quad \textsc{Modus Ponens}$$



Original photo of the whiteboard

The board shows Aristotle's syllogism (Section 1.3) in FOL. Strictly speaking, Modus Ponens needs an implication whose antecedent is *exactly* $human(socrates)$,

so a quantifier step comes first. In full, with the Frege system above: Writing h for *human*, m for *mortal* and s for *socrates*:

1	$\forall x\,(h(x) \to m(x))$	premise
2	$h(s)$	premise
3	$\forall x\,(h(x) \to m(x)) \to (h(s) \to m(s))$	quantifier axiom with $t = s$
4	$h(s) \to m(s)$	Modus Ponens 1, 3
5	$m(s)$	Modus Ponens 2, 4

In Prolog the same knowledge is `mortal(X) :- human(X).` and `human(socrates).`; the query `?- mortal(socrates).` answers `true`: Prolog's resolution does the instantiation ($X = socrates$, called *unification*) and the Modus Ponens step in one go (Section 7).

3.5.2 Provability and soundness

[Slides · Proof theory (FOL): 2]

Definition — Provability $\Gamma \vdash \varphi$ (page 2)

Let Γ be a set of formulas (the hypotheses). We write $\Gamma \vdash \varphi$ ("Γ proves φ") if there is a finite sequence of formulas, each coming from Γ, from the axioms, or from earlier formulas by an inference rule, ending at φ.

Recall that $\Gamma \models \varphi$ ("Γ entails φ") means: in every structure (interpretation) M in which all formulas of Γ are true, φ is true too. $\vdash$ is about *symbols and rules*; $\models$ is about *meaning and truth*.

Key result — Soundness theorem (page 2)

If $\Gamma \vdash \varphi$, then $\Gamma \models \varphi$. (Everything provable is true.)

Proof idea. Take a proof P of φ from Γ and a structure M in which Γ is true, i.e. for all object assignments σ (values for the free variables) and all $\gamma \in \Gamma$, $M \models \gamma[\sigma]$. Show by **induction on the number of lines** of P that every line ψ is true in M under all σ. Base cases: hypotheses (true by assumption) and axioms (true in every structure). Modus Ponens preserves truth (truth table of $\to$). For the $\exists$-rule $\frac{\varphi \to \psi}{\exists x \varphi \to \psi}$ argue by contrapositive:

1. suppose $M \not\models (\exists x \varphi \to \psi)[\sigma]$; then $M \models \exists x \varphi[\sigma]$ and $M \not\models \psi[\sigma]$;
2. the first fact gives an object m of M with $M \models \varphi[\sigma(m/x)]$ (σ changed so that x denotes m);
3. let $\sigma' = \sigma(m/x)$. Since x is not free in ψ, changing the value of x does not affect ψ, so $M \not\models \psi[\sigma']$;
4. under σ', φ is true and ψ false, so $M \not\models (\varphi \to \psi)[\sigma']$: the premise of the rule is not true under all assignments. Contrapositive: if the premise is, so is the conclusion.

(The notes write $M \models \psi[\sigma(m/x)]$ in step 2: it should be φ.) Note that we needed to

change the object assignment to make this work.

Key result — Completeness (Gödel, 1929)

[Not in the slides: added material]

The converse also holds: if $\Gamma \models \varphi$ then $\Gamma \vdash \varphi$. So for FOL, "provable" and "true in all models" coincide. This does not contradict undecidability: a proof always *exists* for valid formulas, but there is no algorithm that also tells you, in finite time, when *no* proof exists.

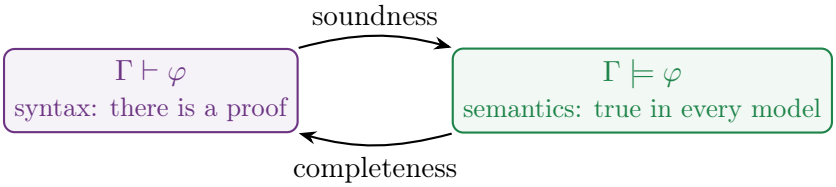


Figure 11: Soundness: the rules never prove something false. Completeness: every truth that follows has a proof.

3.5.3 Sequent calculus LK

[Slides · Proof theory (FOL): 2–4]

A second style of proof system, invented by Gentzen: the **sequent calculus**. A **sequent** has the form

$$\Gamma \to \Delta \qquad (\Gamma, \Delta \text{ finite lists of formulas})$$

and means: *if all formulas in* Γ *are true, then at least one formula in* Δ *is true.* LK (from the German *Logischer Kalkül*) has the opposite philosophy of Frege systems: *one axiom, many rules.*

Variable conventions. $a, b, c, \ldots$ are used only for *free* variables and $x, y, z, \ldots$ only for *bound* variables. Consequence: a sub-formula of a formula is not always a formula. In $\forall x\,(x \leq a)$, the part $x \leq a$ is a *semi-formula* (x is not bound there); similarly $x + a$ is a *semi-term*. With these conventions any term t is always substitutable for a free variable a in a formula φ.

Key result — LK (page 3)

Initial sequents: $A \to A$, for any formula A.
Rules: all rules of the propositional sequent calculus, plus four quantifier rules:

$$\forall\text{-right}\ \frac{\Gamma \to \Delta, A(b)}{\Gamma \to \Delta, \forall x A(x)} \qquad \exists\text{-right}\ \frac{\Gamma \to \Delta, A(t)}{\Gamma \to \Delta, \exists x A(x)}$$

$$\exists\text{-left}\ \frac{A(b), \Gamma \to \Delta}{\exists x A(x), \Gamma \to \Delta} \qquad \forall\text{-left}\ \frac{A(t), \Gamma \to \Delta}{\forall x A(x), \Gamma \to \Delta}$$

In $\forall$-right and $\exists$-left the free variable b is an **eigenvariable**: it must not appear

anywhere in the lower sequent (the substitution x/b "captures" all occurrences of b). In $\exists$-right and $\forall$-left, t is *any* term and may occur in the lower sequent.

Intuition — Eigenvariable = "an arbitrary object we know nothing about"

- $\forall$-*right*: to prove "for all x, $A(x)$", prove $A(b)$ for a *fresh* b about which the hypotheses say nothing. Like in maths: "let b be an arbitrary natural number…". If b appeared in Γ (e.g. Γ says $b = 0$), it would not be arbitrary.
- $\exists$-*left*: if we know "some x has A", we may give it a fresh name b, but we cannot assume it is any object we already know.
- $\exists$-*right* and $\forall$-*left*: to show "something has A", exhibit any witness t; to use "everything has A", instantiate with any t.

3.5.4 LKe: adding equality

[Slides · Proof theory (FOL): 4]

LK has no rules for $=$. **LKe** adds five initial sequents (for any terms s, t, u, s_i, t_i):

- $\to s = s$
- $s = t \to t = s$
- $s = t,\ t = u \to s = u$
- $s_1 = t_1, \ldots, s_k = t_k \to f(s_1, \ldots, s_k) = f(t_1, \ldots, t_k)$ for any *function symbol* f of arity k
- $s_1 = t_1, \ldots, s_k = t_k,\ P(s_1, \ldots, s_k) \to P(t_1, \ldots, t_k)$ for any relation P of arity k

(The notes write "for any formula f": f is a function symbol. The last sequent is printed with a second arrow; the version above is the intended one.)

Worked example — proving symmetry of equality in LKe (page 4)

Goal: $\to \forall x \forall y\,(x = y \to y = x)$. Read the proof bottom-up as a plan, top-down as a derivation:

$$\dfrac{\dfrac{\dfrac{a = b \to b = a}{\to a = b \to b = a}\,{\to}\text{-right}}{\to \forall y\,(a = y \to y = a)}\,\forall\text{-right }(b \text{ fresh})}{\to \forall x \forall y\,(x = y \to y = x)}\,\forall\text{-right }(a \text{ fresh})$$

1. Start from the LKe initial sequent $a = b \to b = a$ (symmetry for the terms a, b).
2. $\to$-right (a propositional rule: from $A, \Gamma \to \Delta, B$ infer $\Gamma \to \Delta, A \to B$) moves $a = b$ to the right as an antecedent.
3. $\forall$-right with eigenvariable b: b does not occur in the lower sequent $\to \forall y\,(a = y \to y = a)$, so the condition holds.
4. $\forall$-right again with eigenvariable a.

3.5.5 Natural deduction rules for quantifiers

[Not in the slides: added material]

To connect with the natural deduction of Section 2.4, here are the four quantifier rules in the same Intro/Elim style (same ideas as LK):

$\forall$ Elim	from $\forall x\,\varphi(x)$ write $\varphi(t)$ for any term t (instantiation)
$\forall$ Intro	if you derived $\varphi(b)$ for a fresh b appearing in no open assumption, write $\forall x\,\varphi(x)$
$\exists$ Intro	from $\varphi(t)$ write $\exists x\,\varphi(x)$
$\exists$ Elim	from $\exists x\,\varphi(x)$, assume $\varphi(b)$ with b fresh in a subproof; if you derive ψ (not containing b), write ψ outside

Worked example — the monitor argument, proved

1	$\forall x\,(\mathit{Monitor}(x) \to \mathit{Ready}(x))$	Premise
2	$\mathit{Monitor}(X12)$	Premise
3	$\mathit{Monitor}(X12) \to \mathit{Ready}(X12)$	$\forall$ Elim: 1
4	$\mathit{Ready}(X12)$	$\to$ Elim: 3, 2

The argument of Section 3.1, which propositional logic could not capture, becomes a two-step proof.

Cheat sheet — First-order logic

- **Terms** denote objects: constants (*socrates*, 8), variables (x), functional terms $f(t_1, \ldots, t_n)$ (*plus*$(2, 3)$, *mother*(x)). A term is never true or false.
- **Atoms** (predicates): $P(t_1, \ldots, t_n)$, true or false under an interpretation. Arity 1 = *property*; arity 0 = *primitive proposition*; an atom without variables is a *proposition*.
- Formulas: atoms combined with $\neg, \wedge, \vee, \to, \leftrightarrow$ and quantifiers $\forall x$ (for all), $\exists x$ (there exists), $\exists_1 x$ (exactly one).
- Translation pattern: "all A are B" $= \forall x\,(A(x) \to B(x))$; "some A is B" $= \exists x\,(A(x) \wedge B(x))$.
- $\forall$ over a finite set = big $\wedge$; $\exists$ = big $\vee$. Duality: $\forall x\,P(x) \equiv \neg\exists x\,\neg P(x)$, $\exists x\,P(x) \equiv \neg\forall x\,\neg P(x)$.
- **Valid** = true under all interpretations. Validity in FOL is **undecidable** (no algorithm always answers yes/no).
- $\Gamma \vdash \varphi$: φ is provable from Γ (syntax, rules). $\Gamma \models \varphi$: φ is true in every model of Γ (semantics). **Soundness**: $\Gamma \vdash \varphi \Rightarrow \Gamma \models \varphi$.
- Frege/Hilbert system: many axioms + Modus Ponens + two quantifier rules. Sequent calculus LK: one axiom $A \to A$ + many rules; $\forall$-right and $\exists$-left need a fresh *eigenvariable*.
- Socrates (Whiteboard 2): instantiate $\forall x(\mathit{human}(x) \to \mathit{mortal}(x))$ with $x = \mathit{socrates}$, then Modus Ponens.

Part III

Agents and Multi-Agent Systems

4 Agents and multi-agent systems

[Slides · Intro to agents & MAS: 1–66]

Part I defined AI as the study of *intelligent agents*. This chapter makes the word *agent* precise, defines *multi-agent systems* (MAS), explains the three ways in which MAS are used (to *design* software, to *implement* it, to *simulate* the world), and tells the history of the idea, from AI planning and objects to intentional systems, modal logic, the Contract Net Protocol and actors.

4.1 What is an agent?

[Slides · Intro to agents & MAS: 5–11]

There is no single definition of agent, but two families of definitions are standard.

4.1.1 The weak definition

[Slides · Intro to agents & MAS: 5–8]

Definition — Agent, weak notion (Jennings, Sycara, Wooldridge, 1998)

An agent is a hardware or software system that is

- **situated**,
- **autonomous**,
- **flexible**, which means:
  - **reactive**,
  - **proactive**,
  - **social**.

The slides only list the words; here is what each one means (from the 1998 paper by N. Jennings, K. Sycara and M. Wooldridge, whose photos are on slide 8).

Property	Meaning	Robot vacuum cleaner
situated	it lives *in* an environment: it receives sensory input from it and acts to change it	senses walls, dirt and stairs; moves and sucks
autonomous	it acts without the direct intervention of humans or other agents, and has control over its own actions and internal state	nobody drives it: it decides where to go next
reactive	it perceives the environment and responds *in a timely fashion* to the changes that occur in it	a chair is moved: it turns to avoid it
proactive	it does not simply act in response to the environment: it exhibits *goal-directed* behaviour and takes the initiative	when the battery is low, it goes back to the dock on its own initiative
social	it interacts with other agents (and humans) to complete its tasks and to help others	coordinates with the mop robot so they don't clean the same room

Table 10: The properties of the weak definition, explained with one everyday agent.

Intuition — The hard part is the balance

Being only reactive is easy (a thermostat), being only proactive is easy (a program that blindly executes a plan). The difficulty, and the reason why agent architectures exist (Section 5), is to be *both*: pursue goals over time, while still reacting quickly when the world changes and the plan no longer makes sense.

Worked example — is it an agent?

[Not in the slides: added material]

System	Verdict
a pocket calculator	no: not situated in an environment it perceives, no autonomy, no goals of its own
a thermostat	a degenerate agent: situated, autonomous, reactive, but not proactive nor social
a spam filter	situated (in the mail stream), autonomous and reactive; weakly proactive
a negotiating bot on an e-commerce platform	yes: it pursues the goal of buying cheaply (proactive), reacts to offers (reactive) and negotiates with sellers' bots (social)

4.1.2 The strong definition

[Slides · Intro to agents & MAS: 9–11]

Besides the properties of the weak definition, an agent may be conceptualised following an **anthropomorphic** approach (the **strong** definition): we describe and build it using notions that we normally apply to humans.

Definition — Agent, strong notion

- **Mentalistic notions** (Y. Shoham, *Artificial Intelligence* 1993; A. S. Rao and M. P. Georgeff, KR&R 1992): the agent has *beliefs*, *desires*, *intentions*, *commitments*, ...
- **Emotional notions** (J. Bates, *Communications of the ACM* 1994): friendliness, trust, distrust; agents must be *believable*.

Worked example — a strong agent: a personal travel assistant

[Not in the slides: added material]

- *belief*: "the 8:15 train to Milan is cancelled";
- *desire*: "my user should be in Milan before 11:00";
- *intention*: "book the 8:45 train" (the desire it committed to, with a plan);
- *commitment*: having promised the user to handle the trip, it keeps trying alternatives (a bus, a car-sharing ride) until the trip is booked or impossible.

Beliefs, desires and intentions are exactly the three components of the **BDI** architecture (Section 8).

Why this matters in practice — Believable agents

Emotional notions matter wherever agents interact with people: virtual characters in video games and training simulations, conversational assistants, social robots in elderly care. A character that reacts with plausible emotions is "believable", and users trust it (or distrust it) accordingly.

4.2 What is a multi-agent system?

[Slides · Intro to agents & MAS: 12–17]

A single agent is rarely enough: interesting problems require several agents, each with its own goals and abilities, that interact. Such a system is a **multi-agent system** (MAS). Slide 12 shows the classic picture, redrawn in Figure 12.

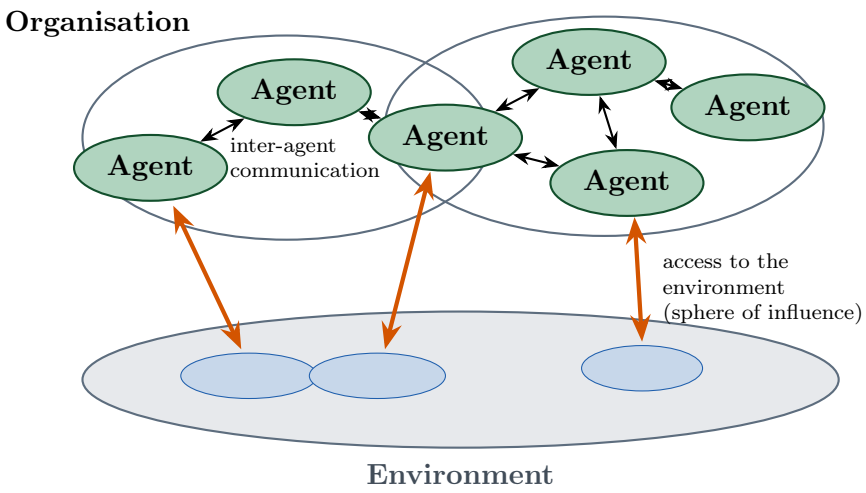


Figure 12: Structure of a MAS (slide 12). Agents (green) communicate with each other, are grouped in organisations (grey ellipses) and each one can perceive and act on its own portion of the shared environment (blue regions, orange arrows). Spheres of influence may overlap: two agents may act on the same part of the environment.

Definition — Characteristics of a MAS (slides 14–17)

In a multi-agent system:

1. each agent has **incomplete information**, or capabilities for solving the problem, thus each agent has a **limited viewpoint**;
2. there is **no global system control**;
3. data is **decentralised**;
4. computation is **asynchronous**.

Worked example — a smart-city ride-sharing MAS

[Not in the slides: added material]

Passenger agents (one per user app), driver agents (one per car) and a pricing agent.

1. *Limited viewpoint*: a driver agent knows its own position and route, not the positions of the other cars or the requests of all passengers.
2. *No global control*: no central program decides every move; drivers accept or refuse rides.
3. *Decentralised data*: each phone stores its own user's preferences and history.
4. *Asynchronous computation*: requests and answers arrive at any time; nobody waits for a global "clock tick".

The same four features describe e-commerce marketplaces, power grids with home batteries, fleets of warehouse robots and swarms of drones.

Intuition — Why "no global control" is a feature, not a bug

A central controller is a single point of failure and a bottleneck, and in an *open* system (agents of different owners, joining and leaving at any time) it simply cannot exist: nobody owns all the agents. MAS replace central control by *interaction*: negotiation, coordination, competition. This is why agent *communication* (Section 9) is a central topic of the course.

4.3 Three ways of using MAS

[Slides · Intro to agents & MAS: 18–41]

Agents and MAS can enter a software project at three different levels:

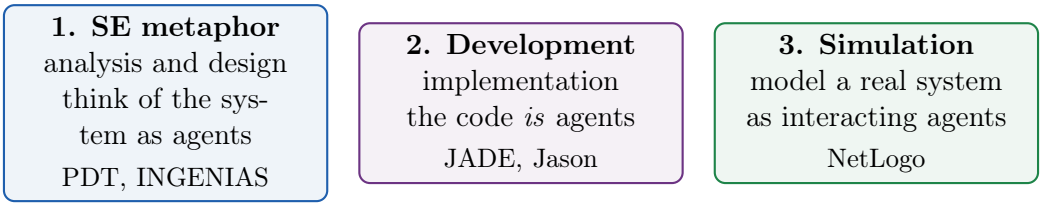


Figure 13: The three uses of MAS (slides 18–41). They can be combined: e.g. design with agents and implement with mainstream technologies.

4.3.1 MAS as a software engineering metaphor

[Slides · Intro to agents & MAS: 18–21]

Key result — Zambonelli and Omicini (JAAMAS, 2004)

"...today's software engineering approaches are increasingly adopting abstractions approaching that of agent-based computing. This trend can be better understood by recognising that *the vast majority of modern distributed systems scenarios are intrinsically prone to be developed in terms of MASs*, and that modern distributed systems are already de facto MASs, i.e., they are indeed composed of autonomous, situated, and social components."

In other words: even if you never write a line of "agent code", thinking in terms of agents (who are the autonomous components? what are their goals? how do they interact?) helps in the *analysis* and *design* of distributed systems. Tools exist to support this: **PDT** (Prometheus Design Tool, RMIT University) and **INGENIAS**, which let you draw agents, goals, messages and data stores as diagrams. Methodologies are the topic of Section 6.

4.3.2 Agent-based development

[Slides · Intro to agents & MAS: 22–36]

Definition — Agent-based development (slides 22–23)

Agents are employed as the technology for **implementing the software application itself**. This choice is usually made when

- the agent metaphor was already applied in the earlier engineering steps (analysis, design);
- the intelligence of the *individual* agent is more relevant than the "emerging intelligence" of the overall system;
- complex patterns of interaction may be required;
- the application is geographically **distributed**, **decentralised**, or **open**.

Two agent platforms are shown: **JADE** (Java Agent DEvelopment framework, Section 10), whose sniffer tool draws the messages exchanged by agents as a sequence diagram (slide 24), and **Jason** (slide 25, an interpreter of AgentSpeak, see the book *Programming multi-agent systems in AgentSpeak using Jason*; Section 8). (Slide 25 is labelled "Jade" but refers to Jason.)

Watch out — Why industry rarely uses agent platforms (slides 26–27)

Many agent-based development tools exist, but they are *not commercial projects supported by large and solid companies*. This is one of the reasons for the limited adoption of this technology by industry. A pragmatic answer: use technologies that are not officially tagged as "agent-oriented", but are very suitable for implementing software agents, in the *implementation* stage, and a "pure" agent approach for the *modelling and design* stages.

The slides then list research works that follow this idea, mapping agents onto mainstream technologies over the years:

Year	Technology	Work shown on the slide
2009	service-oriented computing	*Agents and Service-Oriented Computing for Autonomic Computing: A Research Agenda*
2011	cloud	*Software Agents as Cloud Computing Services*; *Cloud Computing and Software Agents: Towards Cloud Intelligent Services*
2013	web services	JADE Web Services Integration Gateway (WSIG)
2013	Internet of Things	*From Internet of Things to Internet of Agents*
2015	Internet of Things	*Agents of Things (AoT): an intelligent operational concept of the IoT*
2018	IoT and the Web	EMAS 2018 keynote: *Autonomous Agents for Flexible Hypermedia Systems*
2023	virtual reality	*VEsNA*: virtual environments via natural language agents, applied to factory automation (Gatti, Mascardi)
2024	extended reality	XRIA 2024: first workshop on "eXtended Reality & Intelligent Agents" at ECAI

Table 11: Agents implemented with mainstream technologies (slides 28–36).

Why this matters in practice — Agents everywhere, under other names

A microservice that watches a queue, decides and calls other services is an agent in disguise; so is an IoT device that negotiates energy prices, or a non-player character in a VR training environment that understands spoken commands. The agent *concepts* (autonomy, goals, interaction) transfer to these technologies even when no agent platform is used.

4.3.3 MAS for simulation

[Slides · Intro to agents & MAS: 37–41]

Key result — Agent-based modelling and simulation (Macal and North, 2007)

"Agent-based modeling and simulation (ABMS) is a new approach to modeling systems comprised of autonomous, interacting agents. ABMS promises to have far-reaching effects on the way that businesses use computers to support decision-making and researchers use electronic laboratories to support their research."
"Some have gone so far as to contend that ABMS is a *third way of doing science*, in addition to traditional deductive and inductive reasoning. [...] Applications range from modeling agent behavior in the stock market, supply chains, and consumer markets, to predicting the spread of epidemics, the threat of bio-warfare, and the factors responsible for the fall of ancient civilizations."

The tool shown on slide 41 is **NetLogo**, a programmable environment where thousands of simple agents ("turtles") move on a grid of "patches" and the global behaviour is observed in real time. ABMS is the topic of Section 11.

Intuition — Deduction, induction, simulation

Deduction: from general laws to particular consequences (logic, Part II). Induction: from many observations to general laws (statistics, machine learning). Simulation: write the *local* rules of each individual, run them, and *observe* what global phenomenon emerges (a traffic jam, an epidemic wave, a market crash). That is the "third way".

4.4 A short history of agents and MAS

[Slides · Intro to agents & MAS: 42–66]

Current interest in autonomous agents did not emerge from a vacuum. The main contributing disciplines are **artificial intelligence**, **object-oriented programming** (and concurrent object-based systems) and **human-computer interface design** (slide 43).

4.4.1 From artificial intelligence

[Slides · Intro to agents & MAS: 44–47]

1961–71 AI **planning**: Newell and Simon (1961), Fikes and Nilsson's STRIPS (1971). Until the 1980s, AI studies the components of intelligence (learning, reasoning, vision…) *in isolation*; expert systems are "disembodied intelligence": no real-time capabilities, no social ability.

1987 Disillusion with **first-principle planning** (Chapman): building a new plan from scratch for every goal is too slow to be reactive ("calculative rationality") and undecidable in many cases. Georgeff and Lansky propose the **BDI** architecture for practical reasoning.

1991 Brooks: **behavioural** or **reactive AI**, intelligence without symbolic representation.

1992–94 **Hybrid architectures**, usually stratified in layers: TouringMachines (Ferguson, 1992), InteRRaP (Müller and Pischel, 1994).

1993 Shoham, *Agent-oriented programming*: agents are not just a branch of AI, but an autonomous technological and methodological approach. Shoham is considered the father of agent-oriented programming.

Figure 14: From AI to agents (slides 44–47). The architectures named here (reactive, deliberative, hybrid, BDI) are the subject of Section 5.

Definition — Practical reasoning agents (slide 46)

Agents whose architecture is modelled on, or inspired by, a theory of **practical reasoning** in humans: the kind of pragmatic reasoning that we use to decide *what to do* (as opposed to theoretical reasoning, which decides what is true). The most widely known practical reasoning architecture is **Belief-Desire-Intention** (BDI).

4.4.2 From object-oriented programming

[Slides · Intro to agents & MAS: 48–49]

Objects are computational entities that *encapsulate some state*, are able to *perform actions* (methods) on this state, and *communicate by message passing*. This sounds very much like an agent. So, what is the difference?

	Object	Agent
Autonomy	controls its *state* (private fields), but not its *behaviour*: if another object calls a public method, the method runs	controls its state *and* its behaviour: a request is a message the agent may refuse
Motto	"Objects do it for free; agents do it for money."	
Flexibility	the standard object model says nothing about how to build systems that integrate proactive, reactive and social behaviour	reactive, proactive and social behaviour are part of the model
Thread of control	usually passive: acts only when called	has its own thread of control: always running, takes the initiative

Table 12: Agents vs. objects (slide 49; last row added).

Worked example — the same request, two worlds

[Not in the slides: added material]

Object: `account.withdraw(100)` *always* executes the method (perhaps throwing an exception). Agent: Alice's banking agent *receives* the message "please transfer 100 euro to Bob", checks its own goals ("keep at least 500 on the account", "do not pay unknown parties") and may answer "refuse". One could argue that we can program this with objects too; the point (slide 49) is that the object model has nothing to say about it, while for agents it is the core of the abstraction.

4.4.3 From human-computer interaction

[Slides · Intro to agents & MAS: 50]

Instead of *direct manipulation* (the user does everything by clicking), we would like programs that behave like **"digital butlers"** (N. Negroponte, *Being Digital*, 1995):

- "The agent answers the phone, recognizes the callers, disturbs you when appropriate, and may even tell a white lie on your behalf."
- "If you have somebody who knows you well and shares much of your information, that person can act on your behalf very effectively. If your secretary falls ill, it would make no difference if the temping agency could send you Albert Einstein. This issue is not about IQ. It is shared knowledge and the practice of using it in your best interests."
- "Like an army commander sending a scout ahead … you will dispatch agents to collect information on your behalf."

Thirty years later, personal assistants and AI agents that read your e-mail and book your travel are trying to fulfil exactly this vision.

4.4.4 Agents as intentional systems

[Slides · Intro to agents & MAS: 51–60]

When explaining human activity we often say things like:

- Janine took her umbrella because she *believed* it was going to rain;
- Michael worked hard because he *wanted* to possess a PhD.

These statements use a **folk psychology**: human behaviour is predicted and explained by attributing attitudes such as believing, wanting, hoping, fearing. These attitudes are called the **intentional notions**.

Definition — Intentional system (D. C. Dennett, *The Intentional Stance*, 1989)

An **intentional system** is a system made up of entities whose behaviour can be predicted by the method of attributing *beliefs*, *desires* and *rational acumen*.

Intuition — The intentional stance in practice

You do not know the code of a chess program, but you predict its moves by saying "it *wants* to win and *believes* my queen is unprotected, so it will take it". That works much better than reasoning about transistors. Taking the intentional stance towards a system is useful when it is too complex to explain mechanically, yet behaves rationally.

If we adhere to the *strong* definition of agents, intentional systems are a suitable theory for agents (slides 56–60). Representing intentional notions raises delicate syntactic and semantic questions, and the usual answers are:

- **modal logic** languages are suitable for *specifying* agents as intentional systems;
- languages based on **computational (modal) logic** are suitable for *programming* agents;
- axiomatised logic-based languages can undergo **axiomatic verification**; other languages can be used for **model checking**.

Definition — Modal logic (slides 61–63)

Modal logic extends classical logic with a new connective $\Box$ (**necessity**) and its derivable counterpart $\Diamond$ (**possibility**).

- $\Box p$: p is *necessarily* true, i.e. true in every possible scenario ("possible world");
- $\Diamond p$: p is *possibly* true, i.e. true in at least one possible scenario.

$\Diamond$ can be defined from $\Box$: $\Diamond p \leftrightarrow \neg\Box\neg p$.

"p is possible" = "it is not necessary that p is false". It is the same duality as $\exists$ and $\forall$ (Section 3.3): $\Box$ is a "for all worlds", $\Diamond$ a "there exists a world".

Modal logic	$\Box p$ reads	Example
epistemic	the agent knows/believes p	$B_{robot}\ door_open$: the robot believes the door is open
temporal	p always holds (in the future)	$\Box\neg collision$: the drones never collide
deontic	p is obligatory	$O(pay_taxes)$; $\Diamond$ becomes "permitted"
dynamic	after every execution of action a, p holds	$[open]\ door_open$

Table 13: Kinds of modal logics (slide 64; readings and examples added). Combinations exist: **BDI logic** combines beliefs, desires, intentions and time; **KARO** combines knowledge, abilities, results and opportunities.

4.4.5 Two ancestors of MAS: Contract Net and Actors

[Slides · Intro to agents & MAS: 65–66]

Definition — Contract Net Protocol (Davis and Smith, 1983)

Agents can dynamically take two **roles**: **manager** or **contractor**.

1. Given a task, an agent first determines whether it can break it into subtasks that could be performed concurrently.
2. As *manager*, it uses the protocol to **announce** the tasks that could be transferred, and requests **bids** from nodes that could perform them.
3. A node that receives an announcement replies with a **bid**, indicating how well it thinks it can perform the task.
4. The manager collects the bids and **awards** the task to the best bidder, which becomes a *contractor*.

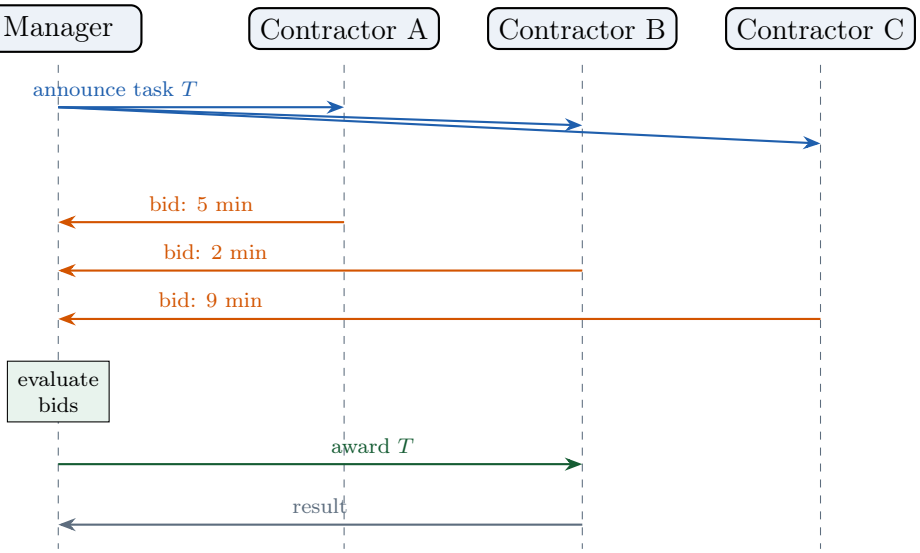


Figure 15: A run of the Contract Net Protocol: one announcement, three bids, the task is awarded to B (the best bid). The same protocol appears in FIPA as an interaction protocol (Section 9).

Watch out — Slide 65: who collects the bids?

The slide says "*the contractor* collects the bids and awards the task to the best bidder". It is the **manager** who announces the task, collects the bids and awards the task; the winner then becomes the contractor.

Why this matters in practice

Contract Net is everywhere where work is distributed by auction: a cloud scheduler choosing the server that can run a job fastest, a logistics platform assigning a delivery to the nearest courier, a factory controller assigning a part to the least loaded machine.

Definition — Actors (Hewitt, Bishop, Steiger, IJCAI 1973; Agha, 1986)

Actors were proposed as universal primitives of concurrent computation: self-contained, interactive, autonomous components of a computing system that communicate by **asynchronous message passing**. The basic actor primitives are:

- **create**: create an actor from a behaviour description and a set of parameters (possibly including existing actors);
- **send**: send a message to an actor;
- **become**: change an actor's local state.

Intuition — Actors today

Each actor has a mailbox, processes one message at a time, and can only affect others by sending messages: no shared memory, so no locks. Erlang (used for telecom switches and WhatsApp's servers) and the Akka framework are built on actors. Agents add to actors *goals* and *reasoning*; the messaging core is the same, and JADE agents (Section 10) work exactly like this.

Cheat sheet — Agents and MAS

- **Weak definition** (Jennings, Sycara, Wooldridge 1998): an agent is a hardware or software system that is **situated**, **autonomous** and **flexible**, i.e. **reactive**, **proactive** and **social**.
- **Strong definition**: anthropomorphic; *mentalistic* notions (beliefs, desires, intentions, commitments: Shoham 1993, Rao & Georgeff 1992) and *emotional* notions (believable agents: Bates 1994).
- **MAS**: each agent has incomplete information/capabilities (limited viewpoint); no global control; decentralised data; asynchronous computation. Agents live in organisations and act on a shared environment.
- Three uses: MAS as a **software-engineering metaphor** (analysis/design:

PDT, INGENIAS); **agent-based development** (implementation: JADE, Jason; or mainstream tech such as SOC, cloud, IoT, VR/XR); **agent-based simulation** (ABMS, NetLogo).
- Roots: AI (planning → reactive (Brooks) → hybrid → BDI practical reasoning; Shoham 1993 = agent-oriented programming), OOP, HCI ("digital butlers").
- **Agents vs. objects**: objects control their state, agents also control their *behaviour* ("objects do it for free, agents do it for money") and integrate reactive, proactive and social behaviour.
- **Intentional stance** (Dennett): predict behaviour by attributing beliefs, desires, rationality. Specified with **modal logics** (□ necessity, ◊ possibility, $\Diamond p \leftrightarrow \neg\Box\neg p$).
- **Contract Net** (1983): manager announces task, contractors bid, manager awards to the best bid. **Actors** (1973): create, send, become; asynchronous message passing.

5 Agent architectures

[Slides · Agent architectures: 1–41]

How is an agent built *inside*? This chapter (slides from a course by URV, Tarragona, with comments by prof. Mascardi) first looks at what an agent must cope with, namely its *environment*, and then at the three main families of **agent architectures**:

1. **reactive**: fast stimulus–response rules, no model of the world (Brooks);
2. **deliberative**: an explicit symbolic model of the world and a planner;
3. **hybrid**: a reactive layer and a deliberative layer together.

The BDI architecture of Part V is a practical-reasoning architecture that combines deliberation (goals, plans) with reactivity (events), in the spirit of the hybrid approach.

5.1 Agents and their environments

[Slides · Agent architectures: 1–10]

5.1.1 Three classic definitions of agent

[Slides · Agent architectures: 3–5]

The slides recall three classic definitions (the sources are added):

1. "An agent is anything that can be viewed as *perceiving its environment through sensors* and *acting upon that environment through effectors*." (Russell and Norvig)

2. "Autonomous agents are computational systems that inhabit some complex dynamic environment, sense and act autonomously in this environment, and by doing so realize a set of goals or tasks for which they are designed." (P. Maes)
3. "An autonomous agent is a system situated within and a part of an environment that senses that environment and acts on it, over time, in pursuit of its own agenda and so as to affect what it senses in the future." (Franklin and Graesser)

All three share the **basic abstract view** of slide 4: a loop between agent and environment, through sensors (input) and effectors (output); the same picture as Section 1.2.4. The third definition adds an important nuance: the agent's actions change what it will *perceive in the future*.

Definition — Reactivity (slide 5)

An agent has to be able to **react**, i.e. adapt its behaviour, in an appropriate way to the dynamic changes in its environment: other computational agents, human agents/users, external information sources (e.g. sensors), physical objects (e.g. robots), the Internet, ... Reactivity is one of several properties an intelligent agent should have (Section 4.1).

5.1.2 Kinds of environments

[Slides · Agent architectures: 6–10]

The difficulty of building an agent depends heavily on the environment it lives in. Five dimensions classify environments; in each pair, the first is the *easy* one.

Dimension	Easy case	Hard case
accessible vs. inaccessible	the agent can obtain *complete, accurate, up-to-date* information about the environment's state	most moderately complex environments (the physical world, the Internet) are inaccessible
deterministic vs. non-deterministic	any action has a *single guaranteed effect*: no uncertainty about the resulting state	the physical world is, for all intents and purposes, non-deterministic
episodic vs. non-episodic	performance depends on discrete *episodes* with no link between them: the agent decides based only on the current episode	the agent must reason about how current actions affect future episodes
static vs. dynamic	remains unchanged except by the agent's own actions	other processes operate on it and change it beyond the agent's control; the physical world is highly dynamic
discrete vs. continuous	a fixed, finite number of actions and percepts	the real world is continuous

Table 14: Kinds of environments (slides 6–10). The more an environment is on the left, the simpler it is to build agents for it.

Exercise E.1 — Practical exercise, step 1: think about the kind of environment (slide 10)

Classify these environments along the five dimensions: (a) a chess program (with a clock); (b) an e-mail spam filter; (c) a self-driving taxi; (d) a robot vacuum cleaner; (e) an online poker bot.

Solution

	access.	**determ.**	**episodic**	**static**	**discrete**
(a) chess with clock	yes	yes	no	semi*	yes
(b) spam filter	no	no	yes	no	yes
(c) self-driving taxi	no	no	no	no	no
(d) robot vacuum	no	no	no	no	no
(e) poker bot	no	no	no**	no	yes

Explanations: chess is fully observable and every move has a certain effect; the board does not change while you think, but the clock does (*semi-dynamic). A spam filter classifies each e-mail independently (episodic), but it cannot see the sender's intentions (inaccessible). The taxi is the hardest case on every dimension, like the vacuum robot (the cat moves, the floor is continuous). Poker hides the opponents' cards (inaccessible), the deck is shuffled (non-deterministic), and (**) hands are episodes, but the opponents' model of you carries over from hand to hand. **Lesson**: almost every real application is on the "hard" side, which is why architectures matter.

5.2 What is an agent architecture?

[Slides · Agent architectures: 11–14]

Definition — Agent architecture (slide 11)

An **architecture** proposes a particular methodology for building an autonomous agent:

- how the construction of the agent can be decomposed into the construction of a set of **component modules**;
- how these modules should be made to **interact**.

These two aspects define how the *sensor data* and the *current internal state* of the agent determine the *actions* (effector outputs) and the *future internal state* of the agent.

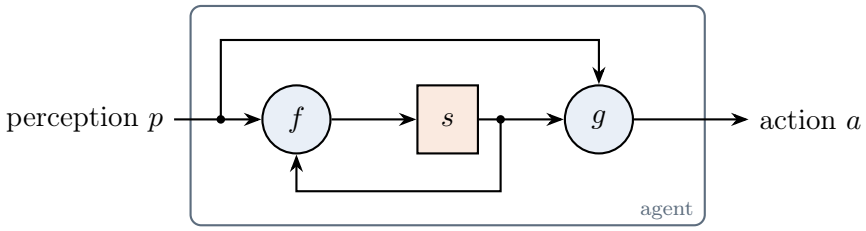


Figure 16: From perception to action (slide 12): f = state update function, s = internal state, g = output function. The new state depends on the old state and the percept; the action depends on the state and the percept.

In formulas, at each step t:

$$s_t = f(s_{t-1},\, p_t) \qquad \triangleleft \text{ update the internal state with what was perceived}$$
$$a_t = g(s_t,\, p_t) \qquad \triangleleft \text{ choose the action}$$

Worked example — a smart thermostat as f, s, g

[Not in the slides: added material]

Percept p_t: the measured temperature. State s_t: the average of the last three temperatures (to ignore a sensor glitch). Output: switch the heater on if the average is below 20 °C.

t	p_t	last three	s_t (average)	$a_t = g(s_t, p_t)$
1	19	21, 21, 19	20.33	off
2	18	21, 19, 18	19.33	on
3	25 (glitch)	19, 18, 25	20.67	off

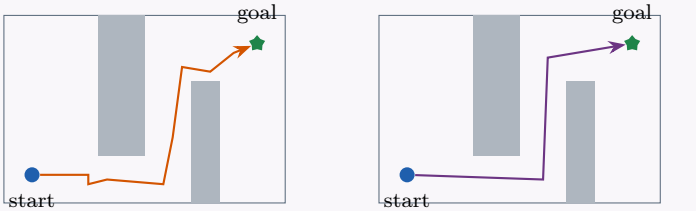
(Initial readings 21, 21.) The internal state is what lets an agent do better than reacting to the last percept only; how much state, and how it is used, is exactly what distinguishes the architectures below.

Key result — Main kinds of agent architectures (slide 13)

- **Reactive** architectures: focused on fast reactions/responses to changes detected in the environment.
- **Deliberative** (symbolic) architectures: focused on long-term planning of actions, centred on a set of basic goals.
- **Hybrid** architectures: combining a reactive side and a deliberative side.

Worked example — reactive vs. deliberative: a robot that has to reach a point (slide 14)

Reactive robot	Deliberative robot
a sensor in the front of the robot	an explicit representation of the environment (a map)
change movement right/left when the sensor detects an obstacle	a planning procedure finds the minimal route between the current position and the destination
minimal computation, based on current location and destination point	high computational cost; possible dynamic re-planning needed



reactive: bumps and turns — deliberative: planned shortest path

Figure 17: Same task, two architectures. The reactive robot computes almost nothing but wanders; the deliberative one finds the best route but must have a correct map and re-plan if a wall moves.

5.3 Reactive architectures

[Slides · Agent architectures: 15–36]

5.3.1 Basic ideas

[Slides · Agent architectures: 15–23]

Definition — Reactive agent (slide 15)

Reactive agents have *at most a very simple internal representation of the world*, but provide a **tight coupling of perception and action**. They follow the **behaviour-based paradigm**: intelligence is a product of the **interaction** between an agent and its environment.

The ant colony (slides 16–18). A single ant has very little intelligence, computing power or reasoning abilities; the union of many ants and their interaction forms a highly complex, structured and efficient system. The slides add pictures of other *swarms*: bees, flocks of starlings, schools of fish and rays, migrating birds, and a *Swarm Intelligence* magazine cover with insect-like robots.

Watch out — Prof. Mascardi's remark on slide 16

"As stressed during the class, an ant colony is an example of **multiagent system**, not of a single agent architecture: I disagree with the contents of this slide." The ant colony shows *emergent* behaviour of many simple agents (the topic of Section 11). An *architecture* describes how *one* agent is built inside. Keep the two levels separate in the exam.

Key result — Main characteristics of reactive architectures (slides 19–21)

1. **Emergent functionality**: simple agents, simple interaction; complex behaviour patterns appear as a result of the dynamic interactions; the global behaviour is *not specified a priori* (e.g. the movement of robots depends on the obstacles they meet).
2. **Task decomposition**: agents are composed of autonomous modules, each managing a given task (sensing, control, computation); minimal, low-level communication between modules; *no global world model*; *no planning/controller agent*.
3. **Raw data**: basic data from sensors, no complex symbolic management of data as in classical AI; refusal of the **physical symbol system hypothesis**, the basic pillar of symbolic AI: "intelligent behaviour can only be obtained in symbol-processing systems".

Definition — Reactive behaviour (slide 22)

Each behaviour continually maps perceptual input to action output. Reactive behaviour is a set of **action rules**

$$action : S \to A$$

where S is the set of states of the environment and A the set of primitive actions the agent can perform. Example (a thermostat):

$$action(s) = \begin{cases} \text{heater off} & \text{if the temperature is OK in state } s \\ \text{heater on} & \text{otherwise} \end{cases}$$

Compared with Section 5.2, a purely reactive agent has *no internal state*: $a_t = g(p_t)$, the action depends only on the current percept.

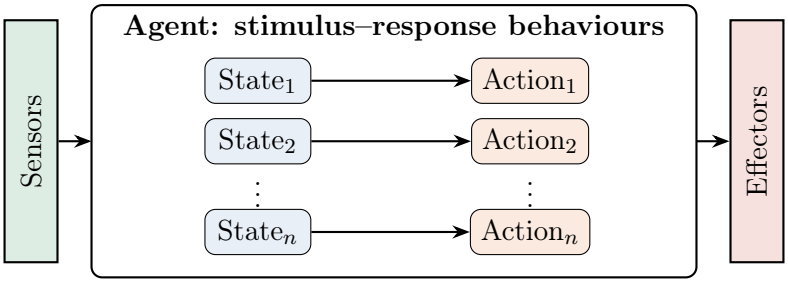


Figure 18: Basic schema of a reactive architecture (slide 23): a table of situation→action rules between sensors and effectors, no world model, no planner.

5.3.2 Rodney Brooks and the subsumption architecture

[Slides · Agent architectures: 24–30]

Rodney Brooks (director of the MIT Computer Science and Artificial Intelligence Lab, 1997–2007, and co-founder of iRobot, the maker of the Roomba vacuum cleaner) is the main critic of symbolic AI from the reactive side.

Key result — Brooks' three theses (slide 25)

1. Intelligent behaviour can be generated **without explicit representations** of the kind that symbolic AI proposes.
2. Intelligent behaviour can be generated **without explicit abstract reasoning** of the kind that symbolic AI proposes (reduced computation on sensor-like data).
3. Intelligence is an **emergent property** of certain complex systems.

Key result — Brooks' key ideas (slides 26–27)

- **Situatedness**: "real" intelligence is situated in the world. "The world is its best model": it is always up-to-date, whereas a model is an abstraction, a simplification that considers some characteristics and disregards others.
- **Embodiment**: "real" intelligence requires a physical body, and cannot be found in disembodied systems such as theorem provers or expert systems (physical robots).
- **Intelligence and emergence**: intelligent behaviour arises from the agent's interaction with its environment; intelligence is "in the eye of the beholder", not an innate, isolated property.

Definition — Subsumption architecture (slides 28–29)

A **subsumption architecture** is a hierarchy of task-accomplishing **behaviours**. Each behaviour is a rather simple rule-like structure. Behaviours *compete* to exercise control over the agent, since several may be applicable at the same time. Behaviours are organised in **layers**:

- lower layers represent more primitive kinds of behaviour (e.g. avoiding obstacles);
- higher layers represent more complex behaviours (e.g. identifying an object);
- lower layers have **precedence** over layers further up the hierarchy.

The resulting systems are, in terms of computation, extremely simple; yet some of the robots do tasks that would be impressive if accomplished by symbolic AI systems.

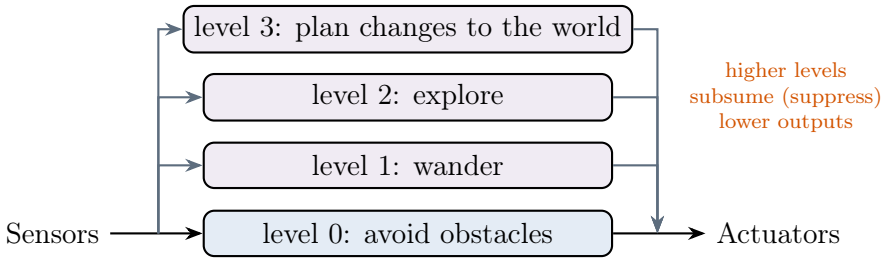


Figure 19: Brooks' layered control system ("However…", slide 30, from Brooks' original paper). Each level is a complete behaviour working directly from sensors to actuators; new levels are added on top of working ones.

Watch out — "However…" (slide 30): who has precedence?

Slide 29 says that *lower* layers have precedence. Slide 30 ("However…") shows the figure of Brooks' original paper, whose caption says that control is layered "with *higher* level layers subsuming the roles of lower level layers when they wish to take control". Both views exist:

- in Brooks' original robots, a higher layer can *suppress* the output of a lower one (that is what "subsume" means): e.g. "explore" overrides "wander";
- in the rule-based presentations (Wooldridge's textbook, Steels' explorer below), the lowest-level behaviour, such as obstacle avoidance, has the *highest priority*, because safety comes first.

In both, each layer is a complete behaviour and the composition of layers produces the overall behaviour without a central planner.

5.3.3 Steels' Mars explorer

[Slides · Agent architectures: 31–33]

Luc Steels' Mars explorer system, using the subsumption architecture, achieves near-optimal cooperative performance in a simulated "rock gathering on Mars" domain: *explore a distant planet and collect samples of a precious rock. The location of the samples is not known in advance, but they tend to be clustered.* The mother-ship emits a radio signal that gets weaker with distance, so "travel up the gradient" (towards a stronger signal) leads back to the base.

Key result — The rules of an individual explorer (slides 32–33), in priority order

(1) **if** detect an obstacle **then** change direction
(2) **if** carrying samples **and** at the base **then** drop samples
(3) **if** carrying samples **and not** at the base **then** travel up gradient
(4) **if** detect a sample **then** pick sample up
(5) **if** true **then** move randomly

Obstacle avoidance (1) is the lowest-level behaviour, hence the one with the highest priority. Rule (5) always applies, so it fires only when nothing better is applica-

ble. At each step the agent fires the *first* rule (smallest number) whose condition holds.

Worked example — a trace of one explorer

[Not in the slides: added material]

Step	Situation	Applicable rules	Fired
1	empty terrain, carrying nothing	(5)	(5) move randomly
2	a rock sample in front	(4), (5)	(4) pick up
3	carrying a sample, far from base	(3), (5)	(3) up gradient
4	carrying, a crater ahead	(1), (3), (5)	(1) change direction
5	carrying, path clear	(3), (5)	(3) up gradient
6	carrying, at the base	(2), (5)	(2) drop samples
7	carrying nothing, at the base	(5)	(5) move randomly

The behaviour "collect rocks and bring them home" was never written anywhere: it emerges from five local rules and the environment. (In the full system, *cooperative* rules make agents drop radioactive "crumbs" on the way back, so that other agents find the clusters; this is how near-optimal cooperation emerges.)

5.3.4 Advantages and limitations

[Slides · Agent architectures: 34–36]

Advantages (slide 34)	Limitations (slides 35–36)
simplicity of individual agents	agents without environment models must have sufficient information available from the *local* environment
flexibility, adaptability: ideal in very dynamic and unpredictable environments	decisions based on the local environment: how to take *non-local* information into account? "short-term" view
computational tractability: no complex planning/reasoning procedures, no continuous model update	no long-term planning capabilities; limited applicability (games, simulations, basic insect-like robots)
robustness against failure: no central planning component (e.g. ant colony)	difficult to make reactive agents that *learn* (dynamic evolution of rules?)
elegance	behaviour emerges from interactions plus environment: hard to *engineer* specific agents (no principled methodology); hard with many behaviours (dynamics too complex to understand)

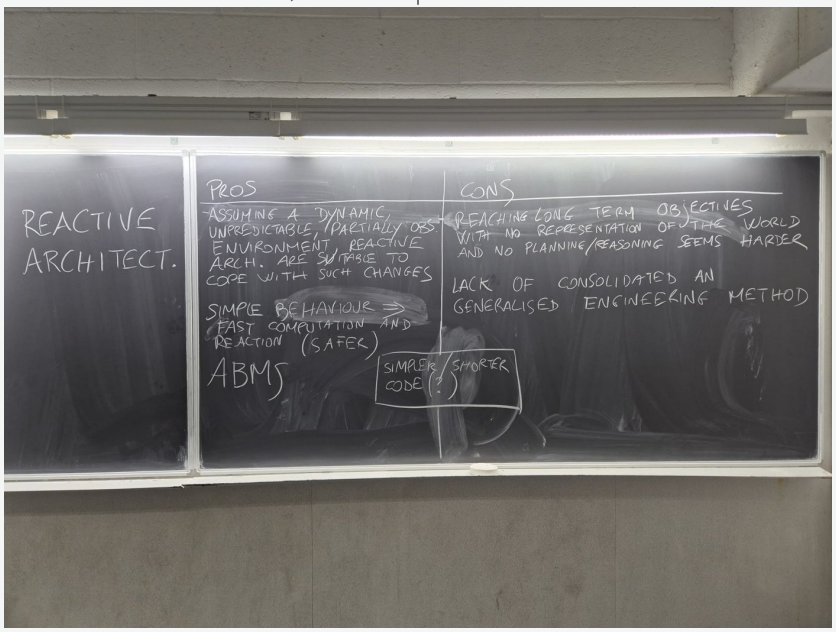
Watch out — Prof. Mascardi's remarks on slide 34

- On *robustness*: "I disagree: an ant colony is a MAS, not a single agent: a reactive architecture is as robust (or not robust) as any other modular software system."

- On *elegance*: "I disagree: elegance is not a scientific, engineering property."

Whiteboard 3 — Reactive architectures: pros and cons written during the agent architectures lecture

PROS	CONS
Assuming a **dynamic, unpredictable, partially observable** environment, reactive architectures are suitable to **cope with such changes**. **Simple behaviour ⇒ fast computation and reaction** (safer). Natural fit for **ABMS** (agent-based modelling and simulation). Simpler / shorter code (?) (written in the middle, with a question mark)	Reaching **long-term objectives** with **no representation of the world** and **no planning/reasoning** seems harder. Lack of a **consolidated and generalised engineering method**.



Original photo of the whiteboard

How to read it: the professor's own summary, more cautious than slide 34.

- The first pro is about the *environment*: it uses the vocabulary of Section 5.1 (dynamic, partially observable = inaccessible).
- "Safer" because a fast reaction (braking, avoiding) can matter more than an optimal but late plan.
- ABMS is listed among the pros: reactive agents are the typical building block of agent-based simulations (Section 11).
- "Simpler/shorter code" is boxed with a *question mark*: individual rules are simple, but a large set of interacting rules is not necessarily simple to write or to understand (compare the second limitation of slide 36).
- The cons match slides 35–36: no long-term view, no principled engineering

method.

5.4 Deliberative and hybrid architectures

[Slides · Agent architectures: 37–41]

5.4.1 Deliberative architectures

[Slides · Agent architectures: 37–39]

Definition — Deliberative agent architecture (slide 37)

- an **explicit symbolic model of the world**;
- decisions are made via **logical reasoning**, based on pattern matching and symbolic manipulation;
- the **sense–plan–act** problem-solving paradigm of classical AI planning systems.

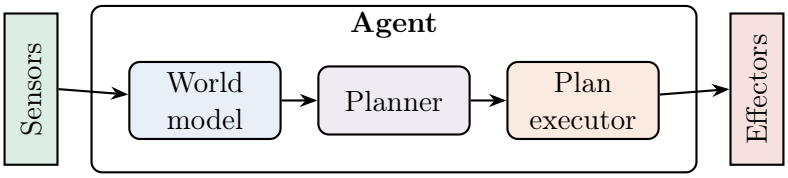


Figure 20: Basic deliberative architecture (slide 38): sense (update the world model), plan, act. Shakey with its STRIPS planner (Section 1.3) is the classic example.

Key result — Problems of the deliberative approach (slide 39)

- **Dynamic world**: the symbolic world model must be updated, and the world changes *while* planning is being done.
- **Representation language**: it must be expressive enough to be useful in any domain, but limited enough to be computationally tractable (recall FOL's undecidability, Section 3.4).
- **Classical planning** looks for complete, optimal solutions: high computational cost, whereas sometimes a sub-optimal, low-cost, fast reaction is effective.

Worked example — when the plan is already wrong

[Not in the slides: added material]

A delivery robot plans the shortest route through a hospital corridor (2 seconds of planning). Meanwhile a bed is pushed into the corridor. The plan was optimal for a world that no longer exists. A purely deliberative robot either drives into the bed or must stop and re-plan every time anything moves. This is the motivation for hybrid architectures.

5.4.2 Hybrid architectures

[Slides · Agent architectures: 40–41]

Many researchers have argued that neither a completely deliberative nor a completely reactive approach is suitable for building agents. They suggest **hybrid systems**, which marry the classical and the alternative approach. The obvious way is to build an agent out of two (or more) subsystems:

- a **deliberative** one, containing a symbolic world model, which develops plans and makes decisions in the way proposed by symbolic AI;
- a **reactive** one, capable of reacting quickly to events without complex reasoning.

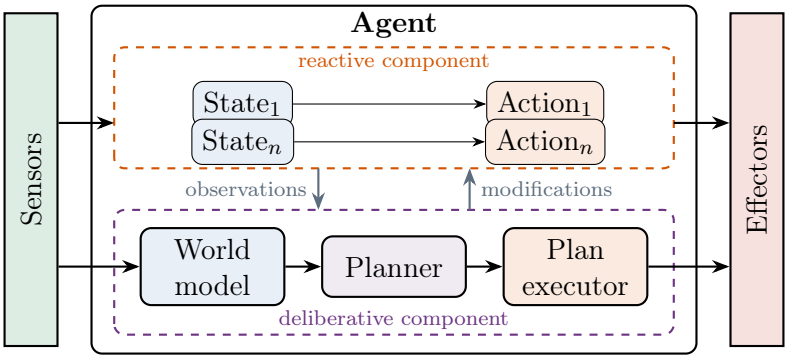


Figure 21: Hybrid agent architecture (slide 41). The reactive layer handles urgent situations immediately; the deliberative layer plans; the two exchange observations and modifications (e.g. the planner can install new reactive rules, the reactive layer reports what it did).

Hybrid architectures are usually **layered**: well-known examples are **TouringMachines** (Ferguson, 1992) and **InteRRaP** (Müller and Pischel, 1994), see Section 4.4; TouringMachines reappear in the EMAS keynote (Section 6).

Why this matters in practice — Hybrid architectures in a self-driving car

[Not in the slides: added material]

The emergency-braking module is reactive: "obstacle closer than 5 m ⇒ brake", in milliseconds, no reasoning. The route planner is deliberative: it uses a map and traffic information and re-plans when a road is closed. A middle layer turns the plan into manoeuvres. Without the reactive layer the car is unsafe; without the deliberative layer it never gets anywhere useful.

Cheat sheet — Agent architectures

- Environments can be: accessible/inaccessible, deterministic/non-deterministic, episodic/non-episodic, static/dynamic, discrete/continuous. The real world is the hardest case on every axis.
- An **architecture** = decomposition of the agent into modules + how they interact; it determines how percepts and internal state produce actions and

the next state: $s' = f(s, p)$, $a = g(s', p)$.
- **Reactive**: rules $S \to A$, no world model, tight perception–action coupling, emergent behaviour. Brooks: intelligence without representation and without abstract reasoning; situatedness, embodiment, emergence; **subsumption** architecture (layers of behaviours competing for control).
- Steels' Mars explorer: 5 prioritised rules (avoid obstacles, drop samples at base, go home up the gradient, pick up samples, move randomly).
- Reactive **pros**: simple, fast, tractable, good in dynamic unpredictable environments. **Cons**: short-term view, no long-term planning, hard to learn, no principled engineering method (Whiteboard 3).
- **Deliberative**: explicit symbolic world model + planner + plan executor (sense–plan–act); problems: world changes while planning, expressiveness vs. tractability, high cost.
- **Hybrid**: reactive component for quick reactions + deliberative component for planning (TouringMachines, InteRRaP).
- Prof. Mascardi's remarks: an ant colony is a MAS, not a single-agent architecture; robustness comes from modularity, not from reactivity; elegance is not an engineering property.

6 Engineering multi-agent systems

[Slides · EMAS keynote: 1–95; AOSE: 1–8]

How do we *engineer* systems made of agents? This chapter follows a keynote given by prof. Mascardi at the European Agent Systems Summer School (EASSS 2024), complemented by the short AOSE deck. It is a map of the field of **Agent-Oriented Software Engineering** (AOSE), also called EMAS (Engineering Multi-Agent Systems):

1. a recap of agents and MAS and a fictional case study, the city of *SteaMas*;
2. why agents are good for software engineering (AOSE);
3. seven research directions: architectures, methodologies and notations, programming languages, infrastructures and toolkits, simulation, symbolic/sub-symbolic integration, safe autonomous systems;
4. the SteaMas design exercise done in class (three whiteboards);
5. conclusions: where are the agents?;
6. engineering *safe* MAS: autonomy, verification techniques, runtime verification.

6.1 Recap and the SteaMas scenario

[Slides · EMAS keynote: 1–15]

The keynote opens with the definitions of Section 4 (slides 2–5): the weak definition of agent (situated, autonomous, flexible: reactive, proactive, social), the strong definition (mentalistic notions: beliefs, desires, intentions, goals, plans), the four

characteristics of a MAS, and the MAS picture of Figure 12 (from Zambonelli and Omicini, 2004). Slide 6 lists further topics of the summer school that belong to the MAS world: people and organisations; reinforcement learning, deep RL, multi-agent RL (MARL) and continual learning; coordination, organisations, institutions, norms and ethics; argumentation; agent-based modelling and simulation.

Worked example — the fictional scenario: SteaMas (slides 7–9)

Dr. Joyce, the new mayor of the (steampunk) city of **SteaMas**, wants to make SteaMas *green, smart and safe* by ensuring that:

1. SteaMas citizens and tourists respect well-defined civil/ethical **norms**;
2. **no private traffic** exists: citizens and tourists may take advantage of different kinds of free public transportation means in the most effective way;
3. both citizens and tourists may be driven around by a **personal drone** provided by the municipality for free.

Dr. Joyce is aware of the existence of agents and MAS, believes that MAS technology can suit his needs, and contacts us to exploit it. The scenario is resumed in Section 6.4, after the tour of the research directions.

Where EMAS research lives (slides 10–15). A Google Scholar search for "agent oriented software engineering" returns hundreds of thousands of results. The main venues: the **EMAS** workshop (International Workshop on Engineering Multi-Agent Systems), born from the merger of three predecessors, AOSE (Agent-Oriented Software Engineering), DALT (Declarative Agent Languages and Technologies) and ProMAS (Programming Multi-Agent Systems), which started around 2000–2003; and the *Engineering and Analysis of Multiagent Systems* area of the AAMAS conference (International Conference on Autonomous Agents and Multiagent Systems), whose topics include requirements and formal specification, architecture and modelling, verification and validation, programming models and languages, testing, fault tolerance and reliability, norms and governance, interaction protocols, BDI-based agents, ethical agents, MAS-based simulations, tools and testbeds, platforms, and applications from finance to health, agriculture and smart cities.

6.2 Agent-oriented software engineering

[Slides · AOSE: 1–8]

[Moved here from the AOSE deck, shown right after the EMAS keynote]

The short AOSE deck (based on Zambonelli and Omicini, *Challenges and Research Directions in Agent-Oriented Software Engineering*, 2004, and Giorgini and Henderson-Sellers, *Agent-Oriented Methodologies: An Introduction*, 2005) explains *why* agents are interesting for software engineering at all.

Key result — Why agents suit modern software (slides 4–5)

The features of agent-based systems are well suited to tackle the complexity of developing software in modern scenarios:

1. the **autonomy** of application components reflects the intrinsically *decentralised* nature of modern distributed systems, and can be considered the natural extension of the notions of *modularity* and *encapsulation* to systems that are owned by different stakeholders;
2. the **flexible** way in which agents operate and interact (with each other and with the environment) suits the *dynamic and unpredictable* scenarios where software is expected to operate;
3. the concept of **agency** provides a unified view of AI results and achievements, making agents and MAS act as sound and manageable *repositories of intelligent behaviours*.

Intuition — Autonomy as "encapsulation of behaviour"

Object orientation encapsulates *state*: nobody outside can change an object's private fields. Agent orientation goes one step further and encapsulates *behaviour*: nobody outside can force an agent to act; you can only ask (Section 4.4). When the components belong to different companies (your bank's agent, a shop's agent, a courier's agent), this is not a design choice but a fact of life.

Definition — Agent-Oriented Software Engineering (slide 6)

With the increasing acceptance of agent-based computing as a novel software engineering paradigm, there has been a great deal of research on suitable models and techniques to support the development of complex software systems in terms of MAS. This research is roughly grouped under the term **Agent-Oriented Software Engineering** (AOSE).

Key result — Mainstream research directions in AOSE (AOSE slide 8; EMAS slide 11)

	AOSE deck (2004 view)		EMAS keynote (2024 view)
1	agent-oriented architectures	1	agent-oriented architectures
2	AOSE methodologies	2	methodologies and notation techniques
3	notation techniques	3	agent-oriented programming languages
4	agent-oriented programming languages	4	MAS infrastructures and toolkits
5	agent-based modelling and simulation	5	agent-based modelling and simulation
6	MAS infrastructures	6	symbolic and sub-symbolic integration
		7	engineering safe autonomous systems

Twenty years later the list is almost the same, plus two new directions: neurosymbolic integration and safety.

(A section of the original AOSE deck, "The promise of AOSE for distributed systems engineering", was removed in the shortened version used in class.)

6.3 Research directions in AOSE

[Slides · EMAS keynote: 16–56]

6.3.1 1. Agent-oriented architectures

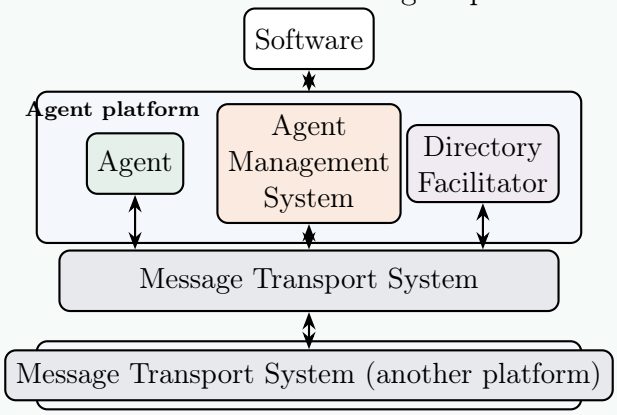
[Slides · EMAS keynote: 16–22]

The three families of Section 5, with their founding papers. The slides illustrate them with SpongeBob pictures; here are the ideas.

- **Belief-Desire-Intention** (Rao and Georgeff, "BDI agents: from theory to practice", ICMAS 1995; plus Georgeff and Lansky 1986, Rao and Georgeff 1992, Shoham 1993). Slide 17 shows the BDI components: *beliefs* (what the agent knows about the environment), *goals*, *plans* (recipes), a *reasoner* that selects plans and turns them into *intentions*, which act on the environment. Studied in depth in Section 8.
- **Subsumption** (Brooks, "A robust layered control system for a mobile robot", 1986). Slide 18: behaviours *Avoid*, *Wander*, *Explore* stacked on top of each other, all connected to the environment (Section 5.3).
- **Hybrid** (Ferguson's TouringMachines, 1992; Müller and Pischel's InteRRaP, 1993). Slides 19–21 picture a hybrid architecture as a layered cake: several horizontal layers, each with its own job, stacked into one agent (Section 5.4).

Key result — FIPA abstract agent platform (slide 22)

FIPA (Foundation for Intelligent Physical Agents) was created in Switzerland in 1996 and accepted as an IEEE standards committee in 2005. It standardises how agents are hosted and communicate. A FIPA agent platform contains:



- **Agent Management System** (AMS): the "white pages": creates and destroys agents, knows every agent's name and address;

- **Directory Facilitator** (DF): the "yellow pages": agents register the *services* they offer and search for providers;
- **Message Transport System**: delivers messages, also to agents on other platforms.

JADE (Section 10) is a FIPA-compliant platform with exactly these components.

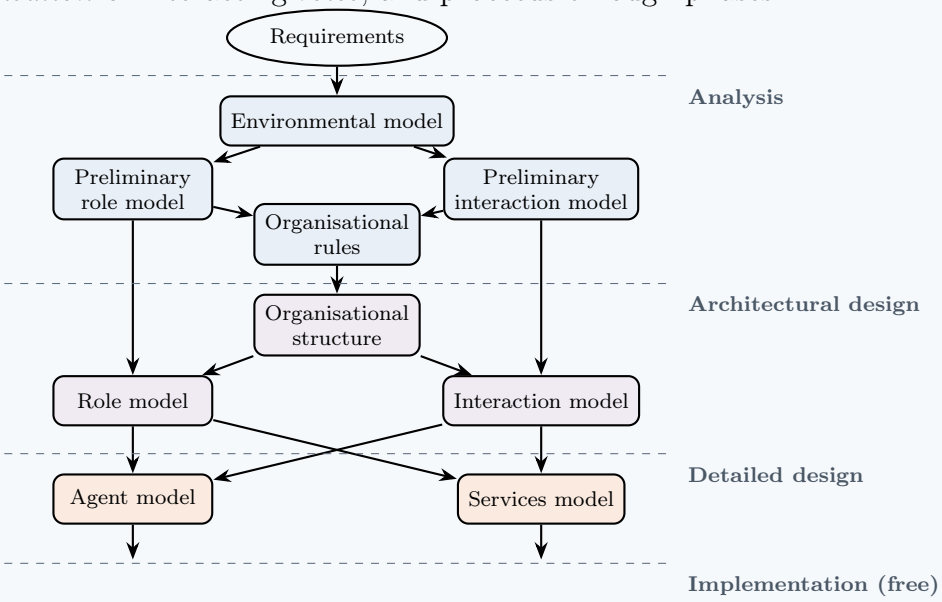
6.3.2 2. AOSE methodologies and notation techniques

[Slides · EMAS keynote: 23–29]

Many agent-oriented methodologies exist (Henderson-Sellers and Giorgini, eds., *Agent-Oriented Methodologies*, 2005). Slide 24 shows their genealogy: most derive from object-oriented methods (OO, RUP, OMT, Fusion, OPEN) or from knowledge engineering (MAS-CommonKADS); among them Gaia, Prometheus, Tropos, MaSE, PASSI, INGENIAS, MESSAGE, Agent OPEN, ADELFE.

Definition — Gaia (Wooldridge, Jennings, Kinny, 2000)

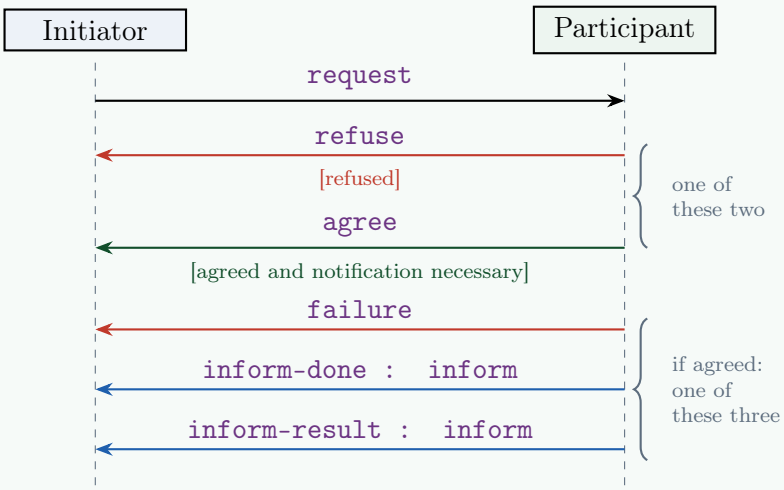
A methodology for agent-oriented **analysis and design**. It sees a MAS as an *organisation* of interacting *roles*, and proceeds through phases:



Gaia deliberately leaves the **implementation** stage free: the design can be implemented with an agent platform or with mainstream technologies (the point made in Section 4.3).

Notations. Agents need diagrams that UML does not offer. **AUML** (Agent UML; Odell, Parunak, Bauer, *Extending UML for agents*, 2000) adds, among other things, **protocol diagrams**; **Prometheus** (Winikoff and Padgham, 2004) has its own icons for agents, actions, messages, data, percepts, protocols, capabilities and plans (slide 29).

Key result — The FIPA Request interaction protocol (slides 26–27: "Remember this protocol!")



1. The initiator sends a `request` (asks the participant to perform an action).
2. The participant answers `refuse` (protocol ends) or `agree`.
3. If it agreed, it performs the action and finally sends one of: `failure` (it tried but failed), `inform-done` (done, nothing to return) or `inform-result` (done, here is the result).

(The `agree` can be skipped if the participant answers immediately with the final message.) This protocol is used again in agent communication (Section 9) and in JADE.

Worked example — the Request protocol in a pizzeria

[Not in the slides: added material]

Customer agent: `request`("deliver one margherita"). Pizzeria agent: `agree` ("ok, 30 minutes"). Later: `inform-done` ("delivered"); or `failure` ("the scooter broke down"). If the pizzeria were closed: `refuse` ("closed on Mondays").

Key result — FIPA ACL message structure (slide 28)

Parameter	Category
performative	type of communicative act
sender, receiver, reply-to	participant in communication
content	content of message
language, encoding	description of content

Performative	Meaning
`REQUEST`	the sender asks the receiver to perform some action
`AGREE`	the sender agrees to perform the requested action
`REFUSE`	the sender refuses to perform the requested action
`NOT_UNDERSTOOD`	someone participating in the conversation didn't understand something
`INFORM`	a positive reply because an action is performed, or the answer to some previous question
`FAILURE`	a failure in performing the requested action happened after an initial agreement

Studied in detail in Section 9.

6.3.3 3. Agent-oriented programming languages

[Slides · EMAS keynote: 30–39]

Logic-based agents (slides 31–32). The "traditional" approach to building artificial intelligent systems: **logical formulae** give a symbolic representation of the agent's environment and desired behaviour, and **logical deduction** (theorem proving) syntactically manipulates this representation. Slide 31's cartoon thinks the Prolog program `mortal(X) :- human(X). human(socrates).` with the query `?- mortal(socrates).` answered `yes` (Whiteboard 2).

AgentSpeak(L) (slides 33–36). The BDI architecture is one of the best known and most studied architectures for *cognitive* agents; **AgentSpeak(L)** (A. S. Rao, "AgentSpeak(L): BDI agents speak out in a logical computable language", MAAMAW 1996) is a logic-based programming language inspired by it:

- **beliefs** represent the knowledge of the agent;
- **goals** are beliefs, or conjunctions and disjunctions of beliefs, which must be *achieved* or *tested* in the current state;
- **plans** contain the *procedural* knowledge of the agent, characterised by a trigger, a context, a body, a maintenance condition, a set of "success actions" and a set of "failure actions";
- **intentions** are partially instantiated plans.

Its syntax (slide 35) and semantics (slide 36) are explained in Section 8.

JadeScript (slides 37–39). The agent-oriented programming language for JADE agents (Bergenti, Caire, Monica, Poggi, 2020). A Ping–Pong example: `Ping` sends a `request` for the action `Reply`; `Pong` answers with an `inform` that the proposition `Received` holds.

```
ontology PingPong
```

```
    action Reply
    proposition Received

agent Pong
    uses ontology PingPong
    on create do
        log "Agent 'Pong' created."
        activate WaitAndReply

cyclic behavior WaitAndReply for agent Pong
    on message request Reply do
        sender = sender of message
        log "received 'Reply' request from agent '" + name of sender + "'"
        inform Received to sender

agent Ping
    uses ontology PingPong
    on create do
        log "Agent 'Ping' created."
        activate WaitFromPong
        activate SendRequest

one shot behavior SendRequest for agent Ping
    on execute do
        log "sending 'Reply' request..."
        send message request Reply to "Pong"@
```

Notice the agent concepts as first-class language keywords: *agent*, *ontology* (the shared vocabulary), *behavior* (cyclic or one-shot, as in JADE, Section 10), *message* with a performative (`request`, `inform`).

6.3.4 4. MAS infrastructures and toolkits

[Slides · EMAS keynote: 40–46]

Slide 41 lists the papers of the *Agent Toolkits* special track at EUMAS 2024 (VEsNA, belief revision in JavaScript, protocol-based MAS, Yggdrasil for hypermedia MAS, SARL, Jadescript, JaKta, Jason+BSPL, JaCaMo, Jadex V, learning and explaining in BDI agents): the field is alive. Two infrastructures are highlighted:

- **JaCaMo** (2007–2020+): a platform combining **Ja**son (agents), **Ca**rtAgO (artifacts: the environment) and **Mo**ise (organisations).
- **JADE** (book *Developing multi-agent systems with JADE*, 2007) — "Remember this framework!" (slide 45). Section 10.

6.3.5 5. Agent-based modelling and simulation

[Slides · EMAS keynote: 47–48]

Pointer to a summer-school tutorial on running agent-based models (e.g. NetLogo) on high-performance computing facilities. See Section 11.

6.3.6 6. Integration of symbolic and sub-symbolic approaches

[Slides · EMAS keynote: 49–56]

The keynote repeats the System 1 / System 2 argument of Section 1.6 (Kahneman 2011; Booch et al., *Thinking Fast and Slow in AI*, 2021) and shows that the integration of symbolic and sub-symbolic ("neuro-symbolic") approaches is a large research area (Google Scholar searches, slides 54–55). It is not new: slide 56 shows J. Shavlik, *Combining Symbolic and Neural Learning* (Machine Learning, 1994), already reviewing hybrids of the two approaches thirty years ago. For agents, the typical combination is a BDI agent (symbolic reasoning, explainable plans) using learned components for perception, language understanding or plan selection.

6.4 Forth into the SteaMas future: designing the MAS

[Slides · EMAS keynote: 57–61]

Back to Dr. Joyce's city (Section 6.1). The keynote gives "5 minutes" of brainstorming to each requirement; in class the discussion was continued on the whiteboard (Whiteboards 4–6).

Requirement	**What we need (slides 58–60)**
#1 citizens and tourists respect civil/ethical **norms**	**norms** expressed in some formalism; **regulators** that verify that the norms are respected; a way to **communicate** the norms to regulators and mainly to "end users" (natural language, pictures); a way to **observe** what people do ("instrumentation" of the system to monitor it).
#2 **no private traffic**; free, effective public transport	collect **feedback** from people offline (natural language understanding and generation come into play); Google Maps integration; **economics**: even if free, transport has a cost, so agents should behave like in a **market** (rational) and negotiate how to best spend their allocated budget; **prediction** and prioritisation; cope with **emotions** and preferences (in the "not rational" sense); **transparency** (how we predict traffic, what we can expect).
#3 a free personal **drone** for everybody	as for traffic: emotions, geolocalisation, etc.

Table 15: Brainstorming on the three SteaMas requirements.

Slide 61: *Can we "sell" the agent technology to Dr. Joyce?* — "Yes we can!" Every requirement maps onto agent concepts: norms and regulators (normative MAS), negotiation (Contract Net, markets), emotions (strong agency), monitoring (runtime verification).

Whiteboard 4 — Methodology questions written during the SteaMas discussion

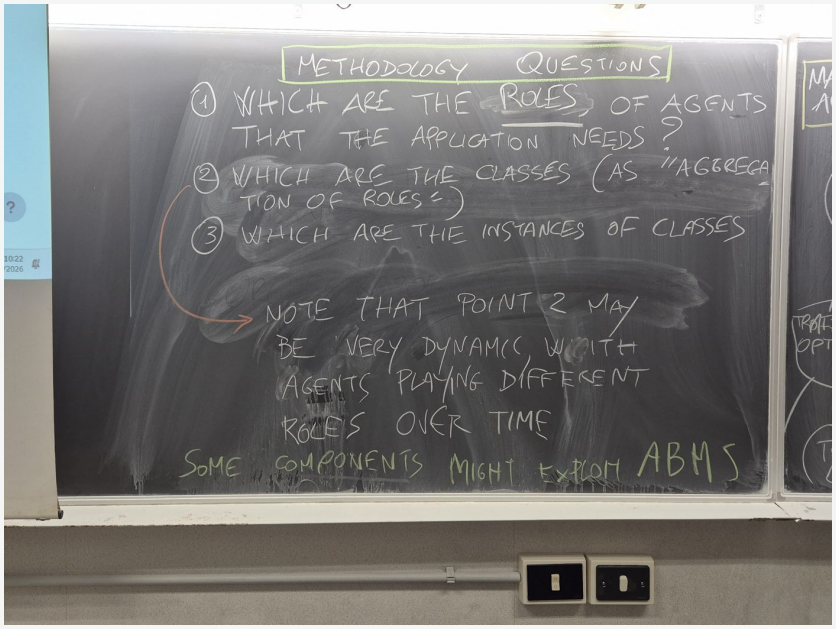
① Which are the **ROLES** of agents that the application needs?

② Which are the **CLASSES** (as "aggregation of roles")?

③ Which are the **INSTANCES** of classes?

note that point 2 may be **very dynamic**, with agents playing different roles over time

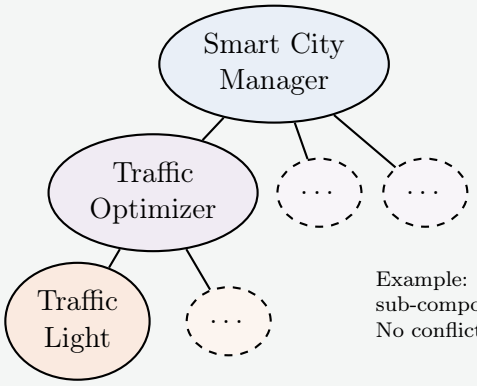
Some components might exploit ABMS.



Original photo of the whiteboard

Reading. Designing a MAS starts from *roles* (what has to be done), not from agents. A *class* of agents aggregates several roles (one agent type can play many roles); *instances* are the concrete running agents. In SteaMas: roles such as *traffic monitor*, *route planner*, *norm regulator*, *drone pilot*; a class *TrafficLightAgent* may aggregate *traffic monitor* and *local controller*; there will be one instance per traffic light. The mapping from roles to classes may change at run time (an agent can take a new role, like a manager/contractor in the Contract Net). Some parts (e.g. predicting crowds) can be done with agent-based simulation (Section 11).

Whiteboard 5 — MAS architecture and role level written during the SteaMas discussion

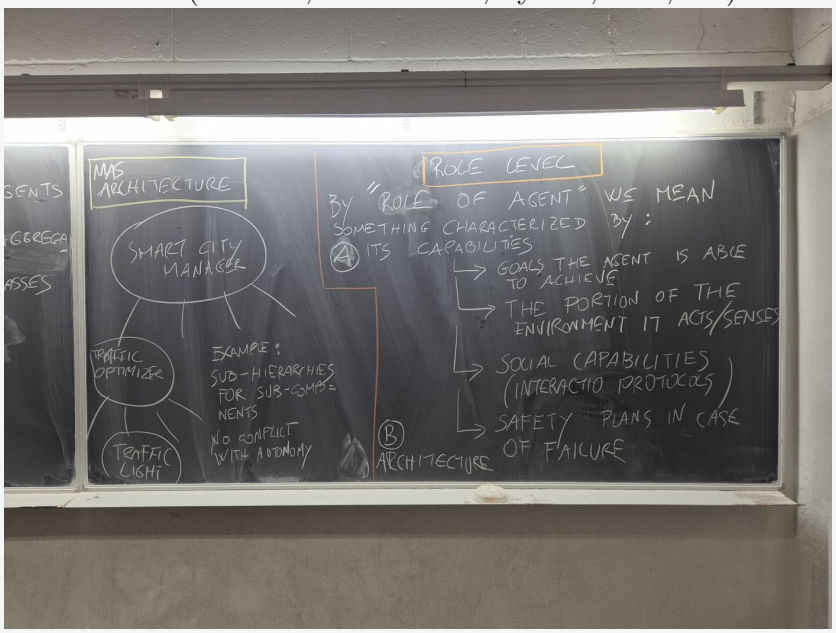


Example: sub-hierarchies for sub-components.
No conflict with autonomy.

Role level. By "role of an agent" we mean something characterised by:

(A) its **capabilities**:
- the **goals** the agent is able to achieve;
- the **portion of the environment** it acts on / senses;
- its **social capabilities** (interaction protocols);
- **safety plans** in case of failure;

(B) its **architecture** (reactive, deliberative, hybrid, BDI, ...).

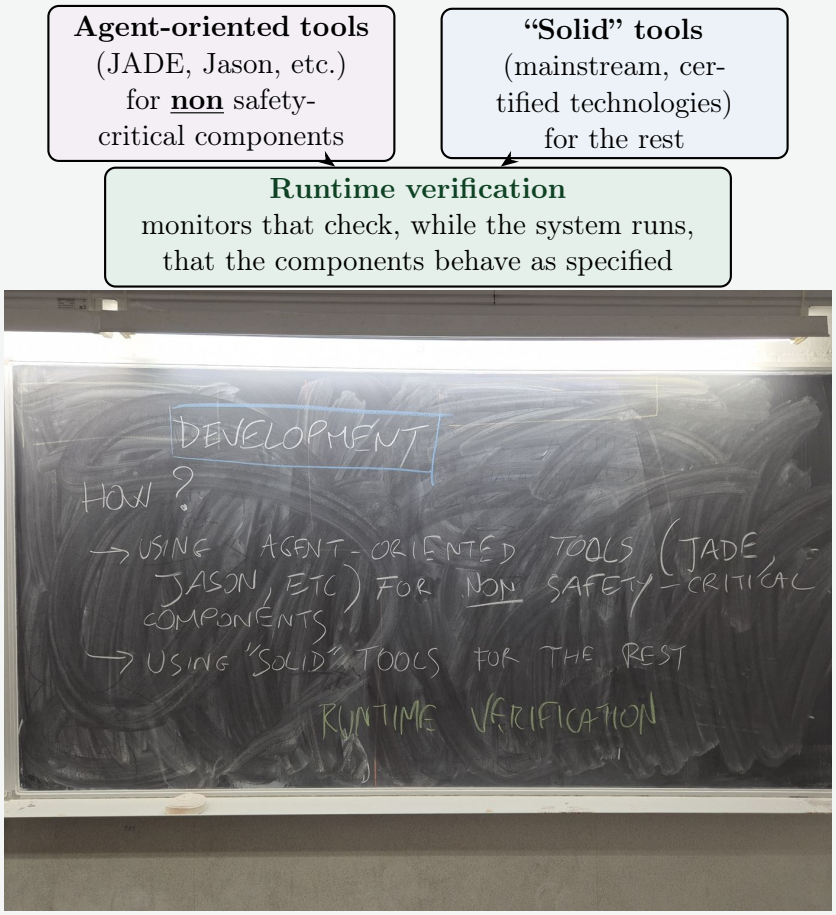


Original photo of the whiteboard

Reading. The MAS can be organised as a *hierarchy*: the Smart City Manager supervises sub-managers (the Traffic Optimizer, and others, e.g. for drones or norms), which in turn coordinate their sub-components (traffic lights). This does *not* conflict with autonomy: each agent still decides how to achieve its goals; the hierarchy only says who coordinates whom. A role is described by *what* it can do (goals, sphere of influence on the environment as in Figure 12, protocols such as FIPA Request, fallback plans when something fails) and by *how* it is built inside

(its architecture, Section 5).

Whiteboard 6 — Development: how? written during the SteaMas discussion



Original photo of the whiteboard

Reading. Agent platforms are research tools (Section 4.3): use them where a failure is not dangerous (e.g. the tourist recommendation agents), and use solid, industrial technologies for the safety-critical parts (e.g. the drones' flight control, the traffic lights). Either way, add *runtime verification* (Section 6.6) to observe the running system and detect violations of the expected behaviour, written in green on the board as the take-home message.

6.5 Conclusions: where are all the agents?

[Slides · EMAS keynote: 62–70]

In 2007 James Hendler, editor of *IEEE Intelligent Systems*, asked in a letter: **"Where are all the intelligent agents?"** In the 1990s many believed that agents would soon be everywhere; ten years later he saw few of them. The answer came in the same magazine from Peter McBurney and Michael Luck: **"The agents are all busy doing stuff!"**: agent technology was in use, but under other names, inside products and

companies, and not labelled "agent".

A good example (slides 65–66) is IBM's **autonomic computing** (*An architectural blueprint for autonomic computing*, 2005): self-managing IT systems in which an *autonomic manager* controls a managed resource through sensors and effectors with the **MAPE-K** loop. It is an agent architecture in all but name.

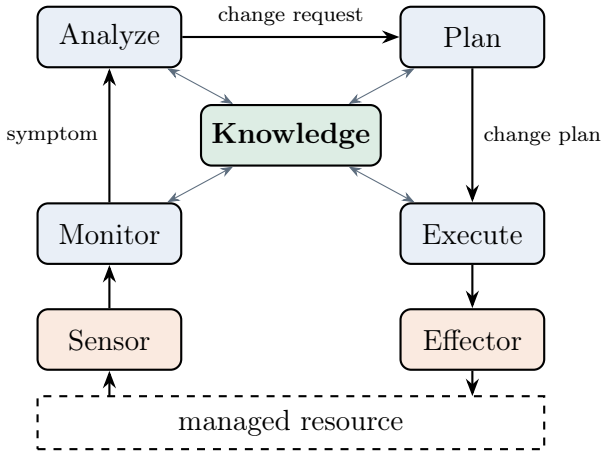


Figure 22: The MAPE-K loop of an autonomic manager (slide 65): monitor, analyse, plan, execute over shared knowledge. Compare with the perceive–decide–act loop of an agent.

Key result — Take-home message (slides 67–69)

Agents *are* all around: sometimes presented with different names, often deployed using mainstream technologies and languages. It's ok: recall the Gaia methodology, which leaves the implementation stage free. If you take agents and MAS as a **"mindset"**, you will find them everywhere, and you will understand the huge potential for their use (recent surveys: agents in healthcare, deep reinforcement learning for multi-agent systems, the agent-based Internet of Things).

6.6 Engineering safe MAS: formal and runtime verification

[Slides · EMAS keynote: 71–95]

The second part of the keynote (with Davide Ancona) addresses the seventh research direction: how can we trust autonomous systems?

6.6.1 Safe and reliable autonomous systems

[Slides · EMAS keynote: 72–79]

The paper *Towards a Framework for Certification of Reliable Autonomous Systems* (Fisher, Mascardi, Rozier, Schlingloff, Winikoff, Yorke-Smith, JAAMAS 2021) describes the progressive transfer of responsibility from humans to machines with four

pictures. In each, three capabilities appear: *adopting ethical principles / meta-reasoning*, *following the rules*, *controlling all the system's operations*.

Fig.	Situation	controls operations	follows rules	ethics
1	human user has full control of the tool (hammer, printing press, bicycle)	human	human	human
2	human (partially) delegates the *control of the operations* (a steam engine, a car, a tractor with automation)	software	human	human
3	human (partially) delegates the *respect of the rules* (a plane, a modern car, a robot with software)	software	software	human
4	human *replaced* by an autonomous software system	software	software	software

Table 16: Levels of delegation to software (slides 73–76). The higher the level, the more the software must be *certified* as reliable.

The same idea in the automotive world is the SAE scale of driving automation (slides 77–79), already seen in Section 1.4: from level 0 (the human drives) to level 5 (the system drives everywhere).

6.6.2 Verification techniques

[Slides · EMAS keynote: 80]

	Formal		**Semi-formal**
	properties & system model formally specified	properties formally specified; the system is real	properties informally specified; the system is real
Non-exhaustive, dynamic		runtime verification	runtime verification, software testing
Non-exhaustive, static	static analysis	static analysis	static analysis
Exhaustive, static	theorem proving, model checking	model checking	

Figure 23: Formal and semi-formal verification techniques (slide 80, redrawn as a table). Exhaustive techniques consider *all* possible behaviours of a *model*; runtime verification and testing observe *some* executions of the *real* system.

- **Theorem proving**: prove mathematically that the model satisfies the property

(Part II gives the flavour).

- **Model checking**: explore all states of a finite model automatically.
- **Static analysis**: inspect the code without running it (types, data flow).
- **Software testing**: run the system on chosen inputs and compare with the expected outputs.
- **Runtime verification**: watch the real system while it runs and check its execution against a formal specification.

6.6.3 Runtime verification

[Slides · EMAS keynote: 81–86]

Introductory reading (slides 82–83): Leucker and Schallhart, *A Brief Account of Runtime Verification* (2009); Bartocci, Falcone, Francalanza, Reger, *Introduction to Runtime Verification* (2018); Falcone et al., *A taxonomy for classifying runtime verification tools* (2021).

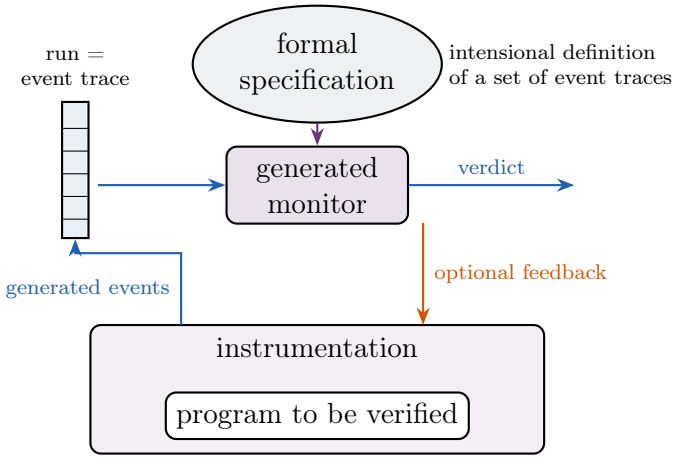


Figure 24: The runtime verification process (slide 84). The program is instrumented to emit events; the sequence of events of a run is the *trace*; a monitor, generated automatically from a formal specification, reads the trace and emits a verdict (and possibly feedback to the program, e.g. to recover from an error).

Worked example — a monitor for the FIPA Request protocol

[Not in the slides: added material]

Property: "every `request` is answered by `agree` or `refuse`; after an `agree`, exactly one of `failure`, `inform-done`, `inform-result` follows". Traces observed between a customer and a pizzeria agent:

Trace	Verdict
request, agree, inform-done	true (correct and complete)
request, refuse	true
request, agree	inconclusive so far (waiting for the outcome)
request, inform-done, agree	false: the outcome came before the agreement

The monitor does not need to know how the pizzeria agent is programmed (JADE, Jason, Python): it only observes messages. This is the "system agnosticism" of the next box.

Key result — RV bridges the gap between formal verification and testing (slide 85)

- The notion of **event trace** abstracts over system executions.
- RV can be adopted **online** (while the system runs) and **offline** (on recorded logs).
- **System agnosticism** better supports reuse and interoperability.
- RV offers more opportunities for **error recovery**, **self-adaptation**, and issues that go beyond software reliability.
- Often some information is available **only at runtime**: then RV is an alternative to formal verification.
- The behaviour of an application may depend heavily on its **environment**: RV adds to formal correctness proofs.
- Where **security** matters, it is useful to monitor properties even when they have been statically proved or tested.

RML (slide 86). The **Runtime Monitoring Language** (Ancona, Franceschini, Ferrando, Mascardi, *Science of Computer Programming*, 2021) is a domain-specific language, developed at the University of Genoa, for writing specifications of event traces from which monitors are generated; it is system-agnostic and has been used to monitor agent interaction protocols.

6.6.4 Conclusions

[Slides · EMAS keynote: 87–95]

Key result — Big concepts (slides 88–94)

- The adoption of AI (and agents and MAS *are* AI systems) raises **ethical concerns** that must be coped with: see the EU AI Act, the US Congress "AI Insight" forum, safety in Industry 4.0, "safety first".
- A reliable approach for engineering applications that involve agents in **safety-critical / ethically critical** domains is needed: formal methods in general, and runtime verification in particular, may help address some of their ethical threats, paving the way to a more **explainable, trustworthy, socially responsible** AI.
- Engineering safe agents and MAS is only one of the many goals of AOSE, which faces the theoretical and technological challenges of today's complex, distributed, heterogeneous, autonomous systems.

Cheat sheet — Engineering MAS

- **AOSE** = models and techniques to develop complex software as MAS. Why: autonomy fits decentralised systems owned by different stakeholders; flexible interaction fits dynamic, unpredictable scenarios; agency unifies AI results.
- Research directions: (1) architectures (BDI, subsumption, hybrid: TouringMachines, InteRRaP; FIPA platform), (2) methodologies and notations (Gaia, Prometheus, AUML), (3) AOP languages (logic-based, AgentSpeak(L), JadeScript), (4) infrastructures (JADE, JaCaMo, . . .), (5) ABMS, (6) symbolic + sub-symbolic integration, (7) safe autonomous systems.
- **Gaia**: analysis (roles, interactions) → architectural design (organisational structure) → detailed design (agent and services models); implementation left free.
- **FIPA Request protocol** (remember it!): request → refuse *or* agree → failure / inform-done / inform-result.
- SteaMas whiteboards: methodology questions (roles → classes → instances); hierarchical MAS architecture; a role = capabilities (goals, environment portion, interaction protocols, safety plans) + architecture; development: agent tools for non-safety-critical parts, solid tools for the rest, **runtime verification**.
- Verification: theorem proving and model checking (exhaustive, formal), static analysis, software testing, **runtime verification** (monitor checks the event trace against a formal specification; online or offline; RML).

Part IV

Prolog

7 Prolog

[Slides · Prolog I: 1–37; Prolog II: 1–9; Prolog IV: 1–15; Prolog V: 1–24; Prolog III: 1–9]

Prolog (PROgramming in LOGic, Colmerauer and Roussel, 1972; Section 1.3) is the language in which symbolic AI is most directly *programmed*. A Prolog program is a set of logical statements (facts and rules, i.e. Horn clauses of first-order logic, Section 3); running a program means asking a question (a *query*) and letting Prolog *prove* it. The professor's slides are based on material by Peter Schachte (University of Melbourne).

The five decks are presented in **lecture order**: I (basics), II (lists), IV (nondeterminism, arithmetic, all-solutions), V (meta-programming and the database), III (logical foundations: Horn clauses and SLD resolution). Part III comes last, as in class: it explains *why* everything seen before works.

Why this matters in practice — How to try the examples

[Not in the slides: added material]

The slides show sessions of tuProlog and SICStus Prolog, which answer `Yes`/`No`. Today the most common system is **SWI-Prolog** (free, `https://www.swi-prolog.org`): write the program in a file `royals.pl`, start `swipl`, load it with `?- [royals].` (or `consult(royals)`) and type queries after `?-`. SWI-Prolog answers `true`/`false` instead of `Yes`/`No`, names fresh variables `_123` or `_A` instead of `_G168`, and asks for more answers when you press `;` (or space). In these notes answers are written in SWI-Prolog style, with the slides' output where it differs.

7.1 Prolog basics

[Slides · Prolog I: 1–37]

7.1.1 Predicates and facts

[Slides · Prolog I: 4–5]

In most programming languages the basic abstraction is the *function* (or method): a mapping from inputs to outputs. In Prolog the basic abstraction is the **predicate** (or **relation**): it indicates *whether or not some entities are related* in a particular way. In logic programming we may *test* whether entities are related, or *find* entities that are related to other entities.

Worked example — the royal family program (slide 5)

```
1 parent(elizabeth, charles).
2 parent(diana, william).
3 parent(diana, harry).
4 parent(philip, charles).
5 parent(charles, william).
6 parent(charles, harry).
```

- `parent` is a **predicate** (of arity 2, written `parent/2`);
- each line is a separate **clause**, terminated by a full stop;
- each clause is a separate *true statement*, understandable on its own: "Elizabeth is a parent of Charles";
- names of individuals (`elizabeth`) are **atoms**: constants, starting with a lower-case letter.

A Prolog program may consist of many files, each defining many predicates, each defined by many clauses.

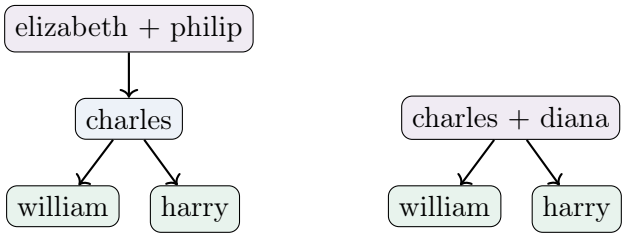


Figure 25: The family encoded by the `parent/2` facts (arrows go from parent to child).

7.1.2 Making queries

[Slides · Prolog I: 6–7]

Prolog systems provide an interactive environment to load and test code. The prompt is `?-`. A source file (extension `.pl`) is loaded by writing its name in single quotes inside square brackets, followed by a full stop: `?- ['royals.pl'].` Queries are typed at the prompt and Prolog answers them:

```
?- parent(diana, harry).
true.
?- parent(elizabeth, diana).
false.
```

(The slides' tuProlog prints `Yes` and `No`.)

Watch out — "false" means "not provable", not "false in reality"

`parent(elizabeth, diana)` fails because it cannot be derived from the program. Prolog works under the **closed-world assumption**: what is not known to be true is treated as false. Ask `?- parent(elizabeth, anne).` and you also get `false`, although Princess Anne is Elizabeth's daughter: she is simply not in the program.

7.1.3 Variables and multiple directions

[Slides · Prolog I: 8–11]

```
?- parent(elizabeth, X).
X = charles.
```

Read the query as "find `X` such that Elizabeth is a parent of `X`".

Definition — Prolog variables (slide 8)

Prolog variables begin with a **capital letter or an underscore**, followed by letters, digits and underscores (`X`, `Kid`, `_tmp`). They are very different from variables in other programming languages: like in Haskell, **you cannot change the value**

of a variable once it is set. A variable stands for an *unknown* object, as in algebra.

Some queries have more than one answer. Prolog prints one answer and waits: type `;` to ask for the next one, Return to stop.

```
?- parent(diana, X).
X = william ;
X = harry.
```

(The slides show `X = harry ;` followed by `No`: tuProlog only discovers that there are no more answers after you ask; SWI-Prolog often knows it already and ends with a full stop.)

Key result — Multiple directions (slides 10–11)

The same predicate can be queried "the other way around":

```
?- parent(X, william).
X = diana ;
X = charles.
```

"Find `X` such that `X` is a parent of William." This works because the program is **declarative**: it specifies information without specifying how it must be used. For each argument we may pass a value or a variable; values *constrain* the solutions, variables do not. `?- parent(X, Y).` imposes no constraint and enumerates all six pairs. Well-written logic programs exhibit this flexibility, although it is not always practical (see `rev/2` in Section 7.4).

Intuition — A predicate is a table, a query is a database question

[Not in the slides: added material]

`parent/2` is like a two-column database table. `parent(diana, X)` is "`SELECT child WHERE parent = 'diana'`"; `parent(X, william)` is "`SELECT parent WHERE child = 'william'`". In Java you would need two methods (`getChildren`, `getParents`); in Prolog one relation answers both questions.

7.1.4 Compound queries and constraints

[Slides · Prolog I: 12–18]

A query can invoke several predicates separated by commas ("and"). If a variable appears more than once, all occurrences must have the *same value*, as in algebra.

```
?- parent(elizabeth, X), parent(X, Y).
X = charles, Y = william ;
```

```
X = charles, Y = harry.
```

Worked example — how Prolog solves a compound query (backtracking)

[Not in the slides: added material]

1. First goal `parent(elizabeth, X)`: scan the facts top-down; the first match is `parent(elizabeth, charles)`, so `X = charles`. Remember this choice point.
2. Second goal, now `parent(charles, Y)`: first match `parent(charles, william)`, `Y = william`. Print the answer.
3. You type `;`: Prolog **backtracks** to the most recent choice point, the second goal, and continues scanning: `parent(charles, harry)`, `Y = harry`. Print.
4. `;` again: no more `parent(charles, _)` facts; back to the first goal: no other `parent(elizabeth, _)` fact. No more answers.

Let us add gender information (slide 13):

```
female(elizabeth).   female(diana).
male(philip).        male(charles).      male(william).     male(harry).
```

Predicates can take any number of arguments; the number is the **arity**. A predicate with one argument defines a *property* that certain entities may have.

Worked example — Motherhood (slides 14–16)

Give a Prolog query that determines the mother of William.

```
?- parent(X, william), female(X).
X = diana.
?- female(X), parent(X, william).     % equally good
X = diana.
```

In the first version Prolog tries `X = diana` (fact 2), which is female: answer. (On `;` it would try `X = charles`, who is not female: no more answers.) In the second, it tries `X = elizabeth` first (not a parent of William), then `X = diana`: answer.

Key result — Constraints (slides 17–18)

Think of each part of a compound query as a **constraint** on the solution: Prolog only shows solutions that satisfy all of them. Some constraints have no solutions; in general constraints can be placed in any order, but some orders are more efficient than others, and some orders may not work as expected (more later).

```
?- parent(X,Y), parent(Y,Z), male(Z), female(X).
X = elizabeth, Y = charles, Z = william ;
```

```
X = elizabeth, Y = charles, Z = harry ;
false.
?- parent(X,Y), parent(Y,X).
false.
?- male(X), female(X).
false.
```

7.1.5 Rules: clause bodies

[Slides · Prolog I: 19–21]

Definition — Clause, head, body, fact, rule (slide 19)

A clause may have two parts, a **head** and a **body**, separated by `:-` (read "**if**"). The head looks like a predicate with arguments (some may be variables), the body looks like a query. Meaning: *the head is true if the body is true.* If the body is empty, `:-` is omitted: a clause with no body is a **unit clause** or **fact**. A clause with a body is a **rule**.

```
1 grandparent(G, C) :-
2     parent(G, P),
3     parent(P, C).
```

Read: "`G` is a grandparent of `C` *if* `G` is a parent of `P` *and* `P` is a parent of `C`". Variables in a rule are universally quantified: in FOL, $\forall G \forall C \forall P\,(parent(G,P) \land parent(P,C) \rightarrow grandparent(G,C))$.

```
?- grandparent(elizabeth, X).
X = william ;
X = harry.
?- grandparent(X, william).
X = elizabeth ;
X = philip ;
false.
```

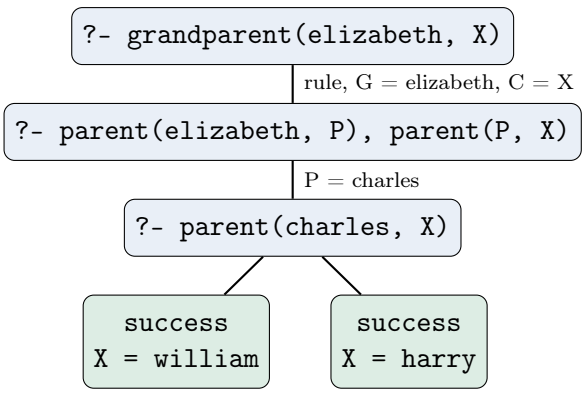


Figure 26: The search tree of `?- grandparent(elizabeth, X).`: each node is the list of goals still to prove; each edge is the choice of a clause. Answers are the leaves where nothing is left to prove. Section 7.5 calls this an *SLD tree*.

Exercise P1.1 — Family relationships (slide 21)

1. Define a predicate `father(Dad, Kid)` that holds when `Dad` is the father of `Kid`.
2. Define a predicate `greatgrandparent(Adult, Child)` that holds when `Adult` is a great-grandparent of `Child`.

Solution

```
% 1. Dad is the father of Kid if Dad is a parent of Kid and Dad is male
father(Dad, Kid) :-
    parent(Dad, Kid),
    male(Dad).

% 2. Adult is a great-grandparent of Child if Adult is a parent
%    of some X and X is a grandparent of Child
greatgrandparent(Adult, Child) :-
    parent(Adult, X),
    grandparent(X, Child).
```

`?- father(X, harry).` gives `X = charles` (the parents of Harry are Diana and Charles; only Charles is male). With the six facts of the program, `greatgrandparent/2` has *no* solutions (the family tree has only three generations): `?- greatgrandparent(A, C).` answers `false`. Adding, e.g., `parent(william, george).` makes `?- greatgrandparent(A, george).` answer `A = elizabeth ; A = philip`. **Lesson**: reuse existing predicates; a great-grandparent is a parent of a grandparent.

7.1.6 Recursive predicates

[Slides · Prolog I: 22–23]

```
ancestor(A, A).
ancestor(A, D) :-
    parent(A, C),
    ancestor(C, D).
```

The first clause says that everyone is their own ancestor (not the usual definition, but common in computer science); the second that the parents of your ancestors are your ancestors. The same code finds both ancestors and descendants:

```
?- ancestor(A, harry).
A = harry ; A = elizabeth ; A = diana ; A = philip ; A = charles ; false.
```

```
?- ancestor(elizabeth, D).
D = elizabeth ; D = charles ; D = william ; D = harry ; false.
```

Worked example — why the answers of `ancestor(A, harry)` come in that order

[Not in the slides: added material]

1. Clause 1 unifies `ancestor(A, A)` with `ancestor(A, harry)`: `A = harry`.
2. Clause 2: prove `parent(A, C), ancestor(C, harry)`, trying the parent facts in order:
   - `parent(elizabeth, charles)`: need `ancestor(charles, harry)`; clause 1 fails (`charles` ≠ `harry`); clause 2: `parent(charles, william)` then `ancestor(william, harry)`: fails (William has no children); `parent(charles, harry)` then `ancestor(harry, harry)`: clause 1 succeeds. Answer `A = elizabeth`.
   - `parent(diana, william)`: `ancestor(william, harry)` fails.
   - `parent(diana, harry)`: `ancestor(harry, harry)` succeeds: `A = diana`.
   - `parent(philip, charles)`: as for Elizabeth: `A = philip`.
   - `parent(charles, william)`: fails; `parent(charles, harry)`: `A = charles`.

The order of answers follows the order of the facts, exactly as in a depth-first search.

Watch out — Put the base case first and recurse on something smaller

If the body were written `ancestor(C, D), parent(A, C)` (recursive call first), the query `?- ancestor(A, harry).` would find the first answers and then call `ancestor` with both arguments unknown, again and again, without ever consuming a `parent` fact: an infinite loop (*left recursion*). Same logic, different behaviour: the order of goals matters for termination (Section 7.5).

7.1.7 Disjunction

[Slides · Prolog I: 24]

Disjunction ("or") in Prolog uses the `;` operator; it is equivalent to writing two clauses:

```
person(X) :-
    ( good(X) ; bad(X) ).
```

≡

```
person(X) :- good(X).
person(X) :- bad(X).
```

7.1.8 Terms and variables

[Slides · Prolog I: 25–28]

How can a variable be a data structure? A Prolog variable is *just a term whose value*

you don't know yet. Don't *assign* to variables, *constrain* them. A variable can be bound to any term, even another variable, but cannot change value once bound. It is more like a variable in algebra than in a conventional programming language.

Definition — Compound term (slide 26)

The **compound term** is Prolog's only data-structuring abstraction. It has a **functor**, which is an atom, and one or more **arguments**, which can be any terms. Syntax: functor first, then the arguments in parentheses, separated by commas. The number of arguments is the **arity**. No need to declare it: just use it.

Compound terms look just like predicate invocations (Whiteboard 1: the same syntax builds terms and atoms), but they are *data*. Use them where you would use (a pointer to) a `struct` in C; the functor indicates the kind of term:

```
position(20, 10)     employee(1234, 'Jones', 'James', 100000)
```

No allocation, no assignment to members: just write it down. Terms may contain variables, whether or not their values are determined: `position(X, Y)`, `employee(Num, Surname, Firstname, Salary)`.

Kind of term	Examples	Rule
atom	`elizabeth`, `[]`, `'Jones'`, `+`	lower-case start, or quoted
number	`42`, `-3`, `3.14`	
variable	`X`, `Kid`, `_`, `_tmp`	upper-case or `_` start
compound	`position(20,10)`, `s(s(0))`, `[1,2]`	functor(args)

Table 17: The kinds of Prolog terms (added). Atoms and numbers are called *atomic*; a term with no variables is *ground*.

7.1.9 Unification

[Slides · Prolog I: 29–33]

In Prolog, `=` is used both for giving values to variables and for comparing terms:

```
?- 4 = 4.          true.
?- 4 = 5.          false.
?- X = 5.          X = 5.
?- 5 = V.          V = 5.
?- X = Y.          X = Y.
```

(For the last query the slides show `X = _G168, Y = _G168`: both variables are bound to the same fresh variable.) Variables appearing in different queries are unrelated.

Definition — Unification (slide 30)

`=` performs **unification**: it tries to make two terms the *same* by binding the variables appearing in them. It is a powerful sort of pattern matching, and Prolog also uses it for argument passing (calling `parent(diana, X)` unifies the call with the head of each clause).

```
?- foo(A,2,C,D) = foo(1,X,Y,X).
A = 1, X = 2, D = 2, C = Y.
?- bar(X, X) = bar(1, 2).
false.
```

Key result — How to unify two terms (the algorithm, added)

[Not in the slides: added material]

To unify s and t:

1. if one of them is an unbound variable, bind it to the other (if both are the same variable, do nothing);
2. if both are atomic (atoms or numbers), they unify iff they are identical;
3. if both are compound, they unify iff they have the **same functor and arity** and their arguments unify **pairwise**, left to right, carrying the bindings along;
4. otherwise they do not unify.

The result is the **most general unifier** (mgu): the bindings that make the terms equal while committing to as little as possible.

Worked example — `foo(A,2,C,D) = foo(1,X,Y,X)` step by step

Same functor `foo`, same arity 4: unify the arguments pairwise.

Pair	Left	Right	Result
1	`A`	`1`	bind `A = 1`
2	`2`	`X`	bind `X = 2`
3	`C`	`Y`	bind `C = Y` (two variables, now the same unknown)
4	`D`	`X`	`X` is already `2`: bind `D = 2`

For `bar(X, X) = bar(1, 2)`: pair 1 binds `X = 1`; pair 2 must unify `1` with `2`: different atomic terms, failure.

Exercise P1.2 — Unification (slides 31–33)

Find bindings that make these pairs of terms the same: (1) `f(X, 17) = f(Y, Y)`; (2) `f(42, U, a) = f(V, f(V,W), W)`. (`f` is used with arity 2 and 3: no problem, they are different functors.)

Solution

(1) Pair 1: `X = Y`. Pair 2: `17 = Y`, so `Y = 17`, hence `X = 17`. Answer: `X = 17, Y = 17`.
(2) Pair 1: `42 = V`, so `V = 42`. Pair 2: `U = f(V, W)` = `f(42, W)`. Pair 3: `a = W`, so `W = a`, and therefore `U = f(42, a)`. Answer: `V = 42, W = a, U = f(42, a)`.

Watch out — Unification is not assignment, and not arithmetic

`X = 1 + 2` binds `X` to the *term* `+(1,2)`, not to `3` (arithmetic needs `is`, Section 7.3). And `X = f(X)`: standard unification would need the *occurs check* (a variable cannot be bound to a term containing itself) and fail; SWI-Prolog skips the check for speed and builds a cyclic term. Avoid such goals.

7.1.10 Recursive structures: numbers as terms

[Slides · Prolog I: 34–37]

Any argument of a compound term can be any term, including a term with the same functor: this is how Prolog implements recursive structures (lists, trees, . . .). A good strategy to write predicates on recursive structures:

1. write a predicate that *recognises* the recursive data structure;
2. use it as a "skeleton" for other predicates, adding arguments and goals.

For example, represent a natural number as either `0` or `s(X)` ("one more than `X`", the *successor*), where `X` is a natural number: 3 is `s(s(s(0)))`. (This is *not* how Prolog really does maths.)

```
1 nat(0).                    % 0 is a natural number
2 nat(s(X)) :- nat(X).       % s(X) is a nat if X is
3
4 add(0, Y, Y).              % adding 0 to Y gives Y
5 add(s(X), Y, s(Z)) :-      % adding X+1 to Y gives Z+1
6     add(X, Y, Z).          %   where Z is X + Y
```

The structure of `add/3` follows that of `nat/1`. We refer to predicates as `name/arity` because Prolog allows different predicates with the same name and different arity.

Worked example — `add(s(s(0)), s(s(s(0))), X)`: 2 + 3

```
add(s(s(0)), s(s(s(0))), X)      ◁ clause 2: X = s(Z1)
add(s(0), s(s(s(0))), Z1)        ◁ clause 2: Z1 = s(Z2)
add(0, s(s(s(0))), Z2)           ◁ clause 1: Z2 = s(s(s(0)))
```

Back-substituting: `X = s(Z1) = s(s(Z2)) = s(s(s(s(0))))`, i.e. 5. Check: $2 + 3 = 5$.

Key result — Example queries (slide 36)

```
?- nat(s(s(0))).                          true.
?- nat(X).                                X = 0 ; X = s(0) ; X = s(s(0)) ; ...
?- add(s(s(0)), X, Y).                    Y = s(s(X)).
?- add(s(s(0)), X, s(s(s(s(0))))).        X = s(s(0)).        % 5 - 2
?- add(X, s(s(s(0))), s(s(s(s(0))))).     X = s(s(0)) ;  false.  % 5 - 3
?- add(X, Y, s(s(0))).                    X = 0, Y = s(s(0)) ;
                                          X = s(0), Y = s(0) ;
                                          X = s(s(0)), Y = 0 ; false.
```

The same definition of addition also computes **subtraction** and enumerates all ways of writing a number as a sum: multiple directions again. `nat(X)` generates the natural numbers forever.

Exercise P1.3 — Multiplication (slide 37)

Define `mult(X, Y, Z)`, true when `X` times `Y` equals `Z`, using `add/3`.

Solution

```
1 mult(0, _, 0).          % multiplying anything by 0 gives 0
2 mult(s(X), Y, Z) :-     % Z = (X+1)*Y if
3     mult(X, Y, W),      %   W = X*Y
4     add(W, Y, Z).       %   and Z = W + Y
```

This follows the skeleton of `nat/1` on the first argument: $0 \cdot Y = 0$ and $(X + 1) \cdot Y = X \cdot Y + Y$. Trace of $2 \cdot 3$: `mult(s(s(0)), s(s(s(0))), Z)` calls `mult(s(0), ..., W1)`, which calls `mult(0, ..., W0)`, giving `W0 = 0`; then `add(0, s(s(s(0))), W1)` gives $W_1 = 3$; then `add(s(s(s(0))), s(s(s(0))), Z)` gives $Z = 6 =$ `s(s(s(s(s(s(0))))))`.

7.2 Lists

[Slides · Prolog II: 1–9]

7.2.1 List syntax

[Slides · Prolog II: 5]

Lists are very widely used in Prolog, and have a special syntax:

- `[]` is the **empty list**;
- `[H|T]` is the list with **head** `H` (the first element) and **tail** `T` (the list of the remaining elements);

- `[1,2]` is a list of two elements; `[A,B,C|D]` is a list whose first three elements are `A`, `B`, `C` and whose remaining elements form the list `D`.

A recogniser, in the "skeleton" style of Section 7.1:

```
list([]).
list([_|Tail]) :- list(Tail).
```

Key result — Lists are ordinary terms

The bracket notation is special syntax, but lists are ordinary compound terms: the list functor is the dot `'.'` with arity 2 (in SWI-Prolog 7 and later it is called `'[|]'`). So

`[1,2,3] = [1|[2,3]] = [1|[2|[3|[]]]] = '.'(1, '.'(2, '.'(3, [])))`

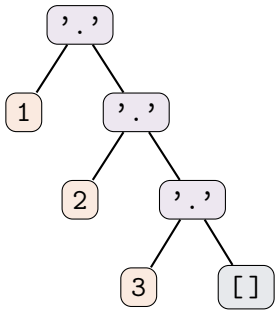


Figure 27: The list `[1,2,3]` as a tree: a chain of dot cells, each holding one element (left) and the rest of the list (right), ending with `[]`. Exactly a linked list.

Worked example — unifying lists

[Not in the slides: added material]

Query	Answer
`[H|T] = [a,b,c]`	`H = a, T = [b,c]`
`[H|T] = [a]`	`H = a, T = []`
`[H|T] = []`	`false` (the empty list has no head)
`[A,B|R] = [1,2,3,4]`	`A = 1, B = 2, R = [3,4]`
`[A,B] = [1,2,3]`	`false` (different lengths)

7.2.2 List membership

[Slides · Prolog II: 6–7]

`member/2` checks list membership. It is a built-in of SWI-Prolog, but could be defined as:

```
member(X, [X|_]).        % X is member of a list beginning with X
member(X, [_|L]) :-      % X is member of a list
```

```
3       member(X, L).          %   if it is member of its tail
```

Key result — Member in action (slide 7)

```
?- member(c, [a,b,c,d]).          true.
?- member(e, [a,b,c,d]).          false.
?- member(X, [a,b,c]).            X = a ; X = b ; X = c.
?- L = [_,_,_], member(a, L).     L = [a, _, _] ; L = [_, a, _] ; L = [_, _,
↪  a].
```

The same predicate *checks* membership, *enumerates* the elements, and even *builds* lists containing a given element.

Worked example — trace of `member(c, [a,b,c,d])`

[Not in the slides: added material]

Goal	Clause 1 `member(X,[X|_])`	Clause 2
`member(c, [a,b,c,d])`	`c` vs `a`: fail	recurse on `[b,c,d]`
`member(c, [b,c,d])`	`c` vs `b`: fail	recurse on `[c,d]`
`member(c, [c,d])`	`c` vs `c`: **success**	

For `member(e, [a,b,c,d])` the recursion reaches `member(e, [])`: neither clause unifies with an empty list, so the query fails.

7.2.3 List concatenation: append

[Slides · Prolog II: 8–9]

`append/3` concatenates two lists (`++` in Haskell). Again a built-in, but could be defined as:

```
1  append([], L, L).                % [] ++ L gives L
2  append([J|K], L, [J|KL]) :-      % to ++ a list with first element J and
   ↪  remainder K
3      append(K, L, KL).            %   to list L, determine K ++ L and then add
       ↪  J to front
```

Compare with Haskell: `append [] l = l` and `append (j:k) l = j : append k l`. The same, but Haskell can only append lists, while Prolog's `append` is more flexible:

Key result — Append in action (slide 9)

```
?- append([a,b,c], [d,e], L).       L = [a,b,c,d,e].
?- append(L, [d,e], [a,b,c,d,e]).   L = [a,b,c] ;  false.
?- append([a,b,c], L, [a,b,c,d,e]).L = [d,e].
?- append(K, L, [a,b]).             K = [], L = [a,b] ;
                                    K = [a], L = [b] ;
```

```
K = [a,b], L = [] ; false.
```

Concatenation, "what must I prepend?", "what remains?", and "all ways to split a list": four functions in one relation.

Worked example — `append([a,b], [c], L)` step by step

```
append([a,b], [c], L)        ◁ clause 2: J=a, K=[b], L=[a|KL1]
append([b], [c], KL1)        ◁ clause 2: KL1=[b|KL2]
append([], [c], KL2)         ◁ clause 1: KL2=[c]
```

So `L = [a|[b|[c]]] = [a,b,c]`. The work is proportional to the length of the *first* list.

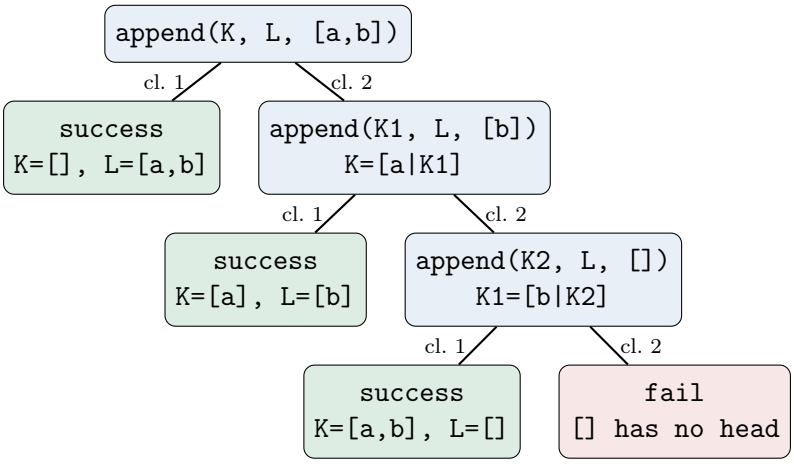


Figure 28: Search tree of `?- append(K, L, [a,b]).`: Prolog explores it depth-first, left to right; each green leaf is one answer, in the order shown in the slides; the red leaf is why the last `;` answers `false`.

Why this matters in practice

[Not in the slides: added material]

Lists are Prolog's workhorse: an agent's list of pending tasks, a path in a route planner, the moves of a game (the exam exercises often ask to move elements between lists with `select/3`, `append/3`, `member/2`). `append/3` used "backwards" is also a compact way to express search: "find a split of the list such that…".

7.3 Nondeterminism, arithmetic, all solutions

[Slides · Prolog IV: 1–15]

7.3.1 Arithmetic in Prolog: a short primer

[Not in the slides: added material]

The Prolog IV deck uses arithmetic without introducing it (that part was in a deck not shown in class). The essentials:

Operator	Example	Meaning
`X is Expr`	`X is 2 + 3` gives `X = 5`	**evaluate** the right side (must be fully known), unify the result with the left side
`=:=`, `=\=`	`1 + 2 =:= 3` true	arithmetic equal / not equal (both sides evaluated)
`<`, `>`, `=<`, `>=`	`X >= 0`	arithmetic comparison (both sides must be known numbers)
`=`	`X = 2 + 3` gives `X = 2+3`	**unification**: no evaluation, `X` becomes the term `+(2,3)`
`==`	`X == Y`	identical terms, without binding (Section 7.4)

Table 18: The four "equalities" of Prolog. Note `=<` (not `<=`).

Watch out — The classic `=` vs. `is` trap

- `X = 3 + 4, X == 7` fails: `X` is the term `3+4`, not the number 7.
- `7 is X + 4` with `X` unknown raises an error ("Arguments are not sufficiently instantiated"): `is` cannot solve equations, it only evaluates. Arithmetic predicates therefore work in *one direction only*, unlike `add/3` on `s(...)` numbers.
- `N is N + 1` always fails (for a bound `N`): variables cannot change value. Use a new variable: `N1 is N + 1`.

7.3.2 Managing nondeterminism

[Slides · Prolog IV: 4–7]

A common Prolog programming error is treating multiple clauses as if they formed an **if-then-else** construct. This is an *erroneous* definition of absolute value:

```
1 absolute_value(X, X) :- X >= 0.
2 absolute_value(X, Y) :- Y is -X.
```

```
?- absolute_value(-3, V).      V = 3.
?- absolute_value(42, V).      V = 42 ;
                               V = -42.        % wrong!
```

The problem: the second clause promises that $|n| = -n$ *for any* n, regardless of its sign. Each clause must be *true on its own*, because Prolog may use any of them on backtracking. The correct definition:

```prolog
absolute_value(X, X) :- X >= 0.
absolute_value(X, Y) :- X < 0, Y is -X.
```

Now `?- absolute_value(42, V).` gives only `V = 42` (the second clause fails at `42 < 0`).

Intuition — Read every clause as a separate true statement

In a procedural language, "else" means "in all other cases". Prolog has no implicit "else": clause 2 is a logical statement "for all `X`, the absolute value of `X` is `-X`", which is false for positive numbers. A good test: read each clause aloud alone. If it is not true, add the missing condition (the *guard*).

Exercise P4.1 — Maximum of two numbers (slides 6–7)

Define `maximum(X, Y, Z)` such that `Z` is the larger of `X` and `Y`.

Solution

Two clauses, each with its own guard, so each is true on its own: `maximum(X, Y, X) :- X >= Y.` and `maximum(X, Y, Y) :- X < Y.` `?- maximum(3, 7, Z).` gives `Z = 7`; `?- maximum(7, 3, Z).` gives `Z = 7` and nothing else on backtracking. The guards are mutually exclusive ($\geq$ vs. $<$), so exactly one clause applies for every pair of numbers.

7.3.3 Fibonacci numbers

[Slides · Prolog IV: 8]

Fibonacci numbers: $1, 1, 2, 3, 5, 8, 13, \ldots$ (each is the sum of the previous two).

```prolog
fib(0, 1).
fib(1, 1).
fib(N, F) :-
    N1 is N - 1,
    N2 is N - 2,
    fib(N1, F1),
    fib(N2, F2),
    F is F1 + F2.
```

`?- fib(4, F).` gives `F = 5`. The slide asks three questions:

1. **What if we change the order of goals? What if we do `F is F1 + F2` first?** Then `is` is called while `F1` and `F2` are still unknown: error "Arguments are not sufficiently instantiated". Arithmetic goals must come *after* the goals that compute their inputs. (Moving `N1 is N - 1` after `fib(N1, F1)` breaks it

in the same way, and worse: `fib(N1, F1)` with unknown `N1` would match the facts and then loop.)

2. **Each call makes two recursive calls. Are there better ways?** The number of calls $T(n)$ satisfies $T(n) = 1 + T(n-1) + T(n-2)$ with $T(0) = T(1) = 1$:

n	2	3	4	5	10	20	30
$fib(n)$	2	3	5	8	89	10 946	1 346 269
calls $T(n)$	3	5	9	15	177	21 891	2 692 537

Exponential: the same values are recomputed again and again. Better: carry the last two values as accumulators (linear time), or remember computed values with `asserta` (Section 7.4).

Watch out — Hidden bug: asking for more answers loops forever

[Not in the slides: added material]

After `F = 5`, type `;`. Prolog backtracks into `fib(1, F1)` and tries clause 3 *also for $N = 1$* (nothing forbids it): `N1 = 0`, `N2 = -1`, then `fib(-1, _)`, `fib(-2, _)`, ... forever (stack overflow). Same disease as the absolute value: clause 3 is only true for $N > 1$. Fix: `fib(N, F) :- N > 1, N1 is N - 1, ...`

7.3.4 Lists and arithmetic

[Slides · Prolog IV: 9–13]

Summing a list combines arithmetic with list processing:

```
sumlist([], 0).                 % sum([]) = 0
sumlist([X|Xs], Sum) :-         % sum([X|Xs]) = Sum if
    sumlist(Xs, Sum1),          %   sum(Xs) = Sum1 and
    Sum is Sum1 + X.            %   Sum = Sum1 + X
```

This can be done more efficiently with an **accumulator**:

```
sumlist(L, Sum) :-              % sumlist/2 defined in terms of sumlist/3
    sumlist(L, 0, Sum).         %   (third argument is the accumulator)
sumlist([], Sum, Sum).          % sum([]) + Sum = Sum
sumlist([X|Xs], Sum0, Sum) :-   % sum([X|Xs]) + Sum0 = Sum if
    Sum1 is Sum0 + X,           %   Sum1 = Sum0 + X, and
    sumlist(Xs, Sum1, Sum).     %   sum(Xs) + Sum1 = Sum
```

Worked example — the two versions on `[1,2,3]`

Naive	Accumulator
`sumlist([1,2,3], S)`	`sumlist([1,2,3], 0, S)`
`  sumlist([2,3], S1)`	`  sumlist([2,3], 1, S)`
`    sumlist([3], S2)`	`    sumlist([3], 3, S)`
`      sumlist([], S3): S3 = 0`	`      sumlist([], 6, S): S = 6`
`    S2 is 0 + 3 = 3`	
`  S1 is 3 + 2 = 5`	
`S is 5 + 1 = 6`	

The naive version must keep every call open waiting for the recursive result, then adds on the way back; the accumulator version adds on the way *down*, and the recursive call is the *last* goal (**tail recursion**): Prolog can reuse the same stack frame, like a loop. Both give `S = 6`.

List maximum (slide 10).

```
1 maxlist([X], X).
2 maxlist([X|Xs], Max) :-
3     maxlist(Xs, Max1),
4     maximum(Max1, X, Max).
```

No clause for `[]`: the base case is the singleton list, because `[]` has no maximum. Example: `maxlist([3,1,4], M)`: `maxlist([4], M2)` gives 4, then `maximum(4, 1, M1)` gives 4, then `maximum(4, 3, M)` gives `M = 4`.

List length (slide 11). `length/2` relates a list to its length:

```
?- length([a,b,c], 3).      true.
?- length([a,b,c], N).      N = 3.
?- length(L, 3).            L = [_, _, _].
?- length(L, N).            L = [], N = 0 ;  L = [_], N = 1 ;  L = [_, _], N
↪  = 2 ; ...
```

It is built in most Prolog systems; in SICStus it is in a library, loaded with `:- use_module(library(lists)).`

Key result — List handling with `append` and `length` (slides 12–13)

```
1 member(X, L) :-              % X is a member of L if L is of the form
2     append(_, [X|_], L).     %   any list ++ [X] ++ any list
3
4 element(N, L, E) :-          % E is the Nth element of L (counting from
↪  0)
5     append(F, [E|_], L),     %   if L = F ++ [E] ++ any list
```

```
6       length(F, N).           %   and the length of F is N
7
8   take(N, L, T) :-            % T is the first N elements of L
9       append(T, _, L),        %   if L = T ++ any list
10      length(T, N).           %   and the length of T is N
11
12  drop(N, L, T) :-            % T is what is left after removing the first
    ↪  N elements
13      append(F, T, L),        %   if L = F ++ T
14      length(F, N).           %   and the length of F is N
15
16  listof([], _).              % [] is a list of anything
17  listof([E|Es], E) :-        % [E|Es] is a list of Es if
18      listof(Es, E).          %   Es is
```

`?- element(1, [a,b,c], E).` gives `E = b`; `?- take(2, [a,b,c], T).` gives `T = [a,b]`; `?- drop(1, [a,b,c], T).` gives `T = [b,c]`; `?- listof([x,x,x], x).` is true.

Watch out — Comments of slide 13

The comments of `drop/3` on the slide are copied from `take/3` ("T is the first N elements of L", "if L = T ++ any list"): the correct reading is the one above, where `F` (not `T`) is the prefix of length `N`.

Intuition — Generate and test

`element`, `take` and `drop` all follow the pattern "`append` *generates* all the ways to split the list; `length` *tests* which split has the right size". Simple and declarative, though not the fastest (a direct recursion stops earlier).

7.3.5 All-solutions predicates

[Slides · Prolog IV: 14–15]

Normally Prolog gives one answer at a time. Built-in predicates collect them all:

Definition — findall, setof, bagof (slide 15)

- `findall(T, G, L)`: `L` is the list of all terms `T` that satisfy goal `G`, in the order solutions are found; variables in `G` are left uninstantiated; deterministic (if there are no solutions, `L = []`).
- `setof(T, G, L)`: `L` is a **non-empty** list of all terms `T` that satisfy `G`, **sorted with duplicates removed**; variables appearing only in `G` can be instantiated; can be nondeterministic (one answer for each different instantiation of those variables).
- `bagof(T, G, L)`: like `setof/3` but without the sorting.

The template term `T` is left uninstantiated by all of these predicates.

Worked example — all solutions on the royal family

[Not in the slides: added material]

```
?- findall(C, parent(charles, C), L).     L = [william, harry].
?- findall(C, parent(P, C), L).           L = [charles, william, harry, charles,
↪  william, harry].
?- findall(C, parent(harry, C), L).       L = [].
?- bagof(C, parent(harry, C), L).         false.
?- bagof(C, parent(P, C), L).             P = charles,   L = [william, harry] ;
                                          P = diana,     L = [william, harry] ;
                                          P = elizabeth, L = [charles] ;
                                          P = philip,    L = [charles].
?- setof(C, P^parent(P, C), L).           L = [charles, harry, william].
```

`findall` ignores `P`; `bagof` groups the answers by each value of the free variable `P`; `P^` ("there exists a `P`") tells `setof`/`bagof` not to group by `P`. `setof` sorts and removes duplicates.

Why this matters in practice

[Not in the slides: added material]

All-solutions predicates turn a *search* into a *data structure*: "the list of all free rooms", "all plans applicable now". Agent interpreters do exactly this when they compute the set of relevant plans for an event (Section 8).

7.4 Terms, comparison, program database

[Slides · Prolog V: 1–24]

This deck covers the "non-logical" side of Prolog: predicates that inspect the *structure* of terms, compare them, test whether variables are bound, and read or modify the *program itself* while it runs.

7.4.1 Handling terms in general: univ, functor, arg

[Slides · Prolog V: 3–9]

Sometimes we want predicates that work on *any* kind of term, without knowing its functor in advance.

Definition — Univ `=..` (slide 3)

`Term =.. List` (pronounced "univ") turns a term into a list: the first element is the term's functor, the following elements are its arguments, in order. It also works in the other direction.

```
?- p(1, q(3)) =.. X.      X = [p, 1, q(3)].
?- T =.. [p, 1, q(3)].    T = p(1, q(3)).
?- 1 =.. X.               X = [1].
```

```
?- [1,2,3] =.. X.          X = ['[|]', 1, [2,3]].      % slides: ['.', 1,
↪  [2,3]]
```

Since univ builds a list that is often not needed, it is usually better to use `functor/3` and `arg/3`.

Definition — functor/3 (slides 4–5)

`functor(Term, Name, Arity)` holds when the functor of `Term` is `Name` and its arity is `Arity`. For atomic terms, the whole term is the name and the arity is 0. Either `Term`, or both `Name` and `Arity`, must be bound, otherwise `functor` throws an exception.

```
?- functor(p(1, q(3)), F, A).   F = p, A = 2.
?- functor(T, p, 2).            T = p(_, _).       % builds a "blank"
↪  term
?- functor(1, F, A).            F = 1, A = 0.
?- functor(X, 42, 0).           X = 42.
```

Definition — arg/3 (slides 6–7)

`arg(N, Term, Arg)` holds if `Arg` is the `N`-th argument of `Term` (counting from 1). `Term` must be bound to a compound term. In most systems `N` must be bound; SWI-Prolog backtracks over all arguments. `arg/3` is deterministic when the first two arguments are bound.

```
?- arg(2, foo(a,b), Arg).       Arg = b.
?- arg(N, foo(a,b), Arg).       N = 1, Arg = a ;  N = 2, Arg = b.
?- arg(N, 3, 3).                ERROR: Type error: `compound' expected,
↪  found `3'
```

Worked example — collecting all functors of a ground term (slides 8–9)

```
1  all_functors(Term, List) :- all_functors(Term, List, []).
2
3  all_functors(Term, List, List0) :-
4      functor(Term, Name, Arity),
5      (   Arity > 0 ->
6          List = [Name|List1],
7          all_functors1(Arity, Term, List1, List0)
8      ;   List = List0
9      ).
10
11 all_functors1(N, Term, List, List0) :-
12     (   N =:= 0 ->
13         List = List0
```

```
14      ;   arg(N, Term, Arg),
15          all_functors(Arg, List1, List0),
16          N1 is N - 1,
17          all_functors1(N1, Term, List, List1)
18      ).
```

How to read it:

- `( Cond -> Then ; Else )` is Prolog's **if-then-else**: if `Cond` succeeds, run `Then`, otherwise `Else` (only the first solution of `Cond` is used);
- the pair `List`/`List0` is a **difference list**: "the functors of this term, followed by `List0`"; it lets us concatenate without `append`;
- `all_functors1` visits the arguments from the last (`N` = arity) down to the first: the functors of argument `N` go in front of `List0`, and the functors of arguments $1 \ldots N-1$ go in front of those. So the final order is left to right.

```
?- all_functors(foo(bar([1,2],zip,baz(3)),bar(7)), L).
L = [foo, bar, '[|]', '[|]', baz, bar].          % slides: '.' instead of '[|]'
?- X = bar(42), all_functors(foo(X), L).
X = bar(42), L = [foo, bar].
?- all_functors(foo(X), L), X = bar(42).
ERROR: Arguments are not sufficiently instantiated
```

In the first query: `foo`, then its first argument `bar(...)` gives `bar`, then the list `[1,2]` = `'[|]'(1, '[|]'(2, []))` gives two list functors (numbers and `[]` are atomic: none), `zip` is an atom (none), `baz(3)` gives `baz`; finally the second argument `bar(7)` gives `bar`. The last query fails because `functor(X, Name, Arity)` is called with everything unbound: **the order of goals matters** for these non-logical builtins.

7.4.2 Term comparison

[Slides · Prolog V: 10]

Key result — Standard order of terms (slide 10)

`@<`, `@=<`, `@>`, `@>=`, `==`, `\==` work like `<`, `=<`, `>`, `>=`, `=:=`, `=\=` but on *any* terms (the sorting predicates are built on them):

- **Variables `@<` numbers `@<` atoms `@<` compound terms**;
- atoms are compared lexicographically (alphabetically), numbers numerically;
- compound terms are compared first by **arity**, then by **functor name**, then by the **arguments** in order;
- `==` and `\==` check whether terms are *identical*, without binding any variables.

Beware of variables in terms being sorted!

Worked example — comparing terms

[Not in the slides: added material]

Query	Result and why
`X @< 1`	true (variables come first)
`42 @< abc`	true (numbers before atoms)
`apple @< banana`	true (alphabetical)
`z(1,1) @< a(1,1,1)`	true (arity 2 before arity 3, the name is not even looked at)
`f(1,b) @< f(1,c)`	true (same arity and name: compare arguments)
`X == Y`	false (two different unbound variables)
`X = Y, X == Y`	true (after unification they are the same variable)
`msort([b,2,f(x),a,1], L)`	`L = [1, 2, a, b, f(x)]`

7.4.3 Variable and type testing

[Slides · Prolog V: 11–17]

The problem: `rev/2` in two directions. The simple list reversal

```
1 rev([], []).
2 rev([A|BC], R) :-
3     rev(BC, CB),
4     append(CB, [A], R).
```

works "forwards" (`?- rev([1,2,3], R).` gives `R = [3,2,1]`), but used "backwards" (`?- rev(L, [3,2,1]).`) it goes into an **infinite loop after finding the correct answer**: the recursive call `rev(BC, CB)`, with both arguments unknown, produces longer and longer lists; once the right length has been found, no other length will be correct, but Prolog keeps trying. Exchanging the calls:

```
1 rev([], []).
2 rev([A|BC], R) :-
3     append(CB, [A], R),
4     rev(BC, CB).
```

fixes the backwards mode (when `R` is bound, `append/3` has only one solution), but now `rev/2` loops in the *forwards* mode after finding the solution (`append(CB, [A], R)` with `R` unbound generates longer and longer lists).

Definition — `var/1` and other non-logical builtins (slides 13–14)

To implement a reversible version we must choose the goal order depending on *which arguments are bound*. `var(X)` succeeds if `X` is an unbound variable, and

fails if not: `?- var(_).` is true, `?- var(a).` is false. Similar tests:

`nonvar(X)`	`X` is bound
`ground(X)`	`X` is bound, and every variable in its term is ground (no variables at all)
`atom(X)`	`X` is bound to an atom
`integer(X)`, `float(X)`, `number(X)`	`X` is bound to an integer / float / number
`atomic(X)`	`atom(X) ; number(X)`
`compound(X)`	`X` is bound to a compound term

All of them can make conjunction **not commutative**: `var(X), X = 1` succeeds, `X = 1, var(X)` fails. Logically, "A and B" should not depend on the order.

Worked example — defining `length/2` in both directions (slides 15–17)

When the list is supplied:

```
len1([], 0).
len1([_|L], N) :-
    len1(L, N1),
    N is N1 + 1.
```

When the length is supplied:

```
len2([], 0).
len2([_|L], N) :-
    N > 0,
    N1 is N - 1,
    len2(L, N1).
```

A version that works in both modes chooses with `var/1`:

```
len(L, N) :-
    (   var(N) ->
        len1(L, N)
    ;   len2(L, N)
    ).
```

NB: don't check whether `L` is a variable: even if it is not, its tail may be (e.g. `[a|T]`).

```
?- len(L, 4).          L = [_, _, _, _].
?- len([a,b,c], N).    N = 3.
?- len(L, N).          L = [], N = 0 ;  L = [_], N = 1 ;  L = [_, _], N =
↪  2 ; ...
```

Why the two helpers: `len1` cannot be used with `N` known and `L` unknown (`N is N1 + 1` with `N1` unknown is an error, after the recursion produced infinitely many

candidates); `len2` needs `N` known (`N > 0` would raise an error). The name `len` is used because `length/2` is a built-in that cannot be redefined in SWI-Prolog.

7.4.4 Program access and manipulation

[Slides · Prolog V: 18–24]

So far programs were assumed to be resident in memory, without asking how they are represented or how they got there. Many applications need to *access* the clauses of the program, and to *modify* it at run time (adding and deleting clauses).

Definition — clause/2, assert/1, asserta/1, retract/1 (slides 19–21)

- `clause(Head, Body)`: must be called with `Head` instantiated. The program is searched for the first clause whose head unifies with `Head`; its head and body are unified with `Head` and `Body`. On backtracking it succeeds once for each unifiable clause. Facts have the atom `true` as their body. Clauses cannot be accessed via their body.
- `assert(Clause)`: adds `Clause` as the **last** clause of the corresponding predicate (also `assertz`). `asserta(Clause)` adds it at the **beginning**. If `Clause` is uninstantiated, an error occurs.
- `retract(Clause)`: removes the first clause unifying with `Clause`. To retract `a :- b, c, d.` write `retract((a :- (b, c, d)))` or `retract((a :- Body))`.

Worked example — the program as data

[Not in the slides: added material]

```
?- clause(grandparent(X, Y), Body).   Body = (parent(X, _A), parent(_A, Y)).
?- clause(parent(diana, C), Body).    C = william, Body = true ;  C = harry,
↪  Body = true.
?- assert(visited(rome)), assert(visited(paris)), asserta(visited(oslo)).
?- findall(C, visited(C), L).          L = [oslo, rome, paris].
?- retract(visited(rome)), findall(C, visited(C), L).   L = [oslo, paris].
```

This is how a Prolog agent updates its beliefs when the world changes: `retract` the old fact, `assert` the new one (AgentSpeak's `+b` and `-b`, Section 8).

Watch out — Declare predicates you modify as dynamic

In SWI-Prolog, predicates loaded from a file are *static*: `asserting` a clause of `fib_stupid/2` defined in the file raises "No permission to modify static procedure". Add the directive `:- dynamic fib_stupid/2.` at the top of the file (same for `fib_smart/2` and `visited/1`).

Assert in action: three Fibonacci programs (slides 22–24). All three print a message when the recursive clause is used, so we can count the calls. They have

the `N > 1` guard (Section 7.3) after the `write`.

```
fib_print(0, 1).
fib_print(1, 1).
fib_print(N, F) :-
    write('Call of fib_print on '), write(N), nl,
    N > 1,
    N1 is N - 1,
    N2 is N - 2,
    fib_print(N1, F1),
    fib_print(N2, F2),
    F is F1 + F2.
```

`fib_stupid` is identical but ends with `F is F1 + F2, assert(fib_stupid(N, F))`; `fib_smart` ends with `F is F1 + F2, asserta(fib_smart(N, F))`.

Worked example — what each version prints for `?- X(4, F).`

[Not in the slides: added material]

Version	Printed calls	Why
`fib_print`	4, 3, 2, 2	plain recursion: `fib(2)` is computed twice (once inside `fib(3)`, once as $N-2$ of 4). For larger N the calls grow exponentially.
`fib_stupid`	4, 3, 2, 2	`assert` adds the result *after* the recursive clause: when `fib_stupid(2, F)` is called again, the rule is tried *before* the stored fact, so the memory is never used. A second query prints everything again (and adds duplicate facts).
`fib_smart`	4, 3, 2	`asserta` puts `fib_smart(2, 2)` *first*: the second call `fib_smart(2, F2)` matches the stored fact immediately. After the query, `fib_smart(4, 5)`, `(3, 3)`, `(2, 2)` are stored: a second `?- fib_smart(4, F).` prints nothing, and `fib_smart(5, F)` prints only 5.

All three answer `F = 5`. `fib_smart` is **memoisation** (dynamic programming): each value is computed once, so the cost becomes linear.

7.5 Logical foundations

[Slides · Prolog III: 1–9]

Why does Prolog compute correct answers? Because running a Prolog program is building a *proof* in first-order logic, with a single, very efficient inference rule: **SLD resolution**. This deck was shown last; it ties Part IV back to Part II.

7.5.1 Clauses and Horn clauses

[Slides · Prolog III: 4–6]

Definition — Clause (slide 4)

A **clause** is a disjunction of atoms or their negations (**literals**), where the variables are implicitly *universally quantified*:

$$A_1 \vee A_2 \vee \cdots \vee A_n \vee \neg B_1 \vee \neg B_2 \vee \cdots \vee \neg B_m$$

It is equivalent to

$$(A_1 \vee \cdots \vee A_n) \leftarrow (B_1 \wedge \cdots \wedge B_m)$$

usually written (conclusion ← precondition) as $A_1, \ldots, A_n \leftarrow B_1, \ldots, B_m$.

Why are the two forms equivalent? With the rules of Section 2.3:

$$\begin{aligned}
&(A_1 \vee \cdots \vee A_n) \leftarrow (B_1 \wedge \cdots \wedge B_m) \\
&\quad \equiv \neg(B_1 \wedge \cdots \wedge B_m) \vee (A_1 \vee \cdots \vee A_n) && \triangleleft\ q \leftarrow p \ \equiv\ \neg p \vee q \\
&\quad \equiv (\neg B_1 \vee \cdots \vee \neg B_m) \vee A_1 \vee \cdots \vee A_n && \triangleleft\ \text{De Morgan} \\
&\quad \equiv A_1 \vee \cdots \vee A_n \vee \neg B_1 \vee \cdots \vee \neg B_m && \triangleleft\ \text{commutativity}
\end{aligned}$$

The positive literals end up on the left (conclusions), the negative ones on the right (conditions).

Definition — Horn clause (slide 5)

A **Horn clause** is a clause with **at most one positive literal** (a literal is a predicate symbol applied to its arguments, or its negation).

Kind	Logic	Prolog	Positive literals
definite clause: **rule**	$A \leftarrow B_1, \ldots, B_m$	`a :- b1, ..., bm.`	1
definite clause: **fact**	$A \leftarrow$	`a.`	1
negative clause: **goal**	$\leftarrow B_1, \ldots, B_m$	`?- b1, ..., bm.`	0

Key result — Programs and questions (slide 6)

- **Definite clauses represent knowledge**: a **logic program** is a set of definite clauses.
- **Negative clauses are questions**: the query `?- b1, ..., bm.` asks whether $\exists X_1 \ldots X_n.\, (B_1 \wedge \cdots \wedge B_m)$ holds (the X_i are the variables of the query).
- To answer, Prolog uses **refutation**: it adds the *negation* of the question and tries to derive a contradiction:

$$\begin{aligned}
&\neg\, \exists X_1 \ldots X_n.\, (B_1 \wedge \cdots \wedge B_m) \\
&\quad \equiv \forall X_1 \ldots X_n.\, (\neg B_1 \vee \cdots \vee \neg B_m) && \triangleleft\ \text{duality, De Morgan} \\
&\quad \equiv \leftarrow B_1, \ldots, B_m && \triangleleft\ \text{a negative clause}
\end{aligned}$$

Intuition — Refutation = proof by contradiction

It is the $\neg$ Intro idea of natural deduction (Section 2.4): to show that "someone is mortal", assume "nobody is mortal" and derive a contradiction from it and the program. The contradiction is the **empty clause** $\square$ (a disjunction with no literals, which can never be true). The bindings used along the way tell *who* is mortal: they are the answer.

7.5.2 SLD resolution

[Slides · Prolog III: 7]

Definition — SLD resolution (slide 7)

Selection-function, **L**inear resolution for **D**efinite clauses. Start from a definite program P and a goal

$$G_0 = \leftarrow A_1, \ldots, A_m \qquad (m \geq 0).$$

1. The **selection rule** selects a sub-goal A_i.
2. Choose a program clause, **renamed** with fresh variables, $B_0 \leftarrow B_1, \ldots, B_n$ $(n \geq 0)$, whose head B_0 unifies with A_i, with most general unifier θ_1.
3. The new goal is

$$G_1 = \leftarrow (A_1, \ldots, A_{i-1}, B_1, \ldots, B_n, A_{i+1}, \ldots, A_m)\,\theta_1$$

   i.e. A_i is *replaced by the body* of the clause, and the unifier is applied to the whole goal.

Repeat until the goal is empty ($\square$: success) or no clause unifies with the selected atom (failure of that branch). The answer is the composition of all the unifiers, restricted to the variables of G_0.

Worked example — SLD derivation of `?- mortal(X).`

Program: `mortal(X) :- human(X). human(socrates).`

$G_0 = \leftarrow mortal(X)$

clause renamed $mortal(X_1) \leftarrow human(X_1), \quad \theta_1 = \{X_1/X\}$

$G_1 = \leftarrow human(X)$

clause $human(socrates) \leftarrow, \quad \theta_2 = \{X/socrates\}$

$G_2 = \square$ (empty goal: success)

Answer: $X = socrates$. It is Whiteboard 2 again: unification does the instantiation, resolution does the Modus Ponens.

7.5.3 SLD trees

[Slides · Prolog III: 8–9]

Definition — SLD tree (slide 8)

Let P be a definite program, G_0 a goal and $\mathcal{R}$ a selection rule. The SLD tree of G_0 (using P and $\mathcal{R}$) is a (possibly infinite) labelled tree such that:

- the root is labelled by G_0;
- if $\leftarrow A_1, \ldots, A_n$ is a node and A_m is the atom selected by $\mathcal{R}$, the node has one child for each clause $A \leftarrow B_1, \ldots, B_q$ of P such that A and A_m are unifiable; the child is

$$\leftarrow (A_1, \ldots, A_{m-1}, B_1, \ldots, B_q, A_{m+1}, \ldots, A_n)\,\theta$$

  where θ is the mgu of A_m and A.

Leaves labelled $\square$ are **success** leaves (each gives an answer); leaves where no clause applies are **failure** leaves.

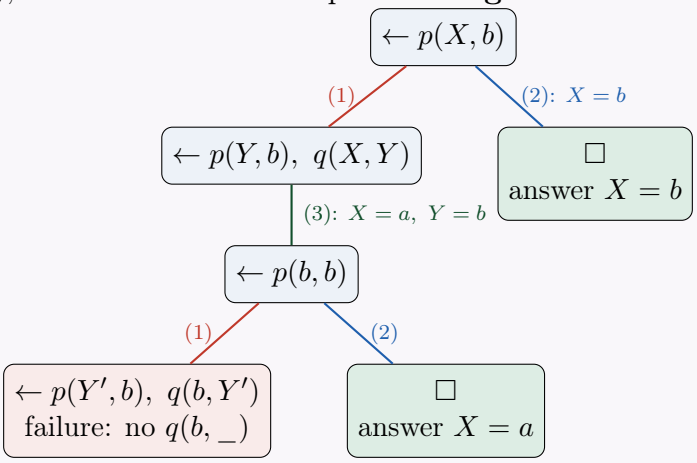
(The slide writes the child as $(A_1, \ldots, A_{m-1}, B_1, \ldots, A_{m+1}, \ldots, A_n)\theta$, dropping B_q: the whole body $B_1, \ldots, B_q$ replaces A_m.)

Worked example — SLD tree with the *rightmost* selection rule (slide 9)

Program (colours as on the slide):

(1) $p(X, Z) \leftarrow p(Y, Z), q(X, Y)$ (2) $p(X, X)$ (3) $q(a, b)$

Goal $\leftarrow p(X, b)$; the selection function picks the **rightmost** atom.



Step by step:

1. Root $\leftarrow p(X, b)$: only one atom. Clause (1), renamed $p(X_1, Z_1) \leftarrow p(Y_1, Z_1), q(X_1, Y_1)$, unifies with $X_1 = X$, $Z_1 = b$: child $\leftarrow p(Y, b), q(X, Y)$ (writing Y for Y_1). Clause (2), $p(X_2, X_2)$, unifies with $X_2 = X = b$: child $\square$, answer $X = b$.
2. Node $\leftarrow p(Y, b), q(X, Y)$: rightmost atom $q(X, Y)$; only clause (3) unifies: $X = a$, $Y = b$; child $\leftarrow p(b, b)$.
3. Node $\leftarrow p(b, b)$: clause (1) gives $\leftarrow p(Y', b), q(b, Y')$, whose rightmost atom

$q(b, Y')$ unifies with no clause: failure. Clause (2) unifies $p(b, b)$ with $p(X_2, X_2)$: $\square$, answer $X = a$.

The tree is *finite*: two answers, $X = b$ and $X = a$.

Watch out — What real Prolog does with the same program

[Not in the slides: added material]

Prolog's selection rule is **leftmost**, its search is **depth-first** with clauses tried **top to bottom**. With the leftmost rule the first child of $\leftarrow p(X, b)$ is $\leftarrow p(Y, b), q(X, Y)$, whose leftmost atom $p(Y, b)$ is again resolved first with clause (1): $\leftarrow p(Y', b), q(Y, Y'), q(X, Y)$, and so on. The leftmost branch of the SLD tree is *infinite*, and depth-first search explores it first: `?- p(X, b).` never returns an answer (stack overflow), although $X = b$ and $X = a$ are logical consequences of the program.

- The *set of answers* does not depend on the selection rule (every SLD tree contains the same success leaves), but the *shape* of the tree, and hence termination, does.
- Depth-first search is fast and memory-cheap, but *incomplete*: it can get lost in an infinite branch before reaching a success leaf. This is the "left recursion" problem of Section 7.1. Writing clause (2) first, or the body as `q(X, Y), p(Y, Z)`, would make Prolog find both answers.

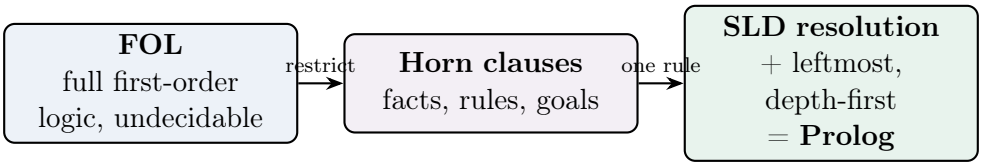


Figure 29: From logic to Prolog: restrict FOL to Horn clauses, use one inference rule (SLD resolution), and fix a search strategy. The price of efficiency is that some logically valid answers may not be found (non-termination).

Cheat sheet — Prolog

- Program = **facts** (`parent(diana, harry).`) + **rules** (`H :- B1, B2.`, "H if B1 and B2"). Query `?- G.`: Prolog tries to prove `G`, returning variable bindings; `;` asks for more answers.
- **Variables** start with upper case or `_`; they are unknowns, *bound once* (no assignment). `_` is anonymous. Arguments can be input or output ("multiple directions").
- **Terms**: atoms, numbers, variables, compound terms `f(t1,...,tn)` (functor/arity, written `name/arity`). Lists: `[]`, `[H|T]`.
- **Unification** `=`: makes two terms identical by binding variables; used for argument passing. `==` tests identity without binding. `is` evaluates arithmetic; `=:=`, `<`, `>=`... compare numbers.
- Execution: goals left to right, clauses top to bottom, depth-first search with **backtracking**. Order of goals/clauses affects efficiency and *termination*, not the logical meaning.

- Several clauses are **not** an if-then-else: every applicable clause can give an answer (absolute value bug); add guards (`X < 0`).
- `member/2`, `append/3`, `length/2`; accumulators for efficiency; `findall/3`, `bagof/3`, `setof/3` collect all solutions.
- Meta: `=..` (univ), `functor/3`, `arg/3`; standard order `@<`; type tests `var`, `nonvar`, `atom`, ...; `clause/2`, `assert/asserta`, `retract` (declare `:- dynamic`).
- Foundations: a clause is a disjunction of literals; **Horn clause**: at most one positive literal (facts, rules, goals). **SLD resolution**: select an atom of the goal, resolve it with a renamed program clause using the mgu; an **SLD tree** shows all choices; success leaves □ give answers. Prolog = leftmost selection + depth-first, top-to-bottom search.

Part V

Cognitive Agents, Communication and Platforms

8 Cognitive agents: BDI and AgentSpeak(L)

[Slides · BDI architecture: 1–19; EMAS keynote: 33–36]

This chapter brings together the two halves of the course: *symbolic* reasoning (logic, Prolog) and *agents*. A **cognitive** (or "strong") agent is described and programmed with mentalistic notions: **beliefs**, **desires** (goals) and **intentions**. The deck is part of a talk on logic-based agents given at the International Conference on Logic Programming (2012).

1. from logic-based agents to the BDI architecture;
2. the language AgentSpeak(L): syntax and informal semantics (the reasoning cycle);
3. a complete example, the *Jelly-fish busters* agent, traced step by step;
4. Jason and the other AgentSpeak interpreters (Whiteboard 7).

8.1 From logic-based agents to BDI

[Slides · BDI architecture: 1–6]

The deck opens with the two definitions of agent already seen in Section 4.1:

- **weak notion** (Jennings, Sycara, Wooldridge 1998): situated, autonomous, flexible (reactive, proactive, social);
- **strong notion**: a computer system either *conceptualised* or *implemented* using concepts that are more usually applied to humans: *mentalistic notions*

(Shoham, *Agent-oriented programming*, 1993) and *emotional notions* (Bates, *The role of emotion in believable agents*, 1994).

Logic-based agents (slide 5). The simplest way to give an agent "a mind" is to give it a logic program: knowledge as clauses, reasoning as proof.

```
mortal(X) :- human(X).
human(socrates).
?- mortal(socrates).
yes
```

This is the "traditional" approach to building intelligent systems (EMAS keynote, slide 32): **logical formulae** give a symbolic representation of the agent's environment and desired behaviour, and **logical deduction** (theorem proving) manipulates it. A pure theorem prover, however, only answers questions; an agent must also *act*, pursue goals over time and react to changes. This is what the BDI architecture adds.

Key result — AgentSpeak(L) (slide 6)

The **BDI architecture** is one of the best known and most studied architectures for **cognitive agents**. **AgentSpeak(L)** is an elegant, logic-based programming language inspired by the BDI architecture (A. S. Rao, "AgentSpeak(L): BDI agents speak out in a logical computable language", MAAMAW 1996).

8.2 The BDI architecture

[Slides · BDI architecture: 7; EMAS keynote: 17]

BDI stands for **Belief–Desire–Intention**. It is a *practical reasoning* architecture (Section 4.4): it models how humans decide *what to do*, in two steps: *deliberation* (which goals should I pursue?) and *means–ends reasoning* (how do I achieve them?).

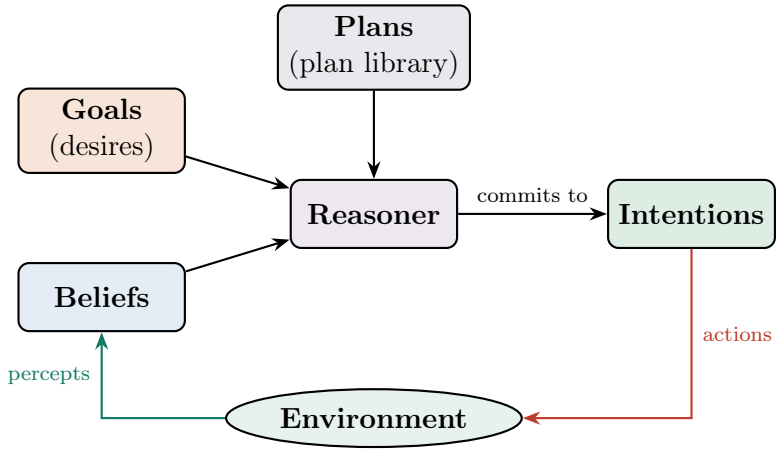


Figure 30: The BDI architecture (slide 7). Percepts update the beliefs; the reasoner, given beliefs and goals, selects plans from the library and commits to them as intentions; executing intentions acts on the environment.

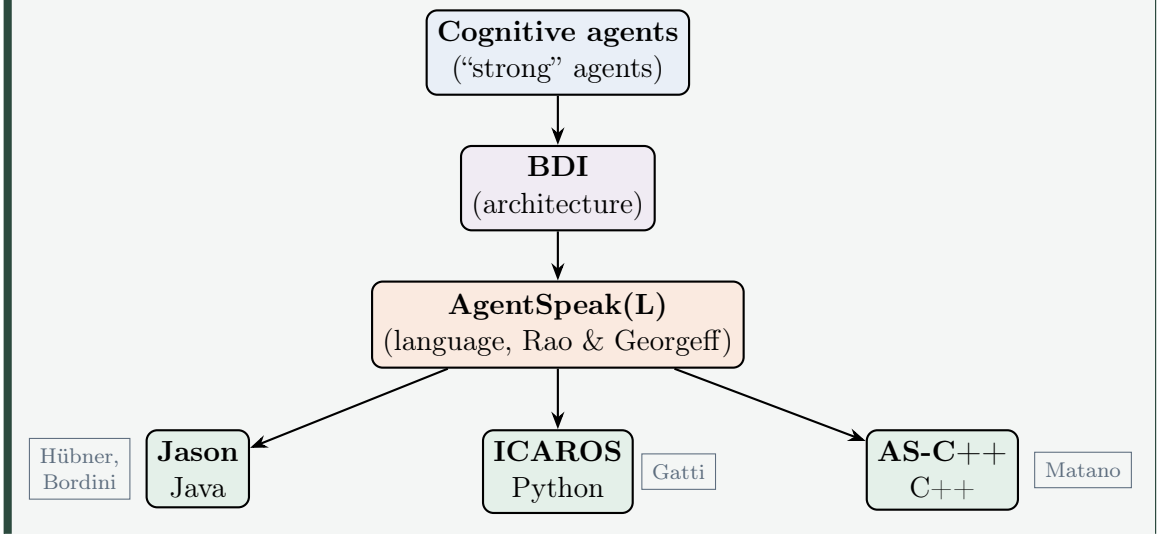
Component	What it is	Robot vacuum example
Beliefs	what the agent knows (or thinks it knows) about the world and itself; may be wrong or out of date	"the kitchen is dirty", "battery at 15%"
Desires / goals	states of the world the agent wants to bring about	"all rooms clean", "never run out of battery"
Plans	recipes: how to achieve a goal, under which conditions	"to clean a room: go there, sweep, report"
Intentions	the plans the agent has *committed* to and is executing now	"currently: going back to the dock"

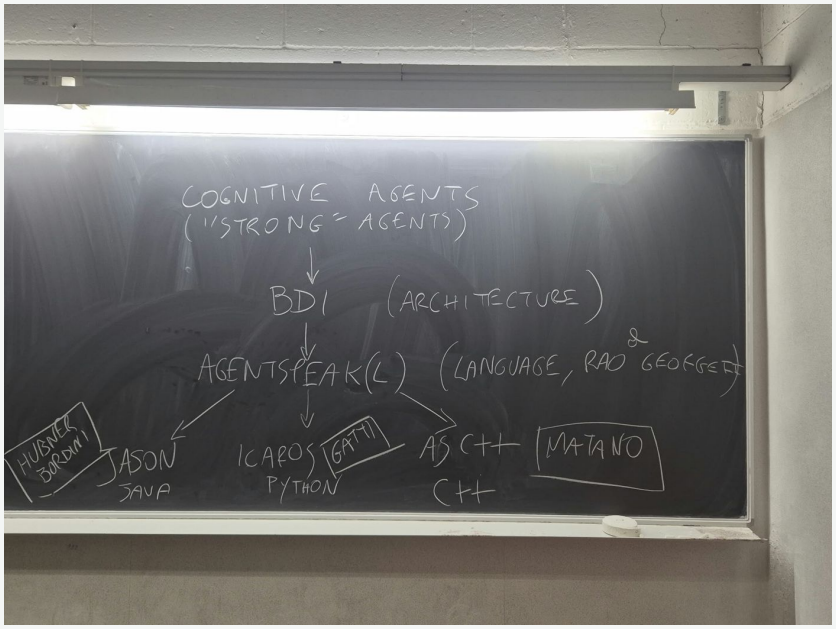
Table 19: The BDI components with an everyday agent.

Intuition — Why intentions, if we already have goals?

Desires can be many and conflicting ("clean everything" and "save battery"). An intention is a desire the agent has *committed* to: it will not reconsider it at every instant (that would be too slow in a dynamic world, the problem of the deliberative architectures of Section 5.4), but it will drop or change it if beliefs make it impossible or pointless. Intentions give *stability* (proactiveness); events and belief updates give *reactivity*. BDI is a principled way of balancing the two.

Whiteboard 7 — From cognitive agents to Jason written during the BDI / Jason lectures





Original photo of the whiteboard

Reading. Three levels of abstraction: a *kind of agent* (cognitive, strong agents with mentalistic notions), an *architecture* that realises it (BDI), a *programming language* for that architecture (AgentSpeak(L)), and finally *interpreters* of the language in different host languages, with their authors (boxed on the board):

- **Jason** (Java), by J. F. Hübner and R. H. Bordini (with M. Wooldridge, book 2007): the reference implementation, used in the labs;
- **ICAROS** (Python), by A. Gatti;
- **AS-C++** (C++), by Matano.

The board credits AgentSpeak(L) to Rao and Georgeff: the BDI model is their joint work (1991–1995); the AgentSpeak(L) paper itself is by A. S. Rao alone (1996).

8.3 The AgentSpeak(L) language

[Slides · BDI architecture: 8; EMAS keynote: 34–36]

8.3.1 Ingredients

[Slides · EMAS keynote: 34]

- **Beliefs** represent the knowledge of the agent: ground atoms such as `location(bin, d)`, exactly like Prolog facts.
- **Goals** are beliefs (or conjunctions/disjunctions of beliefs) which must be **achieved** (`!g`: "bring about a state where `g` holds") or **tested** (`?g`: "check whether `g` holds, retrieving values for its variables") in the current state.
- **Plans** contain the *procedural* knowledge of the agent; in the full model a plan has a trigger, a context, a body, a maintenance condition, a set of "success

actions" and a set of "failure actions"; AgentSpeak(L) keeps trigger, context and body.

- **Intentions** are *partially instantiated* plans: plans the agent is executing, with some variables already bound.

8.3.2 Syntax

[Slides · BDI architecture: 8; EMAS keynote: 35]

Definition — AgentSpeak(L) grammar (slide 8)

$ag ::= bs \;\; ps$		agent = beliefs + plans
$bs ::= at_1 \dots at_n$	$(n \geq 0)$	belief base: a list of atoms
$at ::= P(t_1, \dots, t_n)$	$(n \geq 0)$	atom: predicate with terms
$ps ::= p_1 \dots p_n$	$(n \geq 1)$	plan library: at least one plan
$p ::= te : ct \leftarrow h$		plan: trigger, context, body
$te ::= +at \mid -at \mid +g \mid -g$		triggering event
$ct ::= at \mid \neg at \mid ct \wedge ct \mid \top$		context: a condition on beliefs
$h ::= a \mid g \mid u \mid h; h$		body: sequence of steps
$g ::= !at \mid ?at$		achievement / test goal
$u ::= +at \mid -at$		belief update: add / delete

(a is an action, $\top$ is "true".) In concrete syntax (Jason): `<-` for $\leftarrow$, `&` for $\wedge$, `not` for $\neg$, `;` between body steps, and each plan ends with a full stop.

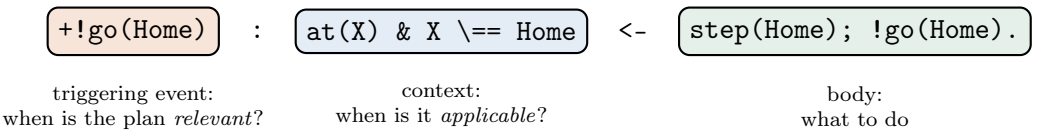


Figure 31: Anatomy of a plan (example added).

Triggering event	Meaning: the plan reacts to...
`+b`	the addition of belief `b` (e.g. a new percept)
`-b`	the deletion of belief `b`
`+!g`	a new achievement goal `g` ("how to achieve g")
`-!g`	the failure of achievement goal `g` ("what to do if g fails")
`+?g`, `-?g`	the addition / failure of a test goal (rarely used)

Table 20: Triggering events. The grammar writes $+g$ and $-g$ with g being $!at$ or $?at$.

Watch out — AgentSpeak looks like Prolog, but it is not Prolog

Beliefs and contexts are Prolog-like (unification, `not` as negation as failure). But plan bodies are *executed*, step by step, interleaved with perception: actions change the world, `!g` posts a new goal (handled by another plan), `+b`/`-b` change the belief

base (like `assert`/`retract`, Section 7.4). There is no backtracking into an action already performed.

8.3.3 Informal semantics: the reasoning cycle

[Slides · EMAS keynote: 36]

Definition — AgentSpeak(L) agent (Rao 1996, Definitions 6–8)

An agent is a tuple $\langle E, B, P, I, A, S_E, S_O, S_I \rangle$: a set of **events** E, a set of base **beliefs** B, a set of **plans** P, a set of **intentions** I, a set of **actions** A, and three **selection functions**: S_E selects an event from E; S_O selects an *option*, i.e. an applicable plan, among the applicable ones; S_I selects an intention from I.

- Each **intention** is a *stack* of partially instantiated plans $[p_1 \ddagger \ldots \ddagger p_z]$, p_1 at the bottom, p_z on top. The special intention $[+!true : true \leftarrow true]$ is the *true intention*, written $\top$.
- Each **event** is a pair $\langle e, i \rangle$: a triggering event e and the intention i that generated it. If $i = \top$ the event is **external** (from perception or from outside); otherwise it is **internal** (a subgoal posted by a plan body).

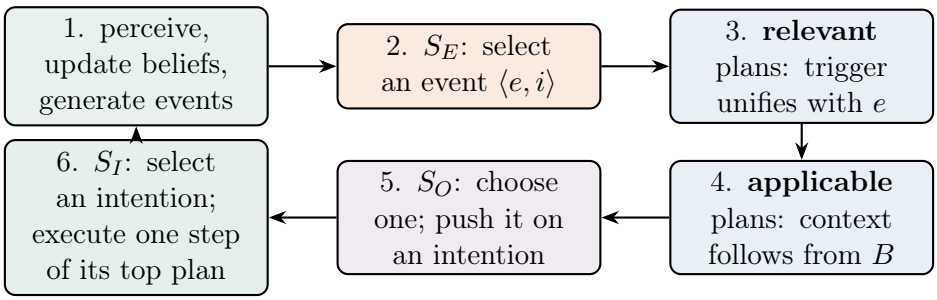


Figure 32: The reasoning cycle of an AgentSpeak(L) interpreter, repeated forever.

In step 5, the new plan, with the unifier applied, becomes a **new intention** if the event is external; if the event is internal (a subgoal `!g` of intention i), it is **pushed on top of** i: the subgoal must be completed before i continues. In step 6, executing the top plan's next body step means: perform an action; or, for `!g`, generate the internal event $\langle +!g, i \rangle$ and suspend i; or, for `?g`, query the belief base; or, for `+b`/`-b`, update the beliefs (generating events). When a plan's body is finished it is popped from the stack.

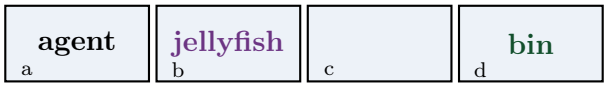
Intuition — Relevant vs. applicable

Relevant = "this plan is about this event" (the trigger matches): like a recipe in the right chapter of the cookbook. *Applicable* = "and I can use it now" (the context holds in my current beliefs): I have the ingredients. Several plans can be relevant for the same goal, each applicable in different situations: this is how an agent chooses *context-dependent* behaviour.

8.4 Example: Jelly-fish busters

[Slides · BDI architecture: 9–18]

A one-dimensional world of four cells a, b, c, d. The agent (SpongeBob in the slides) starts in a, a jellyfish is in b, the bin is in d. Goal: catch the jellyfish and throw it into the bin.




```
/* Initial beliefs */
adjacent(a, b).
adjacent(b, c).
adjacent(c, d).
location(myself, a).
location(jellyfish, b).
location(bin, d).

/* Initial goal */
!bust_jellyfishes.

/* Plan 1 */
+!bust_jellyfishes :
    location(myself, X) &
    location(jellyfish, X) &
    location(bin, Y)
 <- !pick(jellyfish, X);
    move_to(Y);
    !drop(jellyfish, Y).

/* Plan 2 */
+!bust_jellyfishes :
    location(myself, X) &
    not location(jellyfish, X) &
    adjacent(X, Z)
 <- move_to(Z);
    !bust_jellyfishes.
```

- **Initial beliefs** (slide 9): the map (`adjacent/2`) and the positions (`location/2`).
- **Plan 1** (slide 10): *if* I am where the jellyfish is, *then* pick it, move to the bin, drop it. `!pick` and `!drop` are subgoals.
- **Plan 2** (slide 11): *if* the jellyfish is not here, *then* move to an adjacent cell and try again: a *recursive* goal, like a recursive Prolog predicate.
- **Initial goal** (slide 12): `!bust_jellyfishes`, which generates the external event `+!bust_jellyfishes` at start-up.

Worked example — the run, step by step (slides 13–17)

Step	Event	Plan selection	Effect
1	`+!bust_-jellyfishes` (external)	Relevant: plans 1 and 2. Plan 1: context `location(myself,X)` gives `X = a`, but `location(jellyfish, a)` is false: not applicable. Plan 2: `X = a`, `not location(jellyfish, a)` true, `adjacent(a, Z)` gives `Z = b`: applicable.	new intention; executes `move_to(b)`
2	—	the body continues	a percept updates the agent's position to `b`
3	`+!bust_-jellyfishes` (internal)	posted by plan 2's body. Plan 1: `X = b`, `location(jellyfish, b)` true, `location(bin, Y)` gives `Y = d`: applicable. Plan 2: not applicable (`not location(jellyfish, b)` is false).	plan 1 pushed on the same intention
4	`+!pick(jellyfish,b)`	handled by a `pick` plan (not shown)	jellyfish picked up (slide 15)
5	—	`move_to(d)`	agent carries it to `d` (slide 16)
6	`+!drop(jellyfish,d)`	handled by a `drop` plan (not shown)	jellyfish in the bin (slide 17); plans popped: intention completed

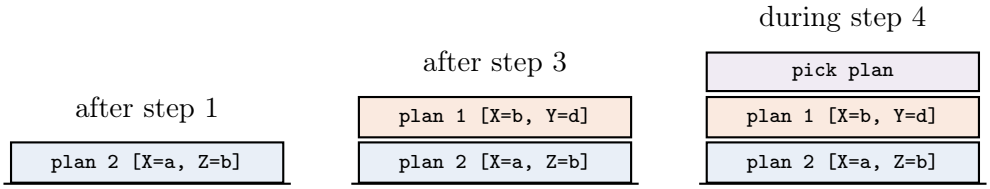


Figure 33: The single intention of the run as a stack of partially instantiated plans: each subgoal pushes a new plan on top; when the top plan finishes it is popped and the plan below continues.

Watch out — What the slides leave implicit

- The program has no plans for `+!pick(...)` and `+!drop(...)`: in a complete program they would be plans (or be written as actions `pick(...)`, `drop(...)` without `!`). An achievement goal with no relevant plan *fails*, and so does the intention.
- The belief `location(myself, ...)` must be updated after each move: in Jason this is done by the *environment* through percepts (or by the plan with

`-+location(myself, Z)`). Otherwise plan 2 would move to `b` forever.
- If the jellyfish were in no reachable cell, plan 2 would reach `d`, find no `adjacent(d, _)`, and no plan would be applicable: goal failure (a `-!bust_jellyfishes` plan could handle it).

Why this matters in practice — Jason (slide 18)

Jason (Bordini, Hübner, Wooldridge, *Programming Multi-Agent Systems in AgentSpeak Using Jason*, Wiley 2007) is the best-known interpreter of an extended AgentSpeak, written in Java. Slide 18 shows its "Mars world" example: robots `r1` and `r2` on a grid, collecting garbage `G` and carrying it to an incinerator, with the mind inspector showing beliefs, events and intentions at each reasoning cycle. Jason adds to AgentSpeak(L) communication between agents (via speech-act based messages, Section 9), annotations on beliefs (e.g. `[source(percept)]`), environments in Java, and much more; it is the platform of the course labs.

Cheat sheet — BDI and AgentSpeak(L)

- **Logic-based agents**: logical formulae represent the environment and the desired behaviour; deduction decides what to do.
- **BDI**: *beliefs* (information about the world), *desires/goals* (states to achieve), *intentions* (plans the agent has committed to); a *reasoner* selects plans from a *plan library* and acts on the environment.
- **AgentSpeak(L)** (Rao, 1996): agent = beliefs + plans. Plan `te : ct <- h`: *triggering event* (`+b`, `-b`, `+!g`, `-!g`, ...), *context* (condition on beliefs), *body* (actions `a`, goals `!g` achieve / `?g` test, belief updates `+b`/`-b`).
- Reasoning cycle: select an *event*; find *relevant* plans (trigger unifies) and *applicable* plans (context is a logical consequence of the beliefs); select one (option); push it on an *intention* (new intention for an external event, on top of the generating intention for a subgoal); select an intention and execute one step of its top plan.
- Intentions = stacks of partially instantiated plans.
- Family tree (Whiteboard 7): cognitive agents → BDI (architecture) → AgentSpeak(L) (language) → Jason (Java), ICAROS (Python), AS-C++ (C++).

9 Agent communication

[Slides · Communication: 1–23]

Agents in a MAS have limited viewpoints and no global control (Section 4.2): they must *talk* to each other to coordinate, cooperate and negotiate. This chapter (from a PhD course on distributed systems with MAS, University of Milano-Bicocca, 2020) explains the idea that **communication is a form of action** and how the FIPA standards turn it into a precise agent communication language.

9.1 Communication as action

[Slides · Communication: 2–8]

Definition — Communication (slide 2)

Communication, in general, is the **intentional** exchange of information brought about by the production and perception of **signs** drawn from a **shared system of conventional signs**.

Two views of language:

- **"classical" view** (before 1953): language consists of sentences that are true or false, as in logic (Part II);
- **"modern" view** (after 1953): language is a **form of action**.

Who	Work	Key idea
Wittgenstein	*Philosophical Investigations* (1953)	The structure of reality determines the structure of language. **Meaning is determined by use**, in the context of a "language game": words are defined not by the objects they designate, nor by the mental representations associated with them, but by how they are used in effective, ordinary communication.
Austin	*How to Do Things with Words* (1962)	**In saying something, we do something**: a minister saying "I now pronounce you husband and wife" does not describe a marriage, he *makes* it.
Searle	*Speech Act Theory* (1969)	Each **illocution** can be described by what it attempts to do: an assertion is an undertaking that something is the case; a question is an attempt to elicit information; thanking is an expression of gratitude. What counts is the assumed intent of the speaker.

Table 21: From "language describes" to "language acts" (slides 3–5).

Intuition — Locution, illocution, perlocution

[Not in the slides: added material]

Austin distinguishes three levels in "Close the window": the *locution* (the words uttered), the *illocution* (the act performed *in* saying them: a request), and the

perlocution (the effect obtained *by* saying them: the hearer closes the window, or refuses). Agent communication languages standardise the *illocutionary* level: a message says explicitly which act it performs (`request`, `inform`, ...).

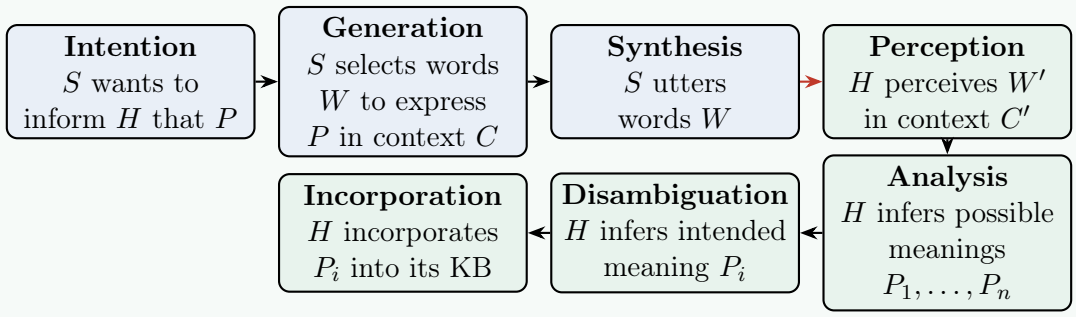
Key result — Speech acts achieve the speaker's goals (slide 6)

Speech act	Example
inform	"There's a pit in front of you"
query	"Can you see the gold?"
command	"Pick it up"
promise	"I'll share the gold with you"
acknowledge	"OK"

(The examples come from the "wumpus world" of Russell and Norvig: agents exploring a cave with pits and gold.) **Speech act planning** requires knowledge of the situation, of semantic and syntactic conventions, and of the hearer's goals, knowledge base and rationality.

Treating speech as action means that an agent can *plan* its utterances like any other action: "I want H to believe P; informing H that P has this effect if H trusts me; so I inform".

Key result — Stages in communication (slide 7)



Blue: speaker S; green: hearer H. Note W' and C': what the hearer perceives and its context may differ from what the speaker uttered and meant.

Watch out — How could this go wrong? (slide 8)

- **Insincerity**: S does not believe P (a lie, or a malicious agent);
- **recognition errors**: the slide's joke "speech wreck ignition failure" is how a speech recogniser might hear "speech recognition failure" ($W' \neq W$);
- **ambiguous utterance**: several meanings $P_1, \ldots, P_n$ remain;
- **differing understanding of the current context** ($C \neq C'$).

Software agents avoid most of these problems by *design*: messages are structured, the act is explicit, and the meaning of the words is fixed by a shared ontology (next sub-sections). Insincerity remains a real issue in open systems (trust and reputation).

9.2 FIPA ACL: the agent communication language

[Slides · Communication: 9–18]

FIPA (Foundation for Intelligent Physical Agents, Section 6.3) publishes the standard specifications for agent platforms (slide 9 lists them by subject: abstract architecture, agent management, message transport, agent communication, applications). The **agent communication** specifications (slide 10) have three parts:

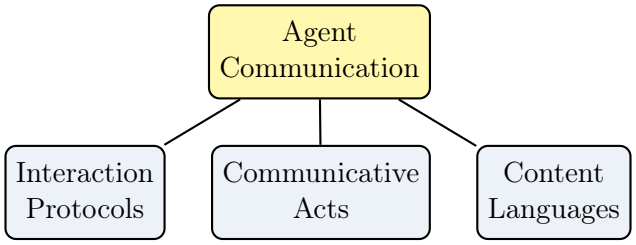


Figure 34: FIPA communication specifications (slide 10): *what* kind of act a message performs (communicative acts), *how* its content is written (content languages), and *which sequences* of messages are allowed (interaction protocols).

9.2.1 Message structure

[Slides · Communication: 11]

Key result — FIPA ACL message parameters (slide 11)

Category	Parameter	Meaning (added)
type of communicative act	`performative`	which act: `inform`, `request`, ...
participants in communication	`sender`	who sends
	`receiver`	who receives (one or more agents)
	`reply-to`	where replies should go (if not to the sender)
content of message	`content`	what the act is about
description of content	`language`	language of the content (e.g. `fipa-sl`)
	`encoding`	encoding of the content
	`ontology`	vocabulary giving meaning to the symbols
control of conversation	`protocol`	interaction protocol being followed
	`conversation-id`	identifies the conversation
	`reply-with`	label the answer must refer to
	`in-reply-to`	label of the message being answered
	`reply-by`	deadline for the answer

Only `performative` is mandatory; most messages also have sender, receiver and

content.

Worked example — a request from a buyer to a seller

[Not in the slides: added material]

```
(request
   :sender          (agent-identifier :name buyer@shop-platform)
   :receiver        (set (agent-identifier :name seller@shop-platform))
   :content         "((action (agent-identifier :name seller)
                              (deliver book42 (address Genova))))"
   :language        fipa-sl
   :ontology        book-trading
   :protocol        fipa-request
   :conversation-id order-7781
   :reply-with      req-1
   :reply-by        20261110T120000000Z)
```

The seller's answer `agree` will carry `:in-reply-to req-1` and the same `:conversation-id`, so the buyer can match it with its question even if it is running many conversations at once.

9.2.2 Communicative acts

[Slides · Communication: 12–13]

Communicative act	Meaning (from the FIPA Communicative Act Library, added)
`accept-proposal`	accept a previously submitted proposal to perform an action
`agree`	agree to perform some action, possibly in the future
`cancel`	tell another agent that the sender no longer intends it to perform an action requested earlier
`cfp` (call for proposal)	call for proposals to perform a given action
`confirm`	tell the receiver that a proposition is true, when it is uncertain about it
`disconfirm`	tell the receiver that a proposition is false, when it believes or suspects it is true
`failure`	tell that an action was attempted but failed
`inform`	tell that a proposition is true
`inform-if`	tell *whether* a proposition is true
`inform-ref`	tell the object that corresponds to a description (e.g. "the price is 12")
`not-understood`	the sender did not understand a message
`propagate`	the receiver must forward the embedded message to the agents matching a description, and treat it as sent to itself
`propose`	submit a proposal to perform an action, under certain conditions
`proxy`	the receiver must forward the embedded message to the agents matching a description
`query-if`	ask whether a proposition is true
`query-ref`	ask for the object referred to by an expression
`refuse`	refuse to perform an action, giving the reason
`reject-proposal`	reject a proposal during a negotiation
`request`	ask the receiver to perform an action
`request-when`	ask to perform an action when a proposition becomes true
`request-whenever`	ask to perform an action every time a proposition becomes true
`subscribe`	ask to be notified of the value of a reference, and again whenever it changes

Table 22: The 22 FIPA communicative acts listed on slides 12–13.

Intuition — Families of acts

Information passing: inform, inform-if, inform-ref, confirm, disconfirm. *Requesting information*: query-if, query-ref, subscribe. *Performing actions*: request, request-when, request-whenever, agree, refuse, cancel, failure. *Negotiation*: cfp, propose, accept-proposal, reject-proposal. *Error handling*: not-understood, failure. *Forwarding*: propagate, proxy.

9.2.3 Anatomy of a communicative act: accept-proposal

[Slides · Communication: 14–18]

Each act is specified by a summary, its content, a description, a **formal model** and an example. The slides show `accept-proposal`.

- **Summary** (slide 14): the action of accepting a previously submitted proposal to perform an action.
- **Content** (slide 15): a tuple of an *action expression* (the action to be done) and a *proposition* (the conditions of the agreement).
- **Description** (slide 16): a general-purpose acceptance of a proposal that was previously submitted (typically through a `propose` act). The agent sending the acceptance informs the receiver that it intends that (at some point in the future) the receiver will perform the action, once the given precondition is, or becomes, true. The proposition indicates the preconditions the agent is attaching to the acceptance: e.g. after an offer to "hold a meeting anytime on Tuesday", the acceptance may add the condition that the meeting is at 11.00. (Future extension: an agent may intend that an action is done without necessarily intending the precondition, e.g. "I bid price p, but I am prepared to accept p'".)

Definition — Formal model of accept-proposal (slide 17)

$$\langle i, \textit{accept-proposal}(j, \langle j, act\rangle, \phi)\rangle \;\equiv\; \langle i, \textit{inform}(j, I_i\, \textit{Done}(\langle j, act\rangle, \phi))\rangle$$

$$\begin{aligned} \text{FP:}\quad & B_i\,\alpha \;\wedge\; \neg B_i\,(\textit{Bif}_j\,\alpha \vee \textit{Uif}_j\,\alpha) \\ \text{RE:}\quad & B_j\,\alpha \\ \text{where}\quad & \alpha = I_i\,\textit{Done}(\langle j, act\rangle, \phi) \end{aligned}$$

How to read it, symbol by symbol (the logic is the BDI modal logic of FIPA-SL):

- $\langle i, X\rangle$: agent i performs communicative act X;
- $B_i\,p$: *i believes p*; $I_i\,p$: *i intends p*;
- $\textit{Done}(\langle j, act\rangle, \phi)$: j has performed act, and ϕ was true when it did;
- $\textit{Bif}_j\,\alpha$: j knows whether α (believes α or believes $\neg\alpha$); $\textit{Uif}_j\,\alpha$: j is uncertain about α;
- **FP** (*feasibility precondition*): what must hold *for the sender* to perform the act sincerely: i believes α, and i does not believe that j already knows or is uncertain about α (no point in telling otherwise);
- **RE** (*rational effect*): what the sender *wants to achieve*: that j comes to believe α. The receiver is not forced to adopt it: it is the expected effect, not a guaranteed one.

In words: *accepting j's proposal means informing j that i intends that j performs the action under condition ϕ.* Every FIPA act is defined this way, in terms of mental states: this is the strong, BDI notion of agency applied to communication.

Worked example — accept-proposal (slide 18)

Agent i informs j that it accepts an offer from j to stream a given multimedia title to channel 19 when the customer is ready. Agent i will inform j of this fact when appropriate.

```
(accept-proposal
  :sender (agent-identifier :name i)
  :receiver (set (agent-identifier :name j))
  :in-reply-to bid089
  :content
    "((action (agent-identifier :name j)
              (stream-content movie1234 19))
      (B (agent-identifier :name j)
         (ready customer78)))"
  :language fipa-sl)
```

The content pairs the action (`stream-content movie1234 19`, by `j`) with the condition (`j` believes that `customer78` is ready). `:in-reply-to bid089` links the acceptance to `j`'s earlier `propose` message.

9.3 Ontologies, content languages, interaction protocols

[Slides · Communication: 19–23]

9.3.1 The FIPA Ontology Agent

[Slides · Communication: 19–21]

The FIPA communication model assumes that two agents who wish to converse **share a common ontology** for the domain of discourse: this ensures that they ascribe the same meaning to the symbols used in the message. Designers may use ontologies that are *explicit*, declaratively represented and stored somewhere, or *implicit*, encoded in the agents' software and not formally published.

Definition — Ontology

[Not in the slides: added material]

An **ontology** is a shared, explicit description of the concepts of a domain, their properties and their relations: e.g. in a book-trading ontology, `Book` has a `title` and a `price`; `sell(seller, Book)` is an action; `owns(agent, Book)` a predicate. It fixes *what the words mean*, so that "price" is understood in the same way (and the same currency!) by buyer and seller.

The FIPA *Ontology Service Specification* deals with explicit ontologies. The service is provided by a dedicated agent, the **Ontology Agent** (OA), which offers some or all of these services:

1. **discovery** of public ontologies, in order to access them;

2. **maintenance** of a set of public ontologies (register with the DF, upload, download, modify);
3. **translation** of expressions between different ontologies and/or content languages;
4. **answers** to queries about relationships between terms or between ontologies;
5. help in **identifying a shared ontology** for communication between two agents.

9.3.2 Content languages

[Slides · Communication: 22]

The FIPA Content Language specifications deal with different representations of the `content` of ACL messages. Four languages are specified (others can be added):

Spec	Language	Note (added)
SC00008	FIPA SL (Semantic Language)	the reference language, a BDI modal logic; used in the examples above
XC00009	FIPA CCL (Constraint Choice Language)	for constraint problems
XC00010	FIPA KIF (Knowledge Interchange Format)	a first-order logic syntax
XC00011	FIPA RDF (Resource Description Framework)	Semantic Web data

(SC = standard, XC = experimental.)

9.3.3 Interaction protocols

[Slides · Communication: 23]

FIPA Interaction Protocols are pre-agreed message exchange patterns for ACL messages:

SC00026	Request	SC00031	English Auction (experimental)
SC00027	Query	SC00032	Dutch Auction (experimental)
SC00028	Request When	SC00033	Brokering
SC00029	Contract Net	SC00034	Recruiting
SC00030	Iterated Contract Net	SC00035	Subscribe
		SC00036	Propose

The **Request** protocol was drawn in Section 6.3 ("remember this protocol!"); the **Contract Net** is the FIPA version of Davis and Smith's protocol (Section 4.4):

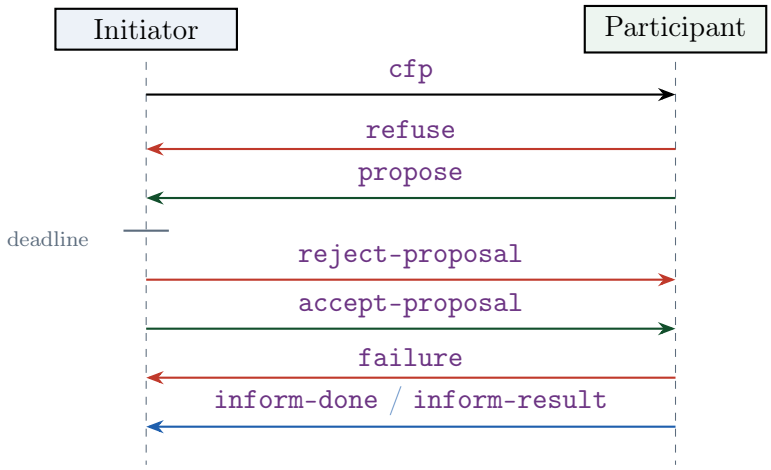


Figure 35: FIPA Contract Net (added drawing): the initiator sends a call for proposals; each participant refuses or proposes; after the deadline the initiator accepts the best proposal(s) and rejects the others; an accepted participant finally reports failure or success.

Why this matters in practice

[Not in the slides: added material]

Protocols make conversations *predictable*: an agent knows which replies are legal at each point, and a monitor can check them at run time (Section 6.6). JADE (next chapter) provides ready-made Java classes for the initiator and responder roles of these protocols, and Jason agents use the same performatives in `.send` actions.

Cheat sheet — Agent communication

- Communication = intentional exchange of information through shared conventional signs. Classical view: sentences are true/false; modern view (after 1953): **language is action**.
- Wittgenstein (1953): meaning is use (language games). Austin (1962): in saying something we do something (performatives). Searle (1969): **speech act theory**: each illocution has a point (assert, ask, thank, ...).
- Speech acts: inform, query, command, promise, acknowledge. Stages: intention, generation, synthesis, perception, analysis, disambiguation, incorporation; failures: insincerity, recognition errors, ambiguity, different contexts.
- **FIPA ACL**: messages with parameters (performative; sender, receiver, reply-to; content; language, encoding, ontology; protocol, conversation-id, reply-with, in-reply-to, reply-by) and 22 **communicative acts** (inform, request, agree, refuse, cfp, propose, accept/reject-proposal, query-if/ref, ...).
- Each act has a formal semantics: **feasibility precondition** (FP) and **rational effect** (RE), in a BDI logic (B_i believes, I_i intends).
- Shared **ontology** (Ontology Agent), **content languages** (SL, CCL, KIF, RDF), **interaction protocols** (Request, Query, Contract Net, auctions,

Subscribe, . . .).

10 JADE: a platform for multi-agent systems

[Slides · JADE: 1–19]

To develop and run a MAS we need software infrastructure: tools that transform MAS specifications (e.g. AUML diagrams) into agent code, and middleware that provides the services needed by distributed agents (naming, message transport, directories). To be *interoperable*, such infrastructures should comply with the **FIPA abstract architecture** (slides 2–3; the abstract agent platform of Section 6.3). **JADE** is one of the most well-known and widely used FIPA-compliant platforms ("Remember this framework!", EMAS keynote slide 45). This chapter explains how a JADE agent is written, how it executes its *behaviours*, how it exchanges FIPA ACL messages, and how it finds other agents through the yellow pages.

10.1 JADE agents

[Slides · JADE: 4–9]

Definition — JADE (slides 4–5)

JADE (Java Agent DEvelopment Framework; Bellifemine, Caire, Greenwood, *Developing Multi-Agent Systems with JADE*, Wiley 2007; latest release December 2022) is a software framework fully implemented in Java. It simplifies the implementation of MAS through

- a **middleware** that complies with the FIPA specifications, and
- a set of **tools** that support debugging and deployment (e.g. the sniffer that draws message exchanges, Section 4.3).

The agent platform can be **distributed** across machines, which need not share the same operating system, and the configuration can be controlled via a remote GUI. It can be changed at run time by **moving agents** from one machine to another, as and when required.

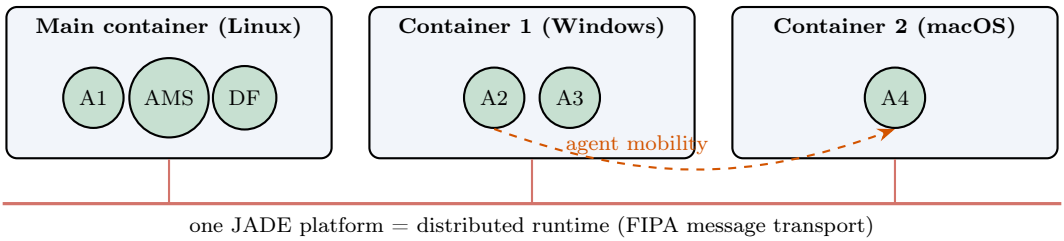


Figure 36: A JADE platform spread over three machines (added drawing). The main container hosts the AMS (white pages) and the DF (yellow pages); agents can move between containers at run time.

10.1.1 The HelloWorld agent and AIDs

[Slides · JADE: 6–7]

A **type of agent** is created by extending the `jade.core.Agent` class and redefining the `setup()` method (called when the agent starts). Each agent instance is identified by an **AID** (`jade.core.AID`), composed of a unique *name* plus some *addresses*; an agent retrieves its own AID with `getAID()`.

```
1  import jade.core.Agent;
2
3  public class HalloWorldAgent extends Agent {
4      protected void setup() {
5          System.out.println("Hallo World! my name is " + getAID().getName());
6      }
7  }
```

Key result — Names, GUID and addresses (slide 7)

- Agent names have the form `<local-name>@<platform-name>`, and the complete name must be **globally unique** (GUID).
- The default platform name is `<main-host>:<main-port>/JADE`; it can be set with the `-name` option.
- Within a single platform, agents are referred to by their (local) names only.
- Given a name, an AID is created as `new AID(localname, AID.ISLOCALNAME)` or `new AID(name, AID.ISGUID)`.
- The addresses in an AID are those of the platform's MTPs (message transport protocols) and are used **only** for communication between agents on *different* FIPA platforms.

Example: an agent started as `peter` on the platform `myhost:1099/JADE` has GUID `peter@myhost:1099/JADE`; `new AID("peter", AID.ISLOCALNAME)` refers to it from the same platform.

10.1.2 Arguments and termination

[Slides · JADE: 8–9]

It is possible to pass arguments to an agent at start-up, `java jade.Boot ... a:myPackage.MyAgent(arg1 arg2)`, and retrieve them with `getArguments()`. An agent terminates when its `doDelete()` method is called; on termination its `takeDown()` method is invoked (intended for clean-up operations).

```
1  protected void setup() {
2      System.out.println("Hallo World! my name is " + getAID().getName());
3      Object[] args = getArguments();
4      if (args != null) {
```

```
5           System.out.println("My arguments are:");
6           for (int i = 0; i < args.length; ++i) {
7               System.out.println("- " + args[i]);
8           }
9       }
10      doDelete();         // terminate immediately after greeting
11  }
12
13  protected void takeDown() {
14      System.out.println("Bye...");
15  }
```

Worked example — what is printed

[Not in the slides: added material]

Started as `a:myPackage.MyAgent(red 42)` on platform `myhost:1099/JADE`:

```
Hallo World! my name is a@myhost:1099/JADE
My arguments are:
- red
- 42
Bye...
```

`doDelete()` does not stop the agent in the middle of `setup()`: `setup()` completes, then the agent sees the deletion request, calls `takeDown()` and dies (see the execution model in the next sub-section). Arguments arrive as `Objects` (strings from the command line).

10.2 Behaviours and the execution model

[Slides · JADE: 10–12]

Definition — Behaviour (slide 10)

The actual job of an agent is typically carried out within **behaviours**. Behaviours are created by extending `jade.core.behaviours.Behaviour`. To make an agent execute a task, create an instance of the corresponding subclass and call the agent's `addBehaviour()` method. Each subclass must implement:

- `public void action()`: what the behaviour actually does;
- `public boolean done()`: whether the behaviour is finished.

Key result — Cooperative scheduling (slide 11)

An agent can execute several behaviours in parallel, however behaviour scheduling is **not preemptive** but **cooperative**, and everything occurs within a **single Java thread**: *a behaviour switch occurs only when the `action()` method of the currently scheduled behaviour returns.*

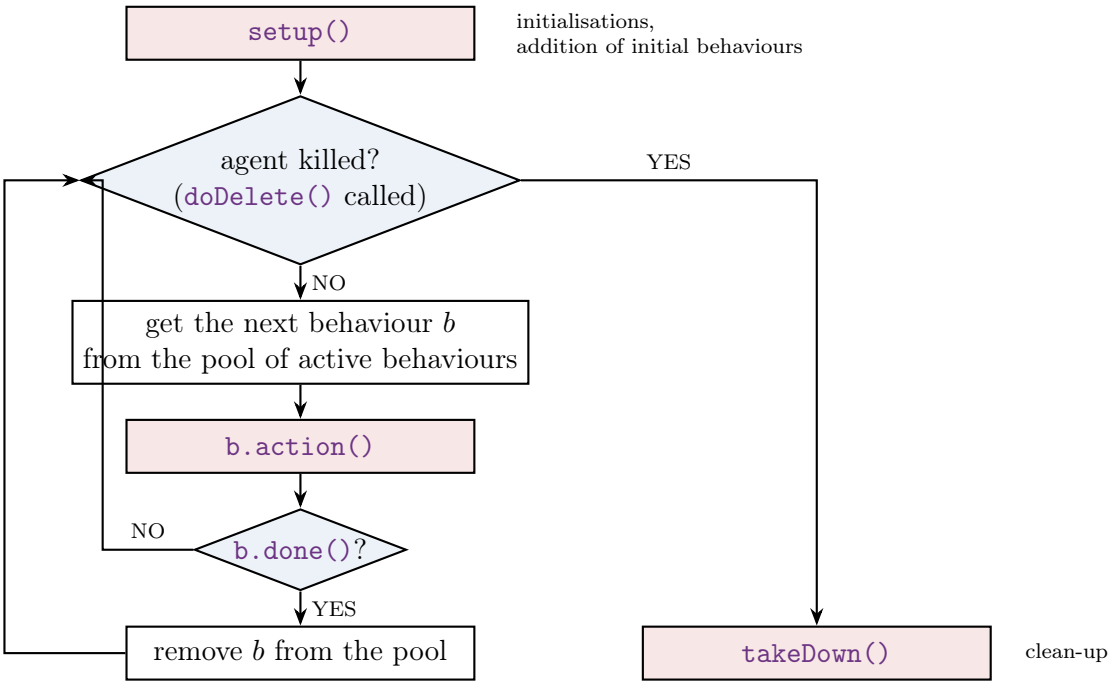


Figure 37: The agent execution model (slide 12). In red the methods the programmer writes. The loop in the middle is the agent's "life": pick a behaviour, run `action()` once, check `done()`, repeat.

Watch out — Never block inside `action()`

Since all behaviours share one thread, an `action()` that never returns (an infinite loop, a `Thread.sleep(60000)`, a blocking read) freezes *all* the behaviours of the agent, including those that should react to messages. Write `action()` as a short step and let the scheduler call it again: long tasks are split into many short `action()` calls, keeping their progress in fields of the behaviour.

Worked example — three kinds of behaviour

[Not in the slides: added material]

JADE provides ready-made subclasses: `OneShotBehaviour` (`done()` always true: runs once), `CyclicBehaviour` (`done()` always false: runs forever), `TickerBehaviour` (runs periodically). A custom behaviour that counts to three:

```
1 public class CountTo3 extends Behaviour {
2     private int i = 0;
3     public void action() { i++; System.out.println("step " + i); }
4     public boolean done() { return i == 3; }
5 }
6 // in setup():  addBehaviour(new CountTo3());
```

If an agent has `CountTo3` and a cyclic behaviour `C` printing "tick", the output interleaves: `step 1`, `tick`, `step 2`, `tick`, `step 3`, `tick`, `tick`, ... Each `action()`

runs to completion; after `step 3`, `done()` is true and `CountTo3` is removed from the pool, while `C` runs forever (round-robin among active behaviours).

10.3 Communication and yellow pages

[Slides · JADE: 13–19]

10.3.1 The communication model

[Slides · JADE: 13–15]

JADE communication is based on **asynchronous message passing**, and the message format is defined by the FIPA **ACL** language (Section 9.2).



Figure 38: The communication model (slide 13): the sender never waits for the receiver; the runtime delivers the message into the receiver's private queue, where it stays until the receiver reads it.

Messages are instances of `jade.lang.acl.ACLMessage`, with accessor methods for all the ACL fields (slide 14):

`get/setPerformative()` `get/setSender()` `add/getAllReceiver()`
`get/setLanguage()` `get/setOntology()` `get/setContent()`, ...

Key result — Sending and receiving (slide 15)

Sending a message = creating an `ACLMessage` and calling the agent's `send()`:

```
ACLMessage msg = new ACLMessage(ACLMessage.INFORM);
msg.addReceiver(new AID("Peter", AID.ISLOCALNAME));
msg.setLanguage("English");
msg.setOntology("Weather-Forecast-Ontology");
msg.setContent("Today it's raining");
send(msg);
```

Reading from the private message queue uses `receive()`, which returns the first message of the queue (removing it), or `null` if the queue is empty:

```
ACLMessage msg = receive();
if (msg != null) {
```

```
    // Process the message
}
```

10.3.2 Waiting for messages: block()

[Slides · JADE: 16–18]

A behaviour that processes incoming messages does not know when a message will arrive. It could poll the queue by calling `myAgent.receive()` continuously, but this would waste CPU time. The `block()` method of `Behaviour` removes the behaviour from the pool of active behaviours and puts it in a *blocked* state (**it is not a blocking call**: `action()` still returns normally). Each time a message is received, all blocked behaviours are put back into the pool and have a chance to read it.

Key result — The strongly recommended pattern (slide 16)

```
public void action() {
    ACLMessage msg = myAgent.receive();
    if (msg != null) {
        // Process the message
    } else {
        block();
    }
}
```

Selective reading (slide 17). `receive()` returns the first message in the queue. If two or more behaviours receive messages, one may "steal" a message the other was interested in. To avoid this, read only messages with certain characteristics (e.g. whose sender is "Peter") by passing a `jade.lang.acl.MessageTemplate`:

```
MessageTemplate tpl = MessageTemplate.MatchOntology("Test-Ontology");

public void action() {
    ACLMessage msg = myAgent.receive(tpl);
    if (msg != null) {
        // Process the message
    } else {
        block();
    }
}
```

Blocking mode (slide 18). `Agent` also provides `blockingReceive()`, which returns only when there is a message in the queue; overloaded versions accept a `MessageTemplate` (wait for a matching message) and/or a timeout (if it expires, the

method returns `null`). Since it is a *blocking* call, it is "dangerous" inside a behaviour: no other behaviour can run until it returns.

Key result — Rule of thumb (slide 18)

- Use `receive()` + `Behaviour.block()` to receive messages **within behaviours**.
- Use `blockingReceive()` to receive messages within the agent's `setup()` and `takeDown()` methods.

Worked example — a weather station and a subscriber

[Not in the slides: added material]

The station sends the forecast to `Peter` with the code of slide 15 (with the ontology name shortened to `"Weather"`). Peter's agent:

```
1  public class PeterAgent extends Agent {
2      protected void setup() {
3          addBehaviour(new CyclicBehaviour(this) {
4              public void action() {
5                  MessageTemplate tpl = MessageTemplate.and(
6                      MessageTemplate.MatchPerformative(ACLMessage.INFORM),
7                      MessageTemplate.MatchOntology("Weather"));
8                  ACLMessage msg = myAgent.receive(tpl);
9                  if (msg != null) {
10                     System.out.println("Forecast: " + msg.getContent());
11                     ACLMessage reply = msg.createReply();
12                     reply.setPerformative(ACLMessage.AGREE);
13                     reply.setContent("thanks, taking an umbrella");
14                     myAgent.send(reply);
15                 } else {
16                     block();
17                 }
18             }
19         });
20     }
21 }
```

Timeline: Peter starts, the cyclic behaviour finds the queue empty and calls `block()` (no CPU used). The station's `INFORM` arrives: the behaviour is woken up, `receive` matches the template, the forecast is printed, a reply is sent (`createReply()` fills in receiver, conversation-id and in-reply-to). Then `action()` returns, is called again, finds the queue empty, blocks. Other messages (e.g. a `REQUEST`) stay in the queue for other behaviours.

10.3.3 The yellow pages service

[Slides · JADE: 19]

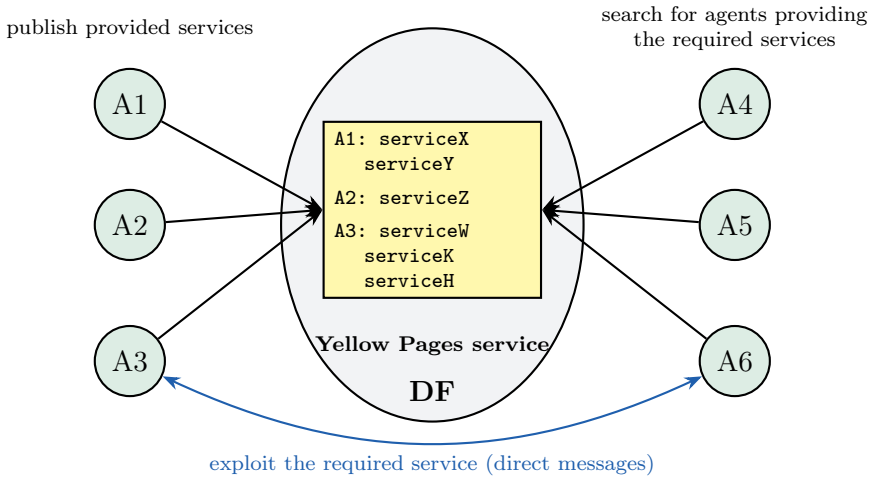


Figure 39: The yellow pages service (slide 19): providers register their services with the Directory Facilitator; consumers search the DF to discover providers, then talk to them directly.

Worked example — registering and searching (JADE API)

[Not in the slides: added material]

```
// Provider, in setup(): publish the service
DFAgentDescription dfd = new DFAgentDescription();
dfd.setName(getAID());
ServiceDescription sd = new ServiceDescription();
sd.setType("weather-forecast");
sd.setName("genoa-weather");
dfd.addServices(sd);
try { DFService.register(this, dfd); } catch (FIPAException e) {
    e.printStackTrace(); }

// Consumer: find all agents offering "weather-forecast"
DFAgentDescription template = new DFAgentDescription();
ServiceDescription wanted = new ServiceDescription();
wanted.setType("weather-forecast");
template.addServices(wanted);
DFAgentDescription[] result = DFService.search(this, template);  // may
    throw FIPAException
for (DFAgentDescription d : result) System.out.println(d.getName());
```

The provider should call `DFService.deregister(this)` in `takeDown()`. The DF is itself an agent: these methods exchange FIPA `request` messages with it under the hood.

Why this matters in practice

[Not in the slides: added material]

Yellow pages give **loose coupling**: the consumer does not need to know the names of the providers at compile time, and providers can join or leave an *open* system at any time. The same idea is service discovery in microservice architectures (a service registry) and, combined with Contract Net, gives dynamic task allocation: search the DF for all agents offering a service, send them a `cfp`, accept the best `propose`.

Cheat sheet — JADE

- JADE = Java Agent DEvelopment framework: FIPA-compliant middleware + tools; a platform can span several machines (different OSes), with agents moving at run time.
- Agent type = subclass of `jade.core.Agent` redefining `setup()` (initialisation) and optionally `takeDown()` (clean-up); `doDelete()` terminates the agent; `getArguments()` reads start-up arguments.
- Every agent has an **AID**: a globally unique name `<local-name>@<platform-name>` plus transport addresses (used only between different platforms).
- Work is done in **behaviours**: subclasses of `jade.core.behaviours.Behaviour` with `action()` and `done()`, added with `addBehaviour()`. Scheduling is **cooperative**, in a single Java thread: a switch happens only when `action()` returns.
- Communication: asynchronous message passing of `ACLMessage` objects; each agent has a private message queue. `send(msg)`; `receive()` returns a message or `null`.
- Pattern: `msg = myAgent.receive(); if (msg != null) {...} else block();` (`block()` is not blocking: it parks the behaviour until a new message arrives). `MessageTemplate` for selective reading. `blockingReceive()` only in `setup()`/`takeDown()`.
- **Yellow pages** (DF): agents publish services; others search them and then interact directly.

11 Agent-based modelling and simulation